

Trabajo Fin de Grado Grado en Ingeniería Electrónica, Robótica y Mecatrónica

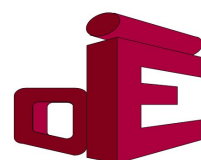
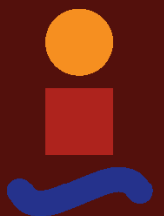
Diseño y realización de un prototipo de gaita electrónica

Autor: César Núñez Bayón

Tutor: Manuel Ángel Perales Estéve

**Dpto. Ingeniería Electrónica
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla**

Sevilla, 2024



Trabajo Fin de Grado
Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Diseño y realización de un prototipo de gaita electrónica

Autor:
César Núñez Bayón

Tutor:
Manuel Ángel Perales Estéve
Profesor Titular

Dpto. Ingeniería Electrónica
Escuela Técnica Superior de Ingeniería
Universidad de Sevilla

Sevilla, 2024

Trabajo Fin de Grado: Diseño y realización de un prototipo de gaita electrónica

Autor: César Núñez Bayón

Tutor: Manuel Ángel Perales Estéve

El tribunal nombrado para juzgar el trabajo arriba indicado, compuesto por los siguientes profesores:

Presidente:

Vocal/es:

Secretario:

acuerdan otorgarle la calificación de:

El Secretario del Tribunal

Fecha:

Agradecimientos

Agradecer a mi familia y amigos, que me acompañaron en esta etapa de mi vida, y celebraron mis éxitos y me animaron a perseverar. Agradecer a mi tutor del TFG por ayudarme durante el desarrollo de este proyecto. Agradecer también a mi tío Ricardo por introducirme en el mundo de la gaita e inspirar este trabajo.

César Núñez Bayón
Sevilla, 2024

Resumen

Este trabajo trata sobre el diseño de un instrumento electrónico de viento o controlador MIDI mediante un microcontrolador, así como del proceso de construcción del mismo. La plataforma de microcontroladores utilizada será Arduino. El instrumento que se imitará es la gaita gallega. El objetivo del proyecto es crear una herramienta con un comportamiento lo más cercano posible al instrumento real, para permitir que las personas interesadas en el instrumento puedan aprender y acercarse al instrumento sin tener que invertir mucho dinero en una gaita tradicional.

Abstract

This project deals with the design of an electronic wind instrument or MIDI controller using a microcontroller, as well as its construction process. The microcontroller platform used will be Arduino. The instrument that will be imitated is the galician bagpipe. The aim of this project is to create a tool with behavior as close as possible to the real instrument, to allow people interested in the instrument to learn to play it without having to invest a lot of money in a traditional bagpipe.

Índice

<i>Resumen</i>	III
<i>Abstract</i>	V
1 Introducción al trabajo	1
1.1 Motivación detrás del trabajo	1
1.2 Objetivo final	1
1.3 Fases de desarrollo	2
2 La gaita: información general	3
2.1 Breve historia de la gaita	3
2.2 Construcción de una gaita	3
2.3 Características musicales de la gaita	4
3 Estado del arte	7
3.1 Instrumentos de viento MIDI	7
3.1.1 Controlador de viento Akai EWI (Electronic Wind Instrument)	7
EWI 5000	7
EWI Solo	7
3.1.2 Yamaha WX5 (Wind MIDI Controller)	7
3.1.3 Roland Aerophone series	8
3.2 Gaitas electrónicas	9
3.2.1 Punteiro electrónico del obradoiro de gaitas Eduardo Represas	9
3.2.2 Punteiro electrónico TR de Arcolris musical	10
3.2.3 Hevia Electronic Bagpipes	10
MBS-300	10
MBS-250	11
3.2.4 Mastergaita	11
3.3 Conclusiones de este capítulo	11
4 Electrónica del proyecto	13
4.1 Microcontrolador	13
4.1.1 Microcontrolador Arduino UNO	14
4.2 Sensorización	15
4.2.1 Sensores táctiles	16
Sensores capacitivos: principio de funcionamiento	16
Sensor TTP223	16

	Sensores capacitivos caseros	18
	Módulo MPR121 Breakout V12	19
4.2.2	Sensores de presión	20
	Sensor de presión absoluta HX710B	20
4.3	Otros componentes electrónicos	21
4.3.1	Botones	21
4.3.2	Encoder digital	22
4.3.3	Pantalla LCD	22
4.3.4	Conector MIDI hembra	22
4.3.5	Cableado	22
4.3.6	Protoboard y placas PCB	23
4.3.7	Resistencias	23
4.3.8	Impresora 3d	24
5	Software del proyecto	25
5.1	Arduino IDE	25
5.2	Protocolo MIDI y conexión MIDI del instrumento	26
5.2.1	Protocolo MIDI	26
	MIDI note ON y note OFF	26
	MIDI Control Change	28
5.2.2	LoopMIDI	29
5.2.3	Hairless MIDI Serial	30
5.3	Sintetizador	30
5.3.1	FL Studio	30
	SIM-BP	31
5.3.2	Universal Piper	32
5.4	Modelado e impresión 3d	34
5.4.1	FreeCAD	36
5.4.2	Prusa	38
5.5	Dibujado de esquemáticos y planos	38
5.6	Librerías de Arduino	40
5.6.1	Librería MIDI.h	40
5.6.2	Librería Q2HX711.h	41
5.6.3	Librería Wire.h	41
5.6.4	Librería Adafruit MPR121.h	41
5.6.5	Librería LiquidCrystal.h	41
5.6.6	Librería Adafruit Debounce.h	42
6	Desarrollo del trabajo	45
6.1	Prototipo inicial básico	45
6.2	Prueba de los sensores de presión	46
6.3	Prueba del encoder digital	48
6.4	Prototipo 1 final	51
6.5	Prueba de sensores capacitivos caseros	53
6.6	Prueba del módulo de sensores capacitivos MPR121	53
6.7	Prueba del sensor de presión conectado a bolsa de aire	53
6.8	Prueba de la pantalla	54

7	Gaita electrónica MIDI. Resultado final	55
7.1	Conexiones eléctricas y soldaduras	55
7.2	Modelado e impresión de piezas	59
7.2.1	Punteiro	59
7.2.2	Tuerca	61
7.2.3	Caja	61
7.2.4	Soportes de la pantalla, los botones y el fol	62
7.2.5	Soprete	63
7.2.6	Bolsa de aire o fol	63
7.3	Aspecto final del instrumento	64
7.4	Código del instrumento final y funcionamiento	65
7.4.1	Funcionamiento del instrumento final	65
7.4.2	Tabla de funciones MIDI	67
7.4.3	Explicación del código	67
8	Presupuesto	71
9	Conclusiones del proyecto y posibles mejoras	73
Apéndice A	Códigos de Arduino	75
A.1	Código del prototipo inicial	75
A.2	Código de prueba de los sensores de presión	80
A.3	Código para probar el encoder y controlar el volumen:	80
A.4	Código final del primer prototipo	82
A.5	Código para probar los sensores capacitivos caseros:	88
A.6	Código para probar el módulo MPR121:	89
A.7	Código para probar el sensor de presión del fol:	91
A.8	Código para probar cuatro pulsadores:	92
A.9	Código de prueba del sensor de presión para el fol	92
A.10	Código para probar la pantalla LCD:	93
A.11	Código del instrumento final	95
Apéndice B	Esquemático de conexión completa del instrumento	115
Apéndice C	Planos de las piezas del instrumento	117
	<i>Índice de Figuras</i>	123
	<i>Índice de Tablas</i>	125
	<i>Índice de Códigos</i>	127
	<i>Bibliografía</i>	129

1 Introducción al trabajo

1.1 Motivación detrás del trabajo

Hace ya unos años el folclore gallego despertó un interés en mi vida. Gracias a mi familia por parte paterna, de origen gallego, he podido establecer contacto con la cultura de esta región. Además desde hace unos siete años he estado aprendiendo a tocar la gaita gallega, de ahí mi interés por el instrumento. Sin embargo, durante este proceso de aprendizaje he podido comprobar las dificultades que acompañan a este bonito instrumento. Algunas de estas limitaciones son:

- La falta de materiales de aprendizaje. Debido a que este instrumento es propio de una región muy concreta y su uso y práctica está poco extendida, la cantidad de material de referencia es escaso. Partituras, software educativo, guías de cuidados de la gaita, en general son difíciles de encontrar.
- La inflexibilidad del instrumento, reflejada principalmente en el hecho de que para practicar debes salir del núcleo urbano si no quieres arriesgarte a una denuncia por ruido. También la necesidad de adquirir componentes nuevos para variar la afinación, además de la relación entre los cambios de humedad de la madera y la afinación.
- La inversión inicial necesaria para adquirir un instrumento con el que practicar. En mi caso he tenido la suerte de que mi tío tiene una gaita para prestarme, pero para cualquiera que no comparta una situación parecida, la barrera de entrada es muy alta. La gaita más barata que se puede encontrar por internet superará seguramente los 300 euros, incluso de segunda mano.

Con todo esto, considero que el desarrollo de una herramienta electrónica de aprendizaje para este instrumento es necesario, y favorecería la difusión de la cultura gallega y de este instrumento tan bonito y característico. Además con mis conocimientos de electrónica adquiridos en el grado, diseñar este instrumento sería una forma interesante de poner en práctica lo aprendido.

1.2 Objetivo final

El objetivo de este trabajo, como ya se ha indicado anteriormente, es el desarrollo y realización del prototipo de una gaita gallega electrónica, que imite de la mejor forma posible el funcionamiento del instrumento real. Ofrecerá también las ventajas unidas a la realización electrónica. Concretando, esto significa:

- La capacidad de variar la afinación del instrumento, de forma rápida mediante la variación de parámetros del sintetizador, y opcionalmente físicos del instrumento como potenciómetros y botones. También se dispone de una pantalla LCD para servir de interfaz con el usuario.

- La capacidad de utilizar cascos para practicar y la flexibilidad de poder practicar en cualquier lugar, así como una salida que permita la conexión con sintetizadores MIDI y con ordenadores, lo que le permite agregar más funciones de las disponibles con el instrumento real.
- Comportamiento cercano al instrumento real, midiendo la presión del aire del instrumento y ofreciendo una construcción del mismo cómoda o por lo menos parecida al instrumento real.
- Independencia del instrumento de las condiciones de humedad, así como el menor mantenimiento del mismo en comparación con el instrumento real.
- Menor coste del instrumento electrónico.

1.3 Fases de desarrollo

El desarrollo del instrumento final va a constar de dos escalones de dificultad ascendente. En primer lugar se intentará desarrollar un instrumento que no necesite de soplido ni medición de presión. En segundo lugar se intentará añadir la funcionalidad de la medición de la presión para modelar el efecto de la misma como en una gaita real.

De forma orientativa, las fases de desarrollo serán las siguientes:

- Adquirir información sobre el instrumento y sus características musicales. Estudiar las diferentes opciones ya disponibles de instrumentos parecidos, buscando implementar alguna función que difiera de lo que ya han hecho otros fabricantes.
- Buscar la sensorización adecuada para implementar las funciones anteriores, así como el microcontrolador adecuado. Adquirir los componentes elegidos. Buscar también algún sintetizador apropiado y el software necesario para el funcionamiento del instrumento.
- Empezar el desarrollo de las funciones, probando los distintos sensores. Se irán agregando funciones de una en una hasta llegar al prototipo final.
- Construcción y montaje del instrumento. Diseño de piezas, dibujo de planos y montaje.
- Redacción de la memoria y preparación final del trabajo y la defensa.

En cuanto a la temporización de estas tareas, se tuvo en cuenta la fecha 1 de julio como fecha límite para la entrega del trabajo. De todas maneras, teniendo 3 meses para el desarrollo del proyecto, no se organizó el tiempo de manera muy estricta.

2 La gaita: información general

Para introducir este trabajo se va a empezar hablando brevemente de la historia de la gaita y de su principio de funcionamiento. El objetivo de esto es arrojar un poco de luz sobre el instrumento, entender cómo funciona para justificar la conversión electrónica posterior y definir las características musicales del instrumento.

2.1 Breve historia de la gaita

La gaita es un instrumento de viento cuyo origen histórico es incierto. Se cree que ya fue utilizado por los pueblos babilonios, hebreos, fenicios y celtas. Debido a la falta de información documental de tanta antigüedad, se desconoce su origen exacto. En cualquier caso se han descubierto pinturas egipcias en las que se representaba un instrumento muy parecido a la gaita. Los griegos ya conocían la gaita (askaulos), y los romanos (tibia utricularis), siendo conocida porque el emperador Nerón la tocaba.

En la Baja Edad Media su popularidad se extendió por toda Europa, pasando a formar parte del folclore de regiones muy distantes dentro del continente. Algunas de estas zonas que todavía conservan este instrumento dentro de la música tradicional son Italia, Hungría, Suecia, Suiza, India, Pakistán, o las más conocidas, Irlanda, Gran Bretaña, Francia y el norte de España.

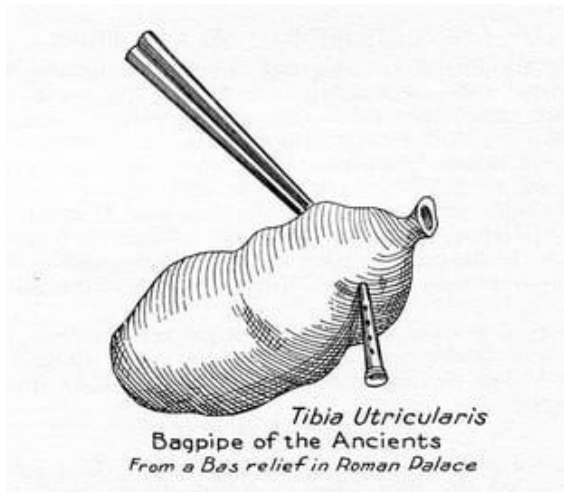
En cualquier caso, el objetivo de este trabajo no es averiguar el origen histórico del instrumento ni analizar su importancia en el folclore.

2.2 Construcción de una gaita

Gracias a la amplia extensión de este instrumento por el mundo, existen gran variedad de configuraciones. Las llamadas uilleann pipes o gaitas irlandesas se accionan mediante una especie de fuelle atado al brazo del músico, evitando así el tener que soplar. Por el contrario en la familia de gaitas más parecidas a las escocesas, el músico debe inflar una bolsa mediante su soplido, y forzar la salida de aire por los tubos de salida presionando con el brazo sobre la bolsa. Las gaitas del norte de España suelen ser de este tipo.

En este caso el trabajo se centrará en la gaita gallega, si bien el resto de gaitas de la península son muy parecidas. Vamos a explicar las partes de la gaita gallega y sus funciones, de cara a tener una idea de como imitar su funcionamiento de forma electrónica. Una gaita gallega esta compuesta por:

- Soprete: es el tubo por el que se introduce el aire dentro de la bolsa. Debe estar en una posición que resulte cómoda para el intérprete. Contiene un sistema, normalmente muy sencillo, con una válvula antirretorno que permite la entrada de aire pero no su salida de la bolsa.



(a) Gaita romana o tibia utricularis.



(b) Gaita medieval.

Figura 2.1 Diferentes configuraciones de gaita en la antigüedad.

- Fol: así se denomina la bolsa en la que se almacena el aire. Típicamente estaba fabricado de la tripa de algún animal. Posteriormente se introdujo la goma, y en la actualidad suelen fabricarse a partir de materiales que eviten la acumulación de humedad como el goretex.
- Roncón, ronqueta, chillón: Son los tubos secundarios de la gaita o bordones. Están conectados al fol y emiten una nota continua llamada nota pedal mediante la vibración de una lengüeta llamada pallón. La nota que se emite da nombre a cada tubo, siendo el roncón el que emite una nota más grave y el chillón la más aguda.
- Punteiro: es la parte de la gaita que permite tocar la melodía. Es un tubo conectado al fol, normalmente de madera o algún material sintético, con una terminación cónica y que consta de nueve agujeros. De esos nueve, ocho son los que tapando con los dedos, nos permiten variar la longitud del tubo y así generar las diferentes notas.
- Palleta: es una lengüeta doble normalmente fabricada de caña, cuya vibración es la que produce el sonido de la gaita. Se encuentra en el principio del punteiro y a través de ella pasa el aire que sale por el mismo.
- Pallón: es la lengüeta simple que al vibrar dentro de los tubos secundarios (roncón, ronqueta y chillón) genera una nota continua.
- Elementos cosméticos: a parte de los elementos que influyen en el sonido, las gaitas también constan de elementos cosméticos como el vestido del fol, las borlas, farrapos, frecos etc.

2.3 Características musicales de la gaita

La gaita es un instrumento de viento o aerófono. Es también polifónico, puesto que emite varios sonidos a la vez, gracias a que tiene varios tubos que emiten sonido.

La afinación de la gaita puede ser muy variada. Las afinaciones más comunes suelen ser en DO, Sib y RE, siendo la más utilizada en DO. El rango de notas que podemos tocar suele ser de una octava y dos o tres notas más. Se debe tener en cuenta, que para tocar las notas de la octava superior se debe ejercer una presión mayor en el fol.

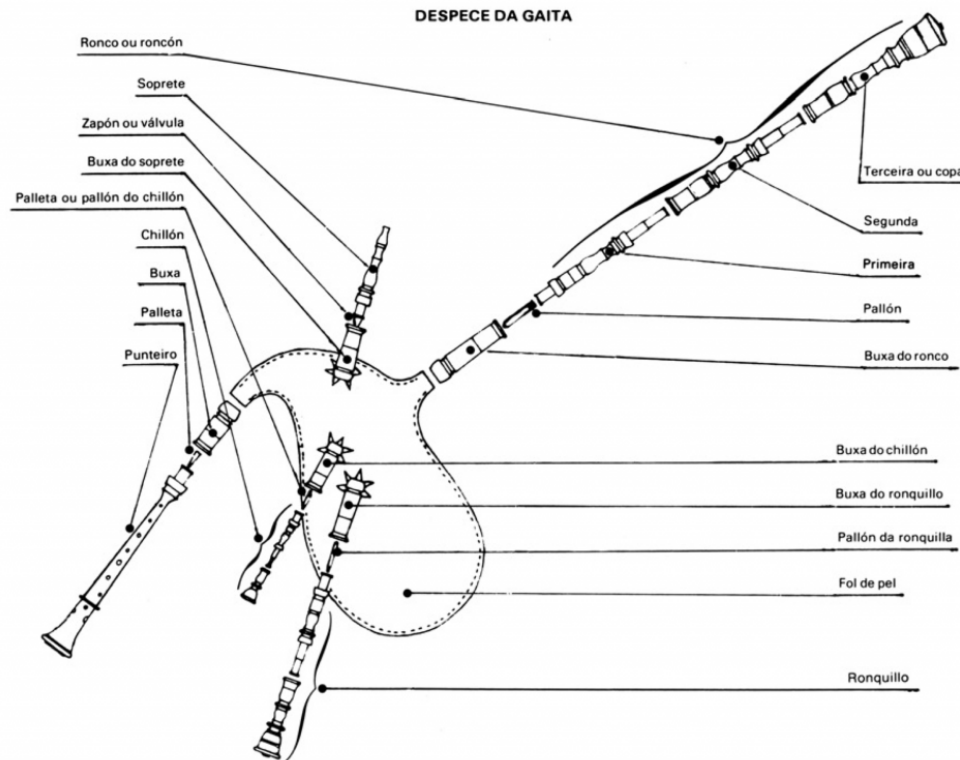


Figura 2.2 Despiece de una gaita gallega.

La polifonía proviene de los tubos secundarios de la gaita, que tocan notas pedales continuas. Dependiendo de la afinación de la gaita, la nota pedal será la misma que la nota en que está afinada, también llamada nota tónica, pero en otra octava.

En el ejemplo más común se tendría una gaita con el puntero afinado en DO con la siguiente digitación:

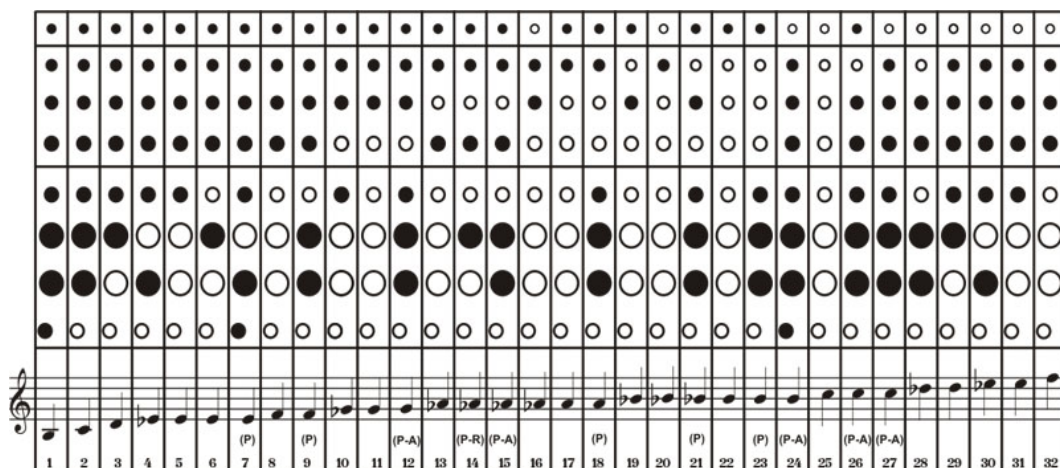


Figura 2.3 Digitación de gaita en DO.

Además en este ejemplo el roncón daría un DO dos octavas por debajo del DO indicado en la posición 2, que sería la tónica. Opcionalmente también se puede incluir una ronqueta que tocaría

un DO una octava por debajo de la tónica que se mencionó anteriormente.

La posición de los dedos en el punteiro seguiría el siguiente esquema. Hay que tener en cuenta que, como en otros instrumentos de viento, se pueden realizar glissandos al desplazar un dedo suavemente desde la posición tapada completamente. Además se pueden obtener notas intermedias tapando parcialmente los agujeros.



Figura 2.4 Digitación de los agujeros de la gaita.

Otro aspecto importante del funcionamiento del instrumento son las ligaduras. En el caso de la gaita, el sonido es continuo. Esto significa que a diferencia de una flauta donde mediante el soplido puedes articular, en una gaita no se tiene esa posibilidad. Esto obliga a separar las notas mediante lo que se llaman picados y batementos.

- Un picado consiste en separar las notas al levantar un dedo rápidamente que esté en una posición superior, dando una nota más aguda.
- Un batemento se realiza para separar las notas tapando rápidamente el agujero inmediatamente inferior a la nota que se quiere separar, dando una nota más grave.

Por otro lado, el tempero consiste en el control sobre el aire que hace vibrar la lengüeta. Temperar correctamente la gaita consiste en conseguir que mediante la presión ejercida con el brazo sumado a la presión al rellenar de aire el fol, se consiga una salida de aire constante. Si no se temperara correctamente se escucharía una cierta variación en el tono, como una especie de variación en el pitch. La diferencia entre un gaitero profesional y uno principiante es precisamente el control sobre la presión. Cuanto mayor control se tenga, mayor consistencia tendrá el sonido y el tono de las notas. Es quizá uno de los problemas de la gaita, la dificultad para obtener un tono exacto y que no desafine con el resto de instrumentos.

Definir estas características es suficiente para entender que requisitos debe cumplir el instrumento electrónico para ser una imitación fiel.

3 Estado del arte

Los instrumentos de viento electrónicos son quizá los menos conocidos. Aun así, existen gran cantidad de modelos de fabricantes muy populares, que ofrecen diferentes características en función de su precio. En este capítulo se encontrará el problema de que los instrumentos mencionados son productos comerciales, y por tanto no será fácil encontrar especificaciones electrónicas muy profundas. Esto es porque las empresas que venden estos productos no quieren, como es lógico, desvelar como fabrican estos instrumentos.

A continuación se mencionan diversos ejemplos de entre los más conocidos.

3.1 Instrumentos de viento MIDI

En esta sección se nombrarán de forma breve las características de algunos instrumentos de viento MIDI o controladores MIDI de las marcas más conocidas, indicando su precio y especificaciones técnicas.

3.1.1 Controlador de viento Akai EWI (Electronic Wind Instrument)

Akai Professional Musical Instrument Corporation es una empresa japonesa dedicada a la fabricación de instrumentos electrónicos y sintetizadores. Fue de las primeras empresas en empezar a fabricar instrumentos de viento MIDI, y ha alcanzado gran reconocimiento gracias a sus productos de alta calidad. La serie EWI de instrumentos de viento electrónicos incluye algunos modelos como:

EWI 5000

Instrumento de viento sin cables, con una batería de litio recargable que permite tocar con libertad. Permite conexiones MIDI y remotas. Se puede configurar de forma que los botones imiten a flauta, oboe y saxofón, así como otros instrumentos de viento metal. Incluye también una librería de sonidos de 3GB y almacenamiento interno para cargar otros sonidos. Su precio ronda los 530 euros.

EWI Solo

Este modelo tiene unas características parecidas al anterior, solo que además incluye un amplificador incorporado para poder tocar sin necesidad de conectarlo. Puede también utilizar sonidos personalizados, a parte de los diferentes instrumentos que ya vienen incluidos. Su precio ronda los 400 euros.

3.1.2 Yamaha WX5 (Wind MIDI Controller)

Controlador MIDI para instrumentos de viento. Permite diferentes configuraciones de los botones, tiene salida MIDI para conectarlo a un sintetizador, sensores de presión de labios e intensidad de soplo de alta resolución. Además incluye una serie de controles que permiten ajustar la ganancia de



(a) AKAI EWI5000.

(b) AKAI EWI Solo.

Figura 3.1 Modelos de controlador electrónico AKAI.

los sensores, cambiar la octava en la que se toca, modificar el pitch mediante una rueda como en un teclado, etc. Su precio de segunda mano oscila entre los 200 y los 500 euros.

**Figura 3.2** Modelo WX5 de Yamaha.

3.1.3 Roland Aerophone series

Presenta una digitación ajustable a diferentes instrumentos, sensores de soplido y mordida y entrada MIDI para conmutadores de pedal. Permite ajustar la afinación mediante una aplicación de móvil, además de tener su propio generador de sonido y mas de 300 presets de diferentes instrumentos. Conexión bluetooth, salida MIDI, USB y entrada y salida de jack estéreo. Su precio es el más elevado, costando más de 800 euros dependiendo del modelo.

**Figura 3.3** Modelo Roland Aerophone Pro AE-30.

Vemos que existen una gran cantidad de productos que ofrecen una amplia variedad de características y especificaciones diferentes. Sin embargo, el precio de los productos originales es bastante elevado, y en general no están pensados para principiantes sino para músicos profesionales, además de no tener configuraciones cercanas a la gaita.

3.2 Gaitas electrónicas

A continuación se van a investigar instrumentos MIDI electrónicos que intenten imitar concretamente la configuración de una gaita y su sonido. Cabe destacar que existen bastantes más productos comerciales de los esperables al comienzo de este proyecto.

Antes de mencionar los diferentes modelos y fabricantes, se debe mencionar una breve introducción a la gaita electrónica. La primera gaita electrónica fue patentada por Alberto Arias, Miguel Dopico y el famoso gaitero asturiano Hevia. Este último popularizó su uso, utilizándola para grabar sus discos que llegaron a vender más de 600.000 copias en España. Sin embargo, la aparición de las primeras gaitas electrónicas se remonta a los años 70, con la Bazpipe creada por el tejano Bazell Ray Cowan. A partir de ese momento, otros artistas e ingenieros comenzaron a desarrollar sus propios prototipos de los diferentes tipos de gaitas, como la gaita irlandesa desarrollada por George H. Boyd.

La invención y popularidad de la gaita electrónica trajo consigo cierto debate. Al ser un instrumento tradicional fabricado por artesanos, muchos gaiteros profesionales consideran que las electrónicas no mantienen la esencia del instrumento y suponen un peligro para la cultura asociada al instrumento.

A continuación vamos a ver algunos de los modelos más populares de gaita electrónica y que especificaciones presentan:



Figura 3.4 José Ángel Hevia utilizando una gaita electrónica.

3.2.1 Punteiro electrónico del obradoiro de gaitas Eduardo Represas

La información sobre este modelo y sus especificaciones electrónicas es bastante escasa. La primera configuración que vamos a ver es la del punteiro. Esto significa que el instrumento resultante consiste en las partes de una gaita analógica normal excepto por el punteiro, que sería lo único electrónico. Las ventajas de esto son la estética del instrumento que se conserva, y una mayor flexibilidad en la afinación.

Este modelo ofrece cuatro afinaciones diferentes, una batería que permite tocar sin estar enchufado a la corriente, y la posibilidad de generar el sonido sin necesidad de conectarlo a un amplificador ni sintetizador, además de poder conectar cascos. Su precio ronda los 300 euros, pero no incluye el resto de componentes de la gaita.



Figura 3.5 Punteiro electrónico de Represas.

3.2.2 Punteiro electrónico TR de Arcolris musical

Siguiendo la misma configuración que el producto anterior tenemos este modelo. De nuevo consiste en únicamente el punteiro, que se debe conectar a una gaita normal. Sus características son muy similares al anterior, aunque incluye diferentes sonidos de gaita, no solo gallega. Permite funcionar con aire o sin aire, soplando o conectándolo al fol de la gaita. Incluye de nuevo una batería, salida para auriculares o amplificador, módulo de sonido interno y una memoria programable para guardar ciertos parámetros. Al igual que en el modelo de Represas, la información disponible es bastante escueta. Su precio es de 400 euros.

3.2.3 Hevia Electronic Bagpipes

Como se dijo en la introducción, dentro de España y siendo los primeros en patentar este instrumento, Hevia Electronic Bagpipes son pioneros en esta idea de la gaita electrónica. Ofrecen dos productos diferentes, la MBS-300 y la MBS-250.

MBS-300

Este modelo es el más avanzado y profesional. Ofrece características propias de un instrumento de concierto profesional. Imita fielmente el sonido y comportamiento de una gaita real, permitiendo glissandos y vibratos, pudiendo variar la afinación y la sensibilidad de los sensores de presión de aire. Permite tocar soplando o sin soplar, conexión bluetooth pensada para escenarios e incluye su propia biblioteca de sonidos. Su precio llega a los 2000 euros.



Figura 3.6 Modelos de Hevia Electronic Bagpipes.

MBS-250

Este modelo es estéticamente igual al anterior pero no ofrece el mismo rango de personalización. No permite conexión bluetooth, solo MIDI o mediante USB.

3.2.4 Mastergaita

Este producto fue desarrollado por José J.P Presedo en 2002. Su fabricación es de momento artesanal, y es bastante parecido al objetivo de este trabajo tanto en la motivación detrás de su creación como en lo que ofrece como instrumento.

Es un controlador MIDI que se conecta mediante USB a un ordenador o smartphone. Requiere de un programa sintetizador en el dispositivo al que se conecta, con lo cual es un producto mucho menos profesional que los anteriormente mencionados. Además no ofrece la posibilidad de controlar la presión del aire, puesto que no tiene ningún sensor de presión.



Figura 3.7 Gaita electrónica Mastergaita.

Su precio de segunda mano es de unos 150 euros. Nueva se puede adquirir por menos de 300.

3.3 Conclusiones de este capítulo

Se puede ver que existen muchas posibilidades dentro del mundo de los instrumentos de viento electrónicos. Desde marcas profesionales y reconocidas hasta pequeños artesanos, todos ofrecen diferentes enfoques sobre como abordar la fabricación de estos instrumentos.

Si bien la idea del trabajo es relativamente nueva y poco estudiada, la realidad es que ya existen modelos de gaitas electrónicas a la venta. Entonces, ¿que espacio queda para lo desarrollado en este trabajo?. La idea de este proyecto es ofrecer un punto intermedio entre las gaitas profesionales como las de Heiva y los instrumentos más económicos como la Mastergaita, que no buscan la imitación de las mecánicas propias del instrumento analógico, obviando aspectos como el control de la presión del aire o el soplido.

4 Electrónica del proyecto

En este capítulo del proyecto se van a explicar los aspectos electrónicos y constructivos del instrumento. Se hará un repaso a la plataforma de microcontroladores utilizada, así como a los sensores necesarios para la implementación de las especificaciones definidas anteriormente.

Recordemos que el objetivo final es crear una herramienta de práctica e iniciación a la gaita, buscando un funcionamiento lo más cercano posible al instrumento real.

4.1 Microcontrolador

Para el control del instrumento lo más sencillo es el uso de un microcontrolador. Las ventajas que ofrece su uso en el desarrollo de este proyecto son la flexibilidad que ofrecen, la facilidad del uso de sensores comerciales desarrollados específicamente para su uso en microcontroladores y su bajo costo en general (sobre todo debido a la gran cantidad de imitaciones de los micros oficiales). Su tamaño compacto favorece su montaje en diferentes configuraciones, además de una programación fácil y la existencia de gran cantidad de material de apoyo de la mayoría de micros.

En un principio se plantearon varias opciones de distintos fabricantes. Algunas de las opciones fueron un microcontrolador de Texas Instruments de la familia MSP430. Se planteó utilizar la placa MSP EXP430G2ET por ofrecer las características necesarias para implementar los sensores y el control de este proyecto, así como que su uso resulta familiar debido a asignaturas del grado en que se utilizó esta plataforma. Sin embargo finalmente no se optó por esta opción debido a que otros microcontroladores ofrecían una mayor facilidad para su programación.

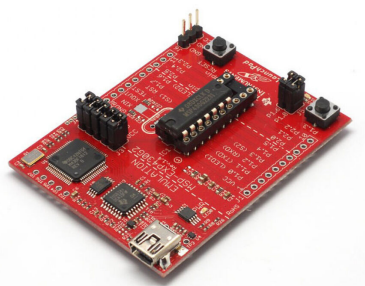


Figura 4.1 Microcontrolador MSP430G2.

Otra opción que se barajó fue algún microcontrolador de la marca Teensy. La mayor ventaja que ofrece es su tamaño compacto para las capacidades que ofrece. La placa Teensy 4.1, siendo extremadamente compacta, ofrece 55 pines digitales, 35 de ellos con soporte para PWM, 18 pines analógicos, 8 seriales, 3 SPI, 3 puertos I2C, etc. En este caso las características de la placa son quizá innecesariamente potentes para el desarrollo de este proyecto.



Figura 4.2 Microcontrolador Teensy 4.1.

Finalmente se optó por utilizar la plataforma Arduino. Las ventajas de esta gama de microcontroladores son principalmente la facilidad de su uso y programación, habiendo sido tratadas también en algunas asignaturas del grado. Además gracias a su uso más extendido que el de otros microcontroladores, existe una gran cantidad de material de apoyo y de módulos que agregan funcionalidades necesarias para este proyecto. Otra ventaja que tiene menor importancia quizá, es que yo ya tenía un Arduino UNO, lo que me permitió ponerme a trabajar prontamente. A continuación se describirán las especificaciones del microcontralador Arduino UNO utilizado.

4.1.1 Microcontrolador Arduino UNO

Este microcontrolador de la familia de Arduino es seguramente el más usado y el más popular de las placas de esta plataforma. Ofrece una serie de características que lo hacen apropiado para el desarrollo de este proyecto.

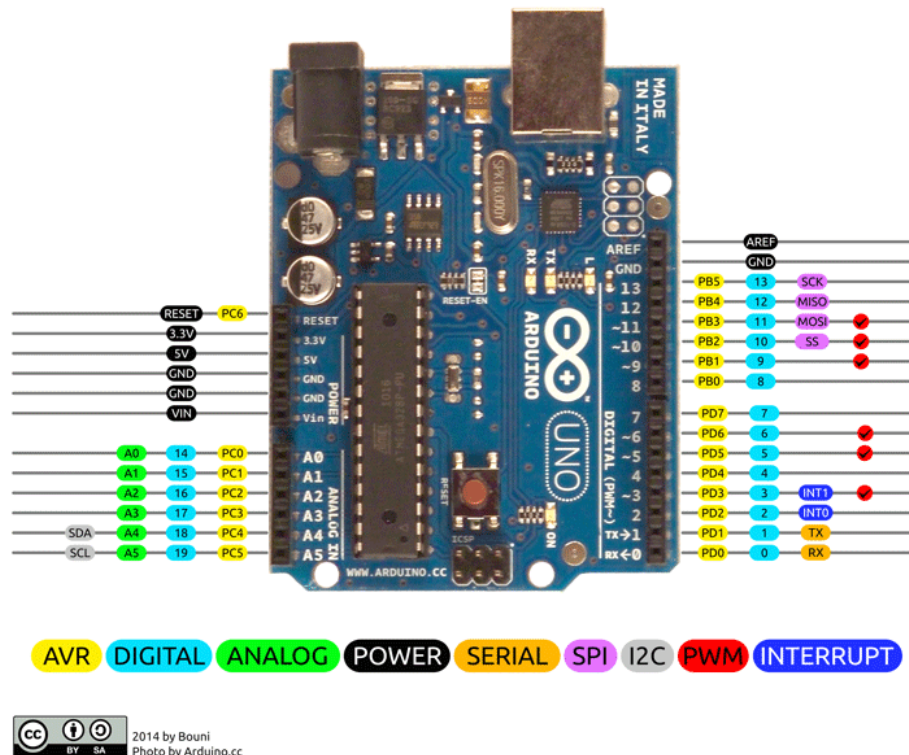


Figura 4.3 Mapa de pines del Arduino UNO.

Las características que ofrece, relacionadas con las necesidades del instrumento a diseñar son las siguientes:

- Tamaño compacto: su pequeño tamaño en comparación con otras placas como el Arduino MEGA favorece un montaje cómodo, sobre todo en algo como un instrumento musical cuya apariencia debe ser parecida a la del instrumento real.

- Número de pines ANALOG: este es de los aspectos más importantes y que más se han tenido en cuenta a la hora de elegir la placa y los sensores. El Arduino UNO consta de 6 pines analógicos, los justos para el montaje deseado. A continuación se explican los diferentes sensores y como se conectan a la placa, pero adelanto que el módulo de los sensores capacitivos utiliza 2 de estos pines para la comunicación I2C. Los 2 sensores de presión utilizados se conectan cada uno a 2 pines analógicos, haciendo el total de los 6 pines ANALOG IN ocupados.
- Número de pines DIGITAL: el Arduino UNO consta de 11 pines digitales que pueden ser configurados como entradas o salidas. En principio con el montaje final con el módulo MPR121, el número de pines resulta suficiente. En el caso de haber utilizado otra configuración para los sensores capacitivos, si podrían haber resultado escasos. La pantalla LCD utiliza 6 de estos pines, el encoder digital utiliza 2, y opcionalmente debería haber espacio para al menos 4 botones adicionales destinados a otras funciones.
- Posibilidad de alimentación mediante batería utilizando el conector barrel plug, permitiendo utilizar la placa sin necesidad de conectarla a un ordenador. Hay que tener en cuenta que aunque no se conecte a un ordenador, el instrumento es solo un controlador MIDI, y necesita ser conectado a un sintetizador, ya sea un sintetizador MIDI virtual en un ordenador mediante el conector USB, o a otros sintetizadores mediante una salida MIDI.
- Memoria del Arduino UNO: 32KB de memoria Flash, 2KB de SRAM y 1KB de EEPROM. Esta cantidad de memoria ha sido suficiente para el desarrollo del proyecto y el almacenamiento de todas las variables.

```
El Sketch usa 10382 bytes (32%) del espacio de almacenamiento de programa. El máximo es 32256 bytes.
Las variables Globales usan 697 bytes (34%) de la memoria dinámica, dejando 1351 bytes para las variables locales. El máximo es 2048 bytes.
```

Figura 4.4 Indicador del Arduino IDE de cantidad de memoria utilizada.

- Capacidad de programación de interrupciones. El Arduino UNO permite programar interrupciones en algunos de sus pines. En nuestro caso en la placa se tienen 2 pines digitales que permiten interrupciones (pines 2 y 3 digitales I/O). Ambos se han utilizado para el uso del encoder digital. Si bien no habría estado mal realizar otras funciones mediante interrupciones, en general tampoco es necesario.
- Comunicación mediante UART, I2C y SPI. La comunicación mediante UART se utiliza para enviar los datos de salida de la placa, esto es los comandos MIDI, y para operaciones de testeo de los sensores utilizando el puerto serie. Adicionalmente se implementará una salida MIDI mediante un conector MIDI hembra que utilizará el pin D1(TX) para enviar los datos MIDI. La comunicación I2C se ha utilizado para la comunicación con el módulo MPR121 que controla los sensores capacitivos. La placa permite esta comunicación mediante los pines A4 (SDA) y A5 (SCK).

4.2 Sensorización

Es importante definir en este apartado algunas concreciones sobre el funcionamiento deseado del instrumento final para justificar el uso de los sensores.

En primer lugar, se pretende imitar el funcionamiento del fol en una gaita real. Esto es, por un lado la necesidad de soplar para inflar un fol "virtual", aunque no se esté usando el aire para inflar realmente nada. Por otro lado el interprete debe apretar con el brazo una bolsa de aire o algún objeto para forzar la salida de aire. Este objeto debe ofrecer una sensación parecida a la del fol de la gaita

real. En segundo lugar, la parte del punteiro, es decir donde se tocan las notas, debe ofrecer un tacto parecido al del instrumento real. Para ello se han considerado varias opciones que se expondrán a continuación.

4.2.1 Sensores táctiles

Como se ha mencionado antes, la parte del punteiro debe tener algún tipo de sensor que pueda ser instalado en un agujero de un tubo, para poder conseguir una sensación parecida a la de tapar un agujero con el dedo. Se deben usar al menos 8 sensores de este tipo para modelar los agujeros tocables de un punteiro.

En este sentido existen dos posibilidades para la sensorización: sensores capacitivos y pulsadores. En algún caso se ha visto que se usen sensores ópticos, sin embargo su precio es bastante mayor y su implementación más compleja y no se ha planteado su utilización.

La ventaja de los sensores capacitivos es que al ser táctiles, permiten una sensación más parecida a la deseada, a diferencia de los pulsadores. En cualquier caso, esto significa que se va a intentar utilizar sensores capacitivos frente a pulsadores, y estos últimos solo se utilizarán en el caso de que los capacitivos no ofrezcan una respuesta suficientemente estable y confiable.

Sensores capacitivos: principio de funcionamiento

Nos vamos a centrar en los sensores táctiles capacitivos, ya que el efecto del "capacitive coupling" o acoplamiento capacitivo se utiliza en gran cantidad de sensores. Principalmente estos detectan y miden cualquier cosa que sea conductora o que tenga una constante dieléctrica diferente de la del aire. Esto se usa en sensores táctiles, de proximidad, presión, fuerza, humedad o incluso aceleración.

Un sensor táctil capacitivo funciona mediante la detección de cambios en la capacitancia eléctrica en la superficie del sensor cuando se produce un contacto físico con un objeto conductor, como un dedo humano. La capacitancia es la capacidad de almacenamiento de carga eléctrica de un objeto, como ya sabemos. Cuando un objeto conductor, como un dedo, se acerca a la superficie del sensor, altera el campo eléctrico del sensor y provoca un cambio en la capacitancia.

La capacidad se mide típicamente de manera indirecta, usándola para variar la frecuencia de un oscilador o la atenuación de una señal alterna. Básicamente se carga la capacidad con una corriente conocida y usando las relaciones entre corriente, capacidad y voltaje de la siguiente forma se obtiene un valor de la capacidad parásita que aparece al tocar el sensor:

$$I(t) = C \frac{dV(t)}{dt}, \quad C = \frac{I(t)}{\frac{dV(t)}{dt}}, \quad C = \frac{\int_{t_0}^{t_1} I(t) dt}{V(t_1) - V(t_0)}.$$

En Arduino se pueden utilizar sensores capacitivos caseros gracias a la librería capacitive sensing. El principio de funcionamiento es definir un pin del arduino como el sender y otro como el receiver. El sender envía un pulso que atraviesa un circuito sencillo, y que es recibido de vuelta por el receiver. La librería calcula entonces mediante una serie de funciones el retraso que hay entre los dos pulsos y genera un valor que se puede utilizar para programar los umbrales de activación del sensor para que funcione como un botón. Esto se ha utilizado en el desarrollo del proyecto y se explicará más adelante en profundidad.

Sensor TTP223

En un primer momento planteé la utilización de los sensores capacitivos TTP223. Una de las ventajas de estos sensores es que son sensores digitales. Estos es que a pesar de ser sensores capacitivos, que es una característica inherentemente analógica, la salida del sensor es digital, conectándose por tanto a los pines digitales de la placa.

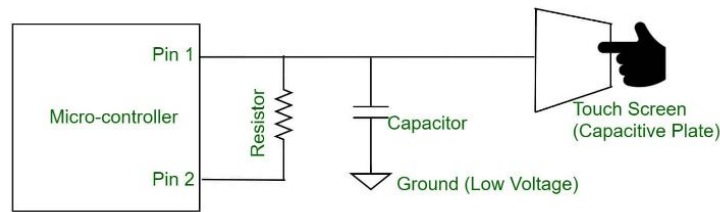


Figura 4.5 Funcionamiento de un sensor capacitivo casero.

No he sido capaz de encontrar el datasheet del fabricante concreto del modelo que se adquirió, sin embargo cualquier datasheet de este mismo sensor aunque sea de otro fabricante sirve para estudiar sus especificaciones.

El sensor se alimenta a un voltaje de entre 2 y 5.5 Voltios, pudiendo alimentarse en el arduino a 3.3V o 5V. La salida del sensor, es decir el pin I/O es digital como ya se ha comentado.

La sensibilidad del sensor se puede ajustar de diferentes maneras. Las primeras son cambiando la superficie de contacto, es decir el área del electrodo, y aumentando el grosor del panel de contacto. Por último también se puede cambiar la capacidad en la parte inferior del sensor. Según el fabricante, esta capacidad puede tener un valor de 0pF a 50pF para el ajuste. La velocidad de respuesta es de 60ms. Además la placa del sensor incluye un led rojo que se activa cuando se toca el sensor.

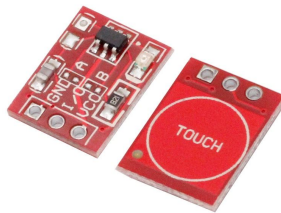


Figura 4.6 Sensor TTP223.

Después de realizar una serie de montajes de prueba, se encontraron una serie de dificultades de cara a utilizarlos. Estas dificultades llevaron a descartar el uso de estos sensores en el montaje final. En este video del prototipo1 se puede ver como se usaron los sensores y algunos de los problemas que generaban [Video del prototipo1](#).

Las desventajas principales fueron:

- Cada uno de estos sensores ocupa un pin digital del Arduino. Esto significa que con 8 sensores de este tipo estoy ocupando 8 de las 11 posibles I/O digitales del Arduino UNO, lo que descartaría el posible uso de una pantalla LCD por ejemplo en el montaje final.
- Estos sensores son bastante difíciles de ajustar. Como ya se ha comentado antes, para cambiar su sensibilidad se debe sustituir el pequeñísimo condensador en la parte inferior del sensor. Buscando información en internet se encontró algunos usuarios que soldaban los condensadores, pero es una operación bastante complicada debido a su pequeño tamaño.
- La necesidad de ajustar la sensibilidad del sensor viene de ciertos problemas que se encontraron. Uno de los montajes que se propusieron era utilizar un tubo de PVC con estos sensores montados dentro a poca distancia de los agujeros para detectar cuando se tapaban. Sin embargo el PVC provocaba la activación de los sensores, lo que dificultaba el montaje. Adicionalmente, tocar el tubo de PVC por los lados, es decir, sin tapar el agujero con el dedo sino por los laterales del tubo, el sensor se activa. En general es difícil aislar el sensor o

controlar su comportamiento. Dependiendo de como se montara la mayoría de materiales lo activaban, incluso cuando tocaban a la placa por su parte inferior.

- Aunque técnicamente estos sensores son táctiles, no es necesario llegar a tocarlos para que se activen en algunos casos. Esto provocaba que por la cercanía entre los sensores provocada por hacer cómoda la digitación, a veces se activara un sensor cuando el contiguo estaba siendo pulsado.
- Tocar el sensor con los dedos ligeramente húmedos provoca que después de retirar el dedo del sensor este siguiera activado. Además soplar o respirar encima del sensor o cerca también provocaba la activación.
- En ocasiones cuando el sensor estaba siendo pulsado, se desactivaba. Esto seguramente se deba a que este sensor esta pensado para pulsaciones cortas y no mantenidas. Revisando el datasheet se pudo ver un apartado llamado "Maximum key on duration time" que define un tiempo máximo en que el sensor puede estar activado de manera continuada, después del cual se desactiva. Este tiempo es de 100 segundos. Esta característica desde luego no favorece su uso para el proyecto.

Por estas razones y en general la dificultad de un montaje apropiado que permita un funcionamiento estable del instrumento, se descartó su uso en este proyecto. Tampoco se pretende desprestigiar la calidad de estos sensores, que son extremadamente baratos y fáciles de montar. Pero no están pensados para este tipo de aplicaciones.

Sensores capacitivos caseros

Como se ha explicado antes, en Arduino se pueden fabricar de forma sencilla sensores capacitivos caseros utilizando las funciones de la librería CapSense. Existen una gran cantidad de tutoriales y material que explican como realizar el circuito. El principio básico es que el Arduino es capaz de medir el retraso entre un pulso que emite por un pin digital y el pulso que recibe por otro y genera una medida de la capacidad parásita que produce ese retraso. Esa capacidad parásita ocurre al tocar un material conductor que puede ser desde una lámina de papel de aluminio hasta un tornillo o una chincheta.

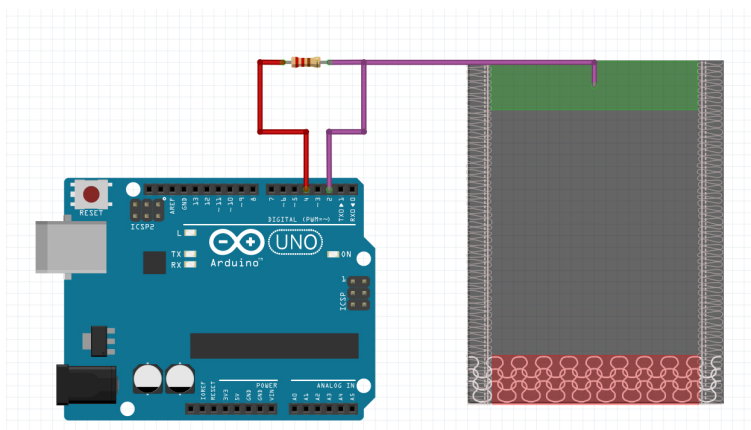


Figura 4.7 Realización de sensores capacitivos mediante librería CapSense.

En un punto del desarrollo de este proyecto se intentó replicar un montaje parecido para detectar cuando se tapaba un agujero. Después de replicar el circuito de la figura, se procedió a un montaje más complejo. El montaje consistía en un tubo de PVC con una tablilla de madera insertada en su interior por su sección central. En esta tabla se atornillaron unos tornillos que sobresalen por los

agujeros del tubo, dispuestos de la misma forma que en un punteiro. Por último estos tornillos se soldaron a un cable cada uno que va hasta la placa del Arduino.

La ventaja del uso de estos sensores y de la librería respecto de los TTP223 es que es mucho más configurable. Se pueden definir unos thresholds o umbrales para la activación de cada sensor a partir de observar las medidas de capacidad por el puerto serie del Arduino.

Los principales problemas son la necesidad de utilizar bastantes pines digitales del arduino, utilizando un pin por cada sensor además del pin del sender. De nuevo esto hace que el número de componentes que se pueden utilizar se vea reducido, sobre todo de cara a utilizar una pantalla LCD, o una serie de botones.

Además es necesario programar algún tipo de filtrado de la medida de la capacidad, ya que al observar las medidas devueltas por el Serial Plotter del Arduino IDE se ven oscilaciones de mucha amplitud. Esto imposibilita el uso de un umbral simple, dado que las oscilaciones de la medida de la capacidad cuando se pulsa el sensor y cuando no se está pulsando se solapan.

Con todo esto también se desechó esta opción de implementación de los sensores de esta forma, siendo sustituido por el uso del módulo MPR121, que ofrece muchas ventajas y mejoras que se expondrán a continuación.

Módulo MPR121 Breakout V12

Este módulo es un controlador de sensores capacitivos, que utiliza una interfaz I2C. El módulo consta de 12 pines para electrodos, con la opción de utilizar estos mismos pines para controlar leds. El módulo permite la implementación de sensores capacitivos en cualquier proyecto de electrónica, se conecta fácilmente a la mayoría de microcontroladores y junto con la librería de Adafruit para el MPR121 y la librería Wire para el protocolo I2C, hacen la programación del módulo muy cómoda.

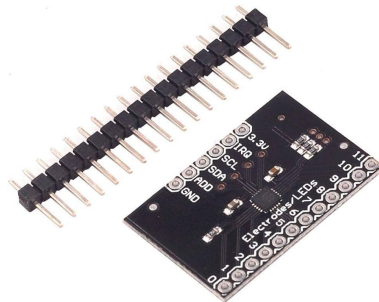


Figura 4.8 módulo MPR121 Breakout V12.

Se van a describir brevemente los pines de la placa y su función:

- Los pines de 0 a 11 son donde se conectan los electrodos o leds.
- El pin indicado por 3.3V es la alimentación. El módulo se debe alimentar a 3.3 Voltios, y es importante que la alimentación sea estable y que no supere los 3.6 Voltios, ya que la placa no tiene regulación interna de tensión y podría averiarse.
- El pin IRQ permite habilitar interrupciones para el control de los electrodos.
- Los pines SCL (Serial Clock) y SDA (Serial Data) corresponden al reloj I2C y al puerto de datos del I2C respectivamente.
- El pin ADD define la dirección I2C del dispositivo. En nuestro caso, si este pin se conecta a tierra, la dirección del dispositivo sería 0x5A.
- Por último tenemos el pin GND que es la tierra

Utilizando este módulo se ha conseguido un funcionamiento mucho mejor que con cualquiera de los otros métodos de sensorización capacitiva. En el siguiente enlace se pueden ver los resultados del prototipo2, que se desarrollará más adelante [Video del prototipo2](#).

4.2.2 Sensores de presión

Como se ha indicado anteriormente, se pretende imitar el funcionamiento de una gaita real lo más cercanamente posible. Esto implica la necesidad de utilizar sensores de presión de fluidos para medir la presión del aire en diferentes partes del controlador MIDI. La idea que se plantea es la siguiente:

- Un sensor de presión que mida la variación de la misma dentro del tubo por el que se sopla. Esto es en el soprete. De esta forma se puede distinguir cuando se esta soplando, para modelar la cantidad de aire que habría en el fol. Yendo un paso más allá, se podría modelar que cuanto mayor fuese el aumento de presión, con mayor fuerza se estaría soplando y más rápidamente se llenaría el fol de aire.
- Otro sensor de presión que mida su variación en una bolsa de aire cuando esta está siendo comprimida por el brazo del intérprete. La bolsa de aire sería estanca, esto es que no necesita ser inflada con aire soplado, sin embargo si debe permitir ser reinflada mediante una bomba de bicicleta por ejemplo. Esto plantea ciertas dificultades de cara a la medición de la presión y la distinción de los diferentes rangos de presión.

Ambos aspectos del funcionamiento se detallarán en los distintos apartados de los prototipos. A continuación se plantean los diferentes modelos de sensor cuyo uso se han barajado, así como el finalmente utilizado:

Las características buscadas en el sensor son principalmente un precio bajo, una fácil implementación en Arduino, y un rango de funcionamiento que permita su uso en el proyecto. En este sentido, buscando información en internet se encontró que la variación en la presión del aire provocada por un soplido humano no suele superar los 7KPa. Por otro lado sobre la presión dentro de una bolsa de aire que apretamos con el brazo no fui capaz de encontrar información, con lo cual se tendrá que confiar en que el sensor que cumpla las especificaciones del primer uso, sirva para el segundo. Si no fuese así habría que adquirir otro sensor diferente.

En el inicio del desarrollo del trabajo encontré información sobre instrumentos de viento MIDI creados por diferentes personas. Ejemplos de ello son el MIDuino EWI, o el KLIK Recorder. La mayoría de ellos utilizaban sensores de la familia MPX del fabricante NXP.

Algunos de los modelos que ofrecen son por ejemplo el MPX2010GP, que tiene un rango de medición de 0 a 10KPa. También el MPX2050DP, con un rango de 0 a 50KPa y modo de actuación diferencial. Consultando la página oficial del fabricante se ve que tienen una gran variedad de modelos, pudiendo ser sensores de presión absoluta o diferencial y con rangos de medición muy variados.

Sin embargo todos ellos tienen precios bastante elevados. Además se debe tener en cuenta que los instrumentos de viento MIDI en los que se han usado, mencionados anteriormente, requieren de la capacidad de articulación. Esto no es necesario en una gaita, puesto que el sonido es continuo y no se puede articular demasiado, solo mediante la presión que se ejerce con el brazo. Esto llevó a optar por algún sensor más barato y con unas prestaciones de menor calidad.

Sensor de presión absoluta HX710B

Este es el sensor por el que finalmente se optó. Ofrece un rango de detección de 0 a 40KPa, más que suficiente para las necesidades del proyecto. Presenta 4 pines, dos de los cuales VCC y GND son la alimentación, que permite un voltaje de entre 3.3 y 5 Voltios, y la tierra. La placa del sensor

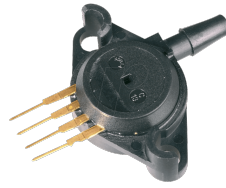


Figura 4.9 sensor de presión absoluta MPX2010GP.

contiene el sensor en sí, que es un PSG010S, además del convertidor A/D de alta precisión de 24 bits, que es el que da nombre al sensor aparentemente.

Para su control en Arduino no hay librerías específicas de este sensor, sin embargo la librería Q2HX711.h ofrece funciones que permiten el control de sensores parecidos, como es nuestro caso.

El montaje del sensor así como las pruebas que se han realizado se detallarán más adelante en los apartados de los prototipos.



Figura 4.10 sensor de presión absoluta HX710B.

4.3 Otros componentes electrónicos

Los sensores mencionados anteriormente son los más importantes y relevantes en el desarrollo de el instrumento, sin embargo hay otros que se han utilizado que permiten una interfaz con el intérprete más cómoda e implementan otras funciones.

4.3.1 Botones

Como botones se han utilizado los típicos pulsadores de arduino. Estos pulsadores utilizan 4 pines y son muy sencillos de utilizar. Si bien no encontré el datasheet de este modelo concreto, se entiende que la mayoría de estos modelos tendrán las mismas características más o menos.

En este proyecto se usarán estos botones para controlar las opciones dentro de los menús y que el intérprete pueda ajustar ciertos parámetros. Más adelante se indicará de que forma se han implementado.



Figura 4.11 Pulsadores.

4.3.2 Encoder digital

El modelo de encoder digital utilizado es un EC-11 rotatorio. Tiene 5 pines, 5V, GND, S1,S2 y KEY. Se alimenta a 5 Voltios, S1 y S2 se utilizan para la salida de datos y la señal de sincronización, y KEY es el pin para el botón que trae el encoder.

En este proyecto se utilizará el encoder para que el intérprete pueda ajustar el volumen. El botón del encoder no se usará en el prototipo final, pero si se usó en ciertas pruebas al principio. Más adelante se explica con más detalle.



Figura 4.12 Encoder digital.

4.3.3 Pantalla LCD

Como interfaz instrumento-intérprete se ha utilizado una pantalla LCD para mostrar cierta información y permitir al usuario variar ciertos parámetros del instrumento.

El modelo de la pantalla es una LCD1602, que permite escribir 16 caracteres en 2 filas. El modulo tiene bastantes pines, aunque algunos de ellos no se utilizarán. Las conexiones se explicarán más adelante, pero se puede ir diciendo que la alimentación es de 5V, y que solo utilizaremos 4 de los 8 pines para datos. Cabe mencionar que la pantalla cuenta con una memoria donde se almacenan los códigos para representar los diferentes caracteres.



Figura 4.13 Pantalla LCD.

4.3.4 Conector MIDI hembra

Una de las especificaciones propuestas para el final del proyecto es la capacidad de conectar el instrumento mediante cableado MIDI a un sintetizador cualquiera. Por ello necesitamos un conector MIDI en el instrumento. Se han utilizado conectores de 5 pines MIDI. Su conexionado se verá más adelante.

4.3.5 Cableado

Los cables que se han utilizado para el primer prototipo y las fases más tempranas del desarrollo y las pruebas de los sensores han sido los cables típicos de Arduino, los Jumper wire cables. No hay



Figura 4.14 Conector MIDI hembra.

mucho que decir de ellos, ya que están pensados para su uso en Arduino. Los conectores Dupont permiten su fácil conexión a los pines de a placa.

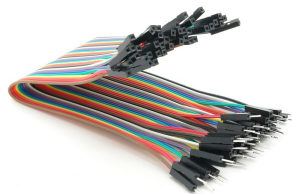
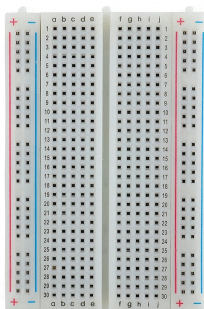


Figura 4.15 Jumper wire cables.

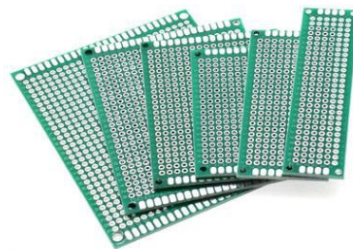
Para etapas del desarrollo posteriores se han utilizado cables más largos que permiten la conexión de los diferentes elementos a una mayor distancia de la placa. Eso si, es necesario soldar.

4.3.6 Protoboard y placas PCB

En las primeras etapas del proyecto se utilizó para el montaje de los componentes un par de protoboards. Más adelante para el montaje final, se soldaron los componentes a unas placas PCB, que permiten un montaje más fijo, y aseguran que los cables no se van a desconectar.



(a) Protoboard.



(b) Placas PCB.

Figura 4.16 Elementos de conexión utilizados.

4.3.7 Resistencias

Para conectar ciertos componentes se necesitan resistencias de distintos valores, por lo que adquirí un paquete de resistencias variadas de entre 1 y 4W. Cuando se expliquen los detalles de las conexiones se verán los valores de las resistencias utilizados.

4.3.8 Impresora 3d

Si bien la impresora 3d no forma parte de los componentes electrónicos necesarios para la construcción en si del instrumento, se incluirá en este apartado. La máquina de impresión 3d utilizada es una Anycubic Kobra 2, con la capacidad de imprimir en un volumen de 22x22x25 centímetros. Para el diseño de las piezas y el software de impresión 3d se utilizaron programas que se verán en el siguiente apartado. El material utilizado para las piezas es PLA, que es un polímero o bioplástico muy utilizado para la impresión 3d.



Figura 4.17 Impresora 3d Anycubic Kobra 2.

5 Software del proyecto

En este capítulo se verán los diferentes programas que se han utilizado a lo largo del desarrollo para la programación, conexión, o fabricación del instrumento, así como el dibujado de esquemáticos y planos.

5.1 Arduino IDE

El entorno de desarrollo integrado (integrated development enviroment, IDE) de Arduino es un programa que se utiliza para programar las placas de Arduino, como nuestro Arduino UNO. El lenguaje que se utiliza para programar es C++ o C.

Su uso es bastante sencillo. En la interfaz se ven una serie de pestañas desplegadas con funciones como abrir archivos, guardar, ver ejemplos de las librerías y descargar nuevas, etc. Las funciones que interesan más son el botón de compilar, que es circular con un tick dentro, que permite compilar el programa, revisar errores de sintaxis, y ver los recursos de la placa que se utilizarían, que se devuelven en la consola de salida. También se usará el botón de subir, circular con una flecha hacia la derecha, que permite cargar los programas en la placa. Además también se elegirá la placa que utilizemos y el puerto USB al que está conectada, y el monitor serie y el Serial Plotter, para poder ver la salida del puerto serie. Esto será muy útil a lo largo de todo el desarrollo.

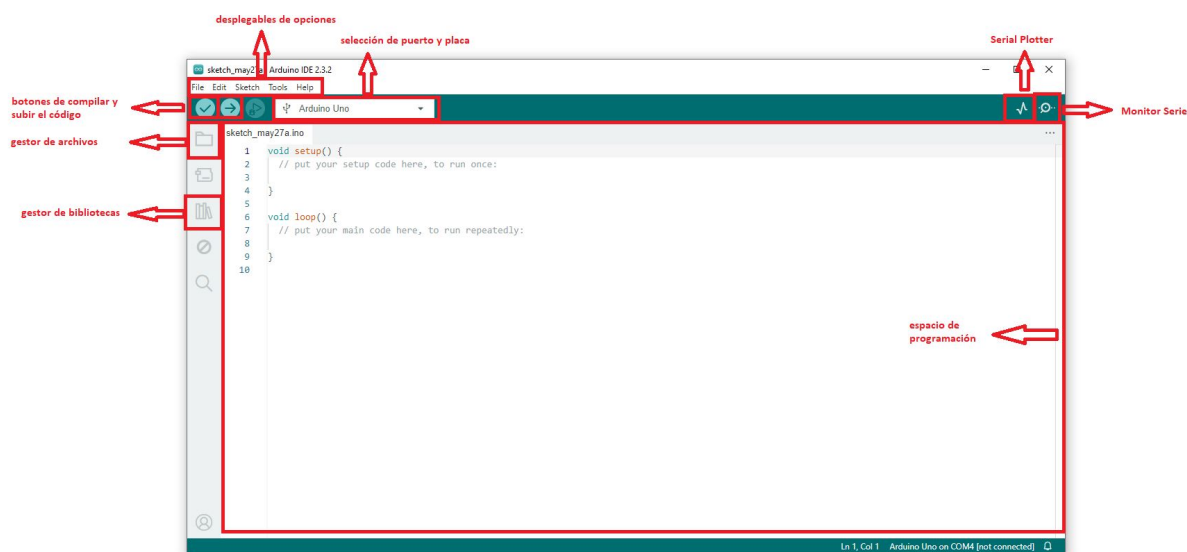


Figura 5.1 Interfaz del Arduino IDE.

5.2 Protocolo MIDI y conexión MIDI del instrumento

5.2.1 Protocolo MIDI

El protocolo MIDI o Musical Instrument Digital Interface es un protocolo de comunicación y conexión entre instrumentos musicales electrónicos. Los mensajes MIDI transmiten mensajes de eventos que especifican ciertas características musicales de la nota transmitida o mensajes de control sobre el sintetizador para variar parámetros del mismo. Los mensajes se envían mediante un cable MIDI, o alguna variación que permita este tipo de comunicación, como en nuestro caso un cable USB, a otro dispositivo electrónico que se encargue de la generación del sonido.

En este proyecto la comunicación MIDI se realizará mediante USB, aunque también se podrá realizar mediante un conector MIDI hembra y cables macho-macho para conectarlo a otros sintetizadores que no sean los virtuales de un ordenador.

Las palabras MIDI transmiten 8 bits o 1 byte de manera serial a 31.25 kbit/s. El primer bit de cada palabra identifica si la palabra es un byte de estatus o de datos, los siguientes siete bits son la información del mensaje. Un bit de inicio y otro de pausa son agregados a cada byte por cuestiones de sincronización, así que una palabra MIDI requiere de diez bits para transmitirse.

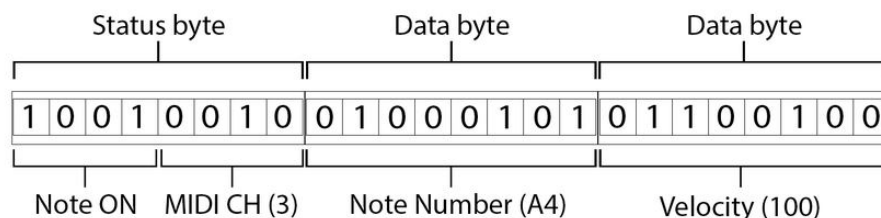


Figura 5.2 Formato de un mensaje MIDI.

Los mensajes MIDI están conformados por 2 o 3 palabras de este tipo. La primera es siempre un byte de status, que indica el tipo de mensaje, y después se envían 1 o 2 palabras más de datos, de 1 byte cada una. el byte de status empieza siempre por un bit a 1 (1XXXXXXX) e indican la acción que se va a desarrollar. Los bytes de datos por otro lado empiezan en 0 (0XXXXXXX) y aportan información adicional sobre esa acción.

En la tabla a continuación (figura 5.3) se pueden de forma más detallada como se compone un mensaje MIDI. Podemos ver que el primer byte, o status byte, contiene el tipo de mensaje, definido por un 1 como primer bit siempre y 3 bits para las diferentes acciones (8 en total,^{2,3}). los siguientes 4 bits del status byte indican el canal para el que se realiza la acción. Como tenemos 4 bits, eso implica 16 canales diferentes posibles, desde el 0000 hasta el 1111 (del canal 0 al 15). Después, dependiendo de la acción que se quiera llevar a cabo, necesitaremos 1 o 2 bytes de datos. Debemos tener en cuenta que los bytes de datos empiezan siempre en 0, por lo que quedarían 7 bits para codificar el valor. Esto implica 128 valores diferentes.

Un dispositivo puede ser configurado para solo escuchar canales específicos e ignorar los mensajes enviados de otros, o puede escuchar a todos los canales sin importar su dirección. Además puede ser monofónico (el inicio de una nueva señal de “note-on” implica el final de la nota previa) o polifónico (múltiples notas pueden sonar al mismo tiempo, hasta que el comando “note-off” apague la nota).

MIDI note ON y note OFF

A continuación se verá un ejemplo de mensaje MIDI de los más utilizados. Este sería un mensaje de MIDI “note ON”. Este mensaje enviaría al sintetizador la orden de inicio del sonido de una nota. Se compondría de la siguiente forma:

MIDI Messages

Channel Messages						
Channel Messages use the lower 4 bits of the status byte to indicate the MIDI channel of the message. 0 is MIDI channel 1, and 15 is MIDI channel 16						
Function	Objects	Status Byte			2nd Byte (0-127)	3rd Byte (0-127)
		Decimal	Hex	Binary		
Note Off	xnotein, xnoteout	128-143	80-8F	1000xxxx	Key Number	Release Velocity
Note On	notein, noteout	144-159	90-9F	1001xxxx	Key Number	Velocity
Poly Pressure	polyin, polyout	160-175	A0-AF	1010xxxx	Key Number	Aftertouch
Control Change	ctlin, ctlout	176-191	B0-BF	1011xxxx	Controller Number	Controller Data
Program Change	pgmin, pgmout	192-207	C0-CF	1100xxxx	Program Number	
Aftertouch	touchin, touchout	208-223	D0-DF	1101xxxx	Aftertouch Value	
Pitch Bend	bendin, bendout	224-239	E0-EF	1110xxxx	Bend (LSB)	Bend (MSB)

Figura 5.3 Formato de un mensaje MIDI más detallado.

Status byte: primer bit 1, siguientes bits 1001 (código correspondiente a la acción "note ON").
Sigüientes 4 bits el número del canal, por ejemplo canal 0001.

Data byte 1: primer bit 0 siempre, los restantes 7 bits del byte un valor entre 0 y 127. En el caso del mensaje "note ON", esto representaría el valor de la nota. En la figura 5.4 se pueden ver los valores de las diferentes notas. Con 128 valores diferentes se cubren 10 octavas y media, más que suficiente para los sonidos interpretables por la mayoría de instrumentos.

Data byte 2: En el caso del mensaje "note ON" necesitamos este segundo byte de datos, y representa la velocidad con la que se pulsa la nota. de nuevo el primer bit es 0, y los siguientes 7 bits serán un valor entre 0 y 127.

Note Numbers												
Octave	C	C#	D	D#	E	F	F#	G	G#	A	A#	B
0	0	1	2	3	4	5	6	7	8	9	10	11
1	12	13	14	15	16	17	18	19	20	21	22	23
2	24	25	26	27	28	29	30	31	32	33	34	35
3	36	37	38	39	40	41	42	43	44	45	46	47
4	48	49	50	51	52	53	54	55	56	57	58	59
5	60	61	62	63	64	65	66	67	68	69	70	71
6	72	73	74	75	76	77	78	79	80	81	82	83
7	84	85	86	87	88	89	90	91	92	93	94	95
8	96	97	98	99	100	101	102	103	104	105	106	107
9	108	109	110	111	112	113	114	115	116	117	118	119
10	120	121	122	123	124	125	126	127				

Figura 5.4 codificación de notas MIDI.

El tipo de mensaje "note OFF" es muy parecido al "note ON". Indica que una nota debe dejar de sonar. El byte de status tendrá el valor 1000, el canal debe ser el mismo en el que se mando la nota que ahora se quiere apagar, el primer y segundo bytes de datos tendrán el número de la nota a

apagar y la velocidad con que se apagará.

MIDI Control Change

Los mensajes tipo "Control Change" se utilizan para variar parámetros que hacen que el sonido sea más orgánico, y más parecido a un instrumento real, así como servir de interfaz para ciertos módulos como pedales, ruedas y demás. Las funciones de estos mensajes más usadas pueden ser variar el volumen, añadir vibrato, tremolos, reverb, chorus, etc.

Como hemos visto que los primeros 4 bits del byte de status deben ser 1011. En el primer byte de datos se debe indicar el número correspondiente al control que queremos tocar. En la figura 5.5 se pueden ver algunos de los valores, si bien la tabla entera no cabe en esta memoria por ser 128 valores diferentes, estos valores están disponibles en muchos sitios de internet.

MIDI CC Number	MIDI CC Purpose	MIDI CC Description
MIDI CC 0	Bank Select	Allows user to switch bank for patch selection. Program change used with Bank Select. MIDI can access 16,384 patches per MIDI channel.
MIDI CC 1	Modulation	Generally this CC controls a vibrato effect (pitch, loudness, brightness). What is modulated is based on the patch.
MIDI CC 2	Breath Controller	Often times associated with aftertouch messages. It was originally intended for use with a breath MIDI controller in which blowing harder produced higher MIDI control values. It can be used for modulation as well.
MIDI CC 3	Undefined	
MIDI CC 4	Foot Controller	Often used with aftertouch messages. It can send a continuous stream of values based on how the pedal is used.
MIDI CC 5	Portamento Time	Controls portamento rate to slide between 2 notes played subsequently.
MIDI CC 6	Data Entry Most Significant Bit(MSB)	Controls Value for NRPN or RPN parameters.
MIDI CC 7	Volume	Control the volume of the channel
MIDI CC 8	Balance	Controls the left and right balance, generally for stereo patches. 0 = hard left, 64 = center, 127 = hard right
MIDI CC 9	Undefined	
MIDI CC 10	Pan	Controls the left and right balance, generally for mono patches. 0 = hard left, 64 = center, 127 = hard right
MIDI CC 11	Expression	Expression is a percentage of volume (CC7).
MIDI CC 12	Effect controller 1	Usually used to control a parameter of an effect within the synth/workstation.
MIDI CC 13	Effect controller 2	Usually used to control a parameter of an effect within the synth/workstation.
MIDI CC 14-15	Undefined	
MIDI CC 16-19	General purpose	
MIDI CC 20-31	Undefined	
MIDI CC 32-63	Controller 0-31 Least Significant Bit (LSB)	

Figura 5.5 Mensajes de control MIDI.

Además, ocurre que en muchos sintetizadores existe la posibilidad de modificar el valor asociado a cada control. La lista que se mencionó anteriormente es una lista convencional de valores, pero cada sintetizador puede permitirte variar el valor asociado a cada control. También se pueden ver

valores que no tienen funciones asignadas, y que pueden servir para implementar aspectos de los instrumentos que no sean tan comunes. En el caso de nuestra gaita se verá que el sintetizador permite algunas funciones que ocuparán precisamente estos valores sin asignar.

Aunque es importante saber como funciona el protocolo MIDI y sus funciones y formatos de mensaje, la realidad es que gracias a algunas librerías de Arduino no es necesario montar cada mensaje bit a bit, sino que mediante funciones de alto nivel la librería genera directamente el formato del mensaje correcto. La librería que he utilizado se verá más adelante, junto con el resto de librerías.

Como dato interesante, el 17 de enero de 2020 se presentó el estándar MIDI 2.0.

5.2.2 LoopMIDI

Como se ha descrito anteriormente, para hacer que el instrumento reproduzca sonidos es necesario un sintetizador. El microcontrolador lo único que hace es generar mensajes MIDI en función de los botones que se estén tocando y de el resto de controles del instrumento. Estos mensajes MIDI deben ser interpretados en un sintetizador o algún módulo que se conecte al Arduino.

Es importante mencionar que si bien se podría haber utilizado algún módulo o shield que se conectase directamente al Arduino y que hiciera que el instrumento generase sonido por si mismo sin necesidad de estar conectado a un ordenador, teniendo en cuenta las funciones que se piensan implementar, no quedan suficientes salidas del arduino libres. Además la ventaja de utilizar un sintetizador en el ordenador permite generar un sonido de calidad muy superior, además de una mayor cantidad de opciones como efectos y diferentes instrumentos.

Así pues, el instrumento se conectará a un ordenador donde se encuentre activo un sintetizador, por lo que es necesario el uso de ciertos programas que interpreten los mensajes que recibe por el puerto USB como mensajes MIDI, y adapte la frecuencia de llegada de los mismos y almacene los datos en un buffer. Para ello se ha usado LoopMIDI y Hairless MIDI<->Serial que se verán a continuación.

LoopMIDI es un software usado para crear puertos virtuales MIDI de tipo loopback, para programas que requieran de una conexión en un puerto MIDI. El loopback en una interfaz de audio permite redirigir el audio interno de nuevo en la interfaz para su reutilización en otras aplicaciones. Esto significa que puedes redirigir o grabar el audio del sistema o las aplicaciones de instrumentos virtuales.

La interfaz del programa es bastante sencilla y permite crear puertos virtuales con nombres personalizados, así como variar ciertos parámetros de configuración en la pestaña "Advanced". Para generar los puertos utiliza el driver "virtualMIDI" del mismo creador.

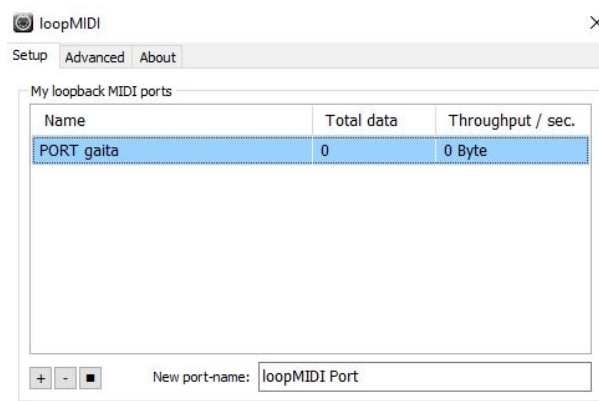


Figura 5.6 Interfaz del software LoopMIDI.

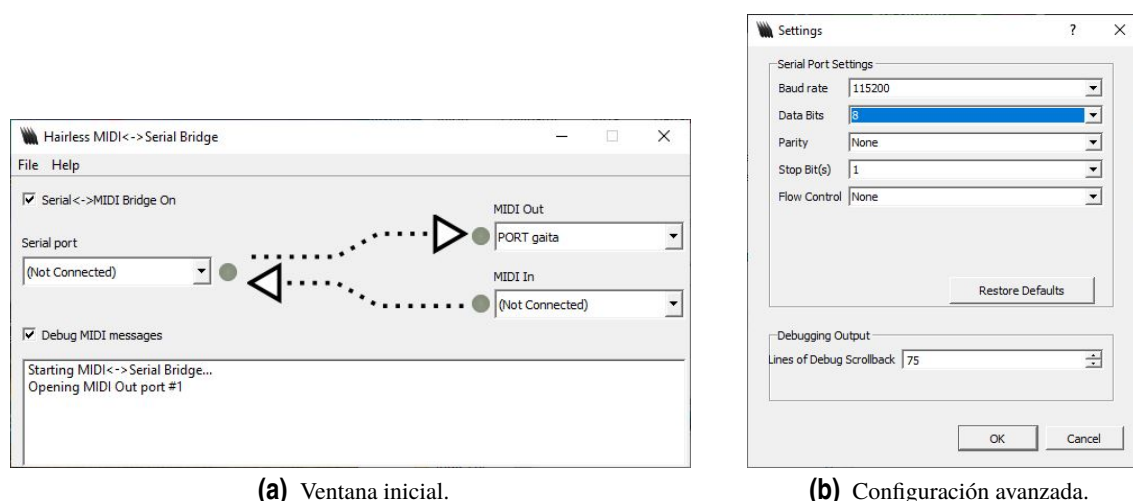
Para que el sintetizador reciba los mensajes del micro, debemos tener activo un puerto virtual de esta aplicación, y definir este puerto como MIDI IN.

5.2.3 Hairless MIDI Serial

Este es otro software necesario para transmitir los mensajes MIDI desde el Arduino. El software se encarga de reconocer la señal de USB, entendiéndola como un puerto serie, adaptar el mensaje del puerto serie a los estándares MIDI de baudios por segundo y demás, y enviar esos datos al puerto MIDI, en esta caso virtual y creado mediante LoopMIDI.

La interfaz es fácil de entender. Para poder utilizarlo debemos definir como puerto de salida el creado con LoopMIDI, y como puerto serie el correspondiente al Arduino. Estos son los COM, normalmente. Si el Arduino fuese solo un elemento intermedio que recibiese mensajes MIDI desde otro puerto, este se definiría en la pestaña MIDI In. Vemos también en la parte inferior una consola donde se muestran los mensajes MIDI que llegan desde la conexión serie.

Si accedemos a la pestaña File->Settings podemos ajustar la tasa de baudios, la longitud de los datos en el mensaje y demás. Es importante denotar que la tasa de Baudios del programa debe coincidir con la del puerto serie del Arduino. Normalmente para este tipo de aplicaciones se utiliza 115200 bps. Además como ya se explicó, el formato de mensajes MIDI utiliza 1 byte u 8 bits para datos.



(a) Ventana inicial.

(b) Configuración avanzada.

Figura 5.7 Interfaz del software Hairless MIDI<->Serial.

5.3 Sintetizador

Para interpretar los mensajes MIDI y generar el sonido de las notas musicales se debe utilizar un sintetizador. Como el controlador está conectado al ordenador, debemos descargar algún sintetizador virtual o VST. En este sentido se plantean una inmensidad de opciones tanto de pago como software libre. Algunos ejemplos pueden ser el VST Dexed, Helix o Tal-Bassline. También otros programas de producción musical como Pro Tools o Ableton.

Como en alguna ocasión ya he enredado con programas de producción musical ya tenía instalado el FL Studio, e intenté utilizarlo para este trabajo.

5.3.1 FL Studio

FL Studio (Fruity Loops), es una estación de trabajo de audio digital o DAW con las características de editor de audio, secuenciador con soporte multipista y MIDI utilizado para la producción musical y desarrollado por la compañía belga Image-line Software.

La herramienta contiene un secuenciador, varios sintetizadores, bibliotecas de samples, cajas de ritmo, etc. Este permite al artista crear música basada en patrones, usando el Step Sequencer o la vista de Piano Roll, un piano virtual. Además soporta una gran cantidad de plugins, como el que se comentará en el siguiente apartado.

El panel de efectos ubicado en el mezclador (mixer) da acceso a una gran variedad de software de efectos, de ecualización, reverberación, delay, además de herramientas de masterización, limitación y compresión de audio.



Figura 5.8 Logo de FL Studio.

Como se puede ver el programa es muy potente en el mundo de la producción musical y da una cantidad de opciones inmensa, incluyendo una gran cantidad de sintetizadores y conexión MIDI para tocar y grabar con controladores MIDI. El programa eso si, es de pago, y quizá su mayor problema es que resulta bastante complicado adaptarse a la interfaz y es un software complejo. Sin embargo la posibilidad de incorporar plugins permite precisamente añadir alguno orientado hacia instrumentos de viento y concretamente a la gaita.

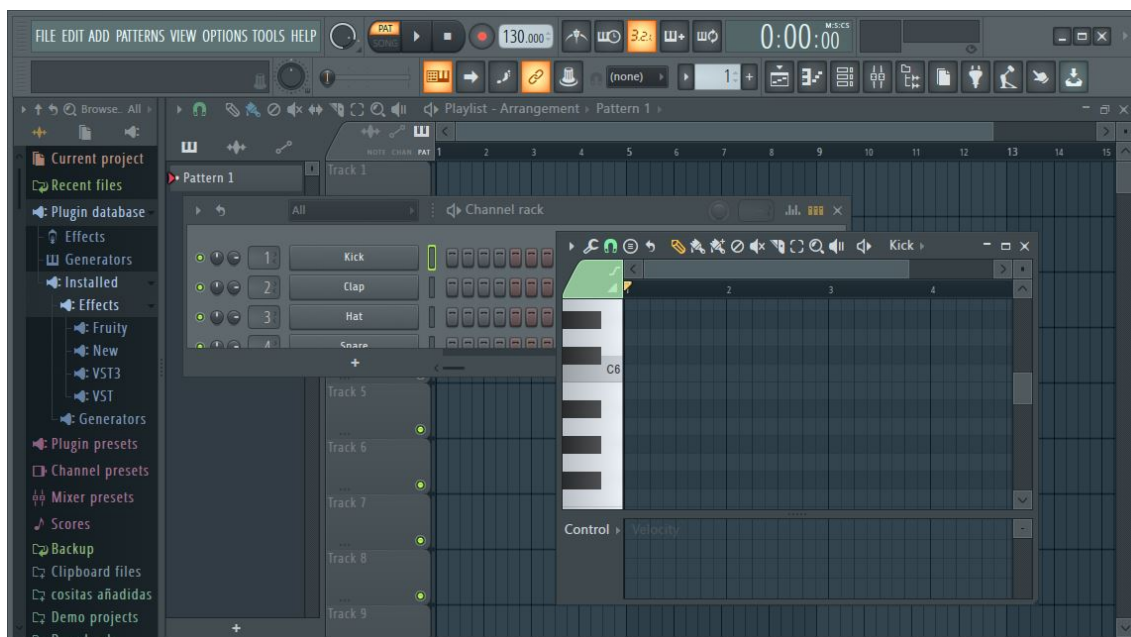


Figura 5.9 Interfaz de FL Studio.

SIM-BP

Quilcom SIM-BP (simulation bag-pipe) es un sintetizador de Quilcom diseñado para simular el sonido de diferentes tipos de gaitas. No utiliza samples, sino solo métodos de síntesis. Esto significa que genera una onda que imita el sonido de la gaita, no utiliza sonidos de gaitas reales. Esto se traduce en un sonido que se nota menos natural, más electrónico. Pero también que se pueden modificar muchos más parámetros y añadir más efectos que si usara samples.

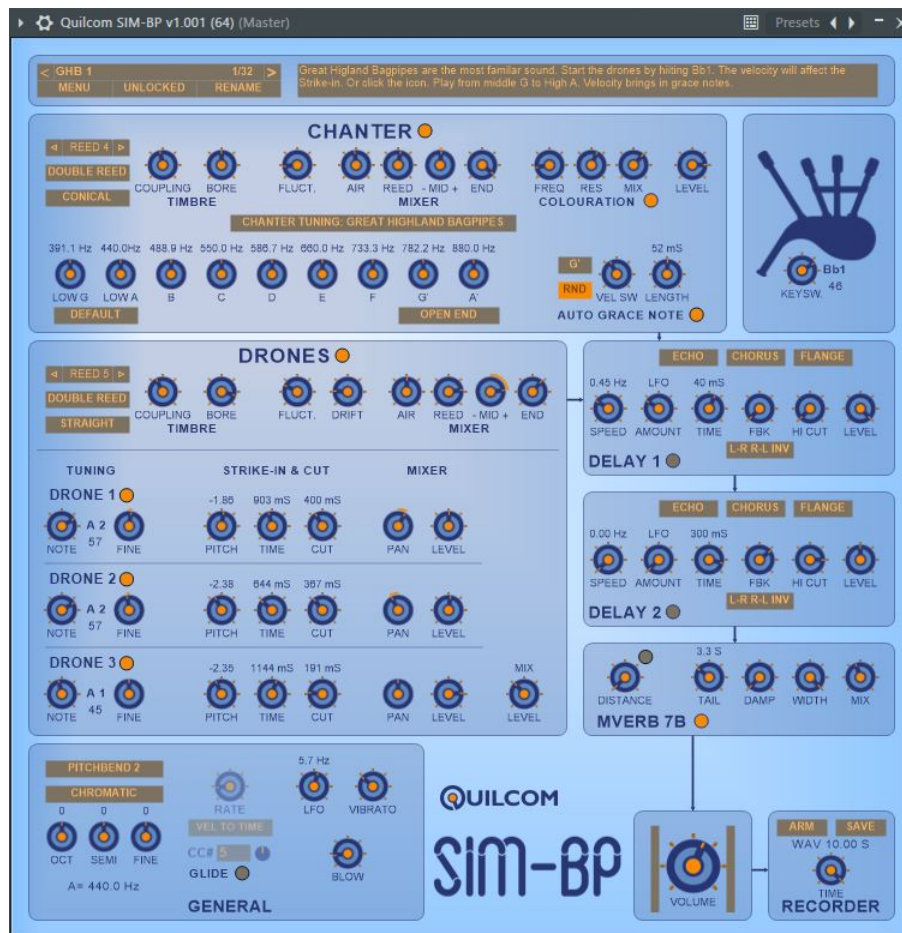


Figura 5.10 Interfaz del plugin SIM-BP.

Aquí es importante definir algunos términos importantes que se van a usar en este tipo de sintetizadores. Los "drones" son los tubos secundarios de la gaita que tocan la nota pedal en diferentes octavas. Vemos que se puede configurar para incluir hasta 3 "drones", pudiendo modificar también la nota continua que tocan. El "chanter" vendría a ser el punteiro, esto es el tubo donde tapamos los agujeros y tocamos la melodía. Podemos modificar la nota asociada a cada agujero, variando su frecuencia.

Podemos añadir también efectos como varios delays y reverbers. Para empezar a tocar pulsaremos sobre la gaita que está en la parte superior derecha de la ventana. Se puede probar con el teclado pulsando desde la tecla "T" hasta la tecla "+".

El problema de este sintetizador es que necesita de un DAW para poder funcionar. La conexión MIDI se haría en el DAW, en este caso en FL Studio, definiendo el puerto creado mediante LoopMIDI como un puerto de un instrumento MIDI.

5.3.2 Universal Piper

Este es el sintetizador utilizado finalmente para el desarrollo de este proyecto. Es un sintetizador completo, no una extensión para un DAW, por lo que nos ahorramos el tener descargado este último.

Este sintetizador es perfecto para su uso en este proyecto. Tiene una librería de 16 tipos diferentes de gaitas, incluyendo la gallega, gaitas escocesas, irlandesas y bretonas. Además permite crear y guardar tus propias configuraciones. A continuación vamos a explicar cada una de las diferentes pantallas y pestañas y que opciones nos brindan.

En la pantalla inicial (figura 5.11) se puede ajustar el volumen general, la afinación de la gaita, activar el sonido de los tubos secundarios o "drones", así como su volumen. Utilizando las flechas del apartado "instrument" se cambia entre los diferentes presets de gaitas.



Figura 5.11 Interfaz del sintetizador Universal Piper. Pantalla Inicial..

En la siguiente pantalla "Harmony" (figura 5.12) se pueden ajustar las notas asociadas a cada agujero que tapemos, además de la nota de los "drones" y cuantos de ellos suenan a la vez. También se tiene la posibilidad de configurar una segunda voz para imitar el sonido de otra gaita que puede sonar una tercera, quinta, séptima, etc. por encima o por debajo del sonido de la gaita principal.

En la pantalla "Effects" (figura 5.13) se pueden añadir distintos efectos tanto a los "drones" como al "chanter". Estos efectos incluyen un ecualizador, reverb, delay, vibrato, etc. Además de la posibilidad de importar efectos.

Esta pantalla "MIDI" (figura 5.14) es la más importante para poder utilizar un controlador MIDI con este sintetizador. Se presenta una lista de opciones del sintetizador como el volumen, los "drones", la afinación, efectos, etc. Si nos fijamos en la parte derecha de cada efecto podemos definir un mensaje de control o nota asociado a cada una de estas acciones, con la posibilidad de hacer el control de tipo momentary, toggle o automatic. Esto es importantísimo para poder enviar mensajes de control desde el micro y que sean recibidos correctamente por el sintetizador. Más adelante se definirán estas funciones en el código de Arduino.

Si vamos a la parte inferior de la ventana podemos pulsar donde pone "MIDI Console" (figura 5.15) para acceder a una consola donde se verán todos los mensajes MIDI que llegan al sintetizador. Esto puede servir para ver si algún mensaje no está llegando correctamente.

En la pantalla "Keyboard" (figura 5.16) se implementa una función parecida a el piano roll de FL Studio, donde puedes tocar con el ratón pulsando sobre las teclas diferentes notas.

La pantalla "Hardware" (figura 5.17) también es bastante importante. Aquí se pueden definir las entradas MIDI, y por tanto debemos indicar el puerto creado por LoopMIDI como la entrada del sintetizador. Además se pueden variar parámetros relacionados con la salida de audio.



Figura 5.12 Interfaz del sintetizador Universal Piper. Pantalla Harmony..



Figura 5.13 Interfaz del sintetizador Universal Piper. Pantalla Effects..

5.4 Modelado e impresión 3d

Como parte final del proyecto se han utilizado varios programas para la fabricación de piezas mediante una impresora 3d. Estas piezas permiten un montaje compacto de los componentes del instrumento, así como un aspecto atractivo y ergonómico, con los controles necesarios para la



Figura 5.14 Interfaz del sintetizador Universal Piper. Pantalla MIDI..

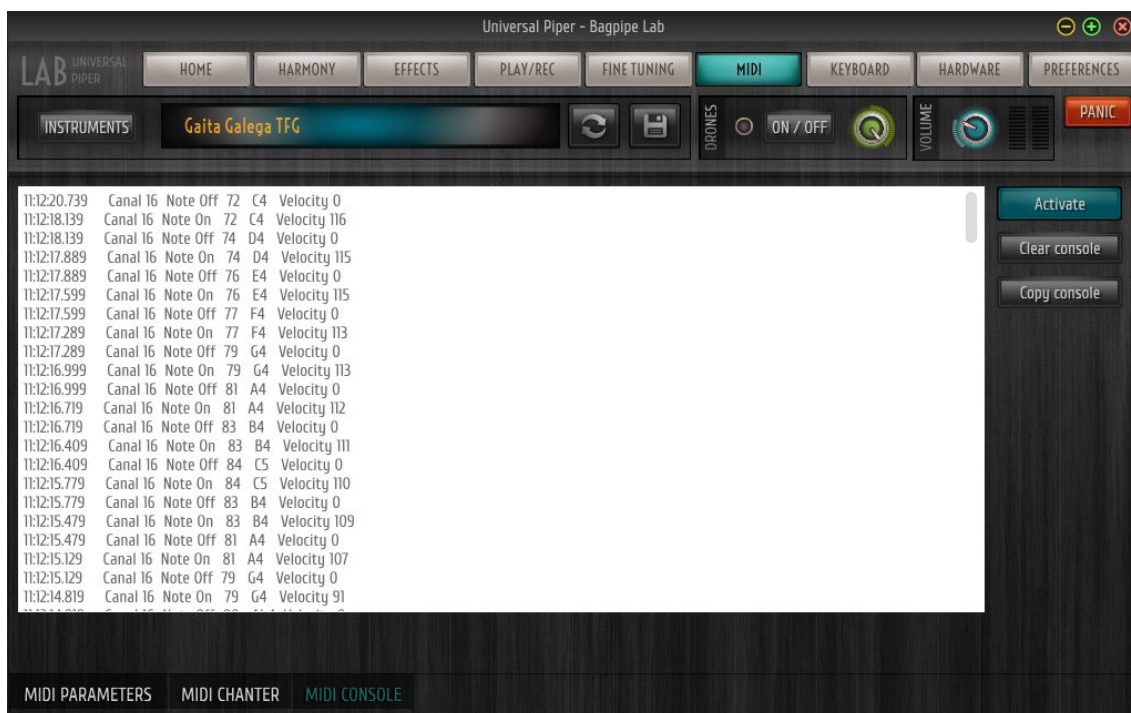


Figura 5.15 Interfaz del sintetizador Universal Piper. Pantalla de la consola MIDI..

interacción con el usuario fácilmente accesibles. En esta parte se incluye software para el modelado de las piezas, además del necesario para su impresión.

Existen una gran cantidad de opciones de software para el modelado 3d, muchas de pago y algunas gratuitas. A continuación se detallan los programas escogidos. Los planos de las piezas, con sus proporciones exactas, así como los archivos de los modelos 3d se incluirán o bien en el

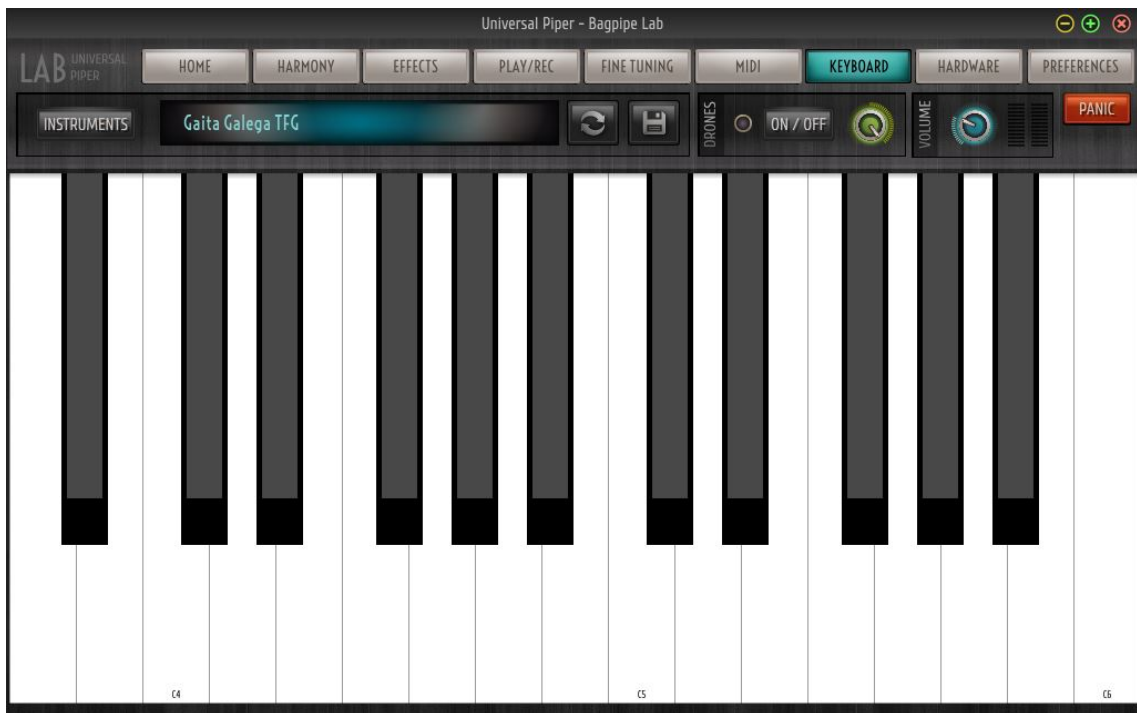


Figura 5.16 Interfaz del sintetizador Universal Piper. Pantalla Keyboard..



Figura 5.17 Interfaz del sintetizador Universal Piper. Pantalla Hardware..

apartado "apéndices" de esta memoria o bien en algún enlace adjunto.

5.4.1 FreeCAD

Para el modelado 3d de las piezas se ha utilizado FreeCAD. Este software ha sido utilizado en la asignatura de expresión gráfica en primero de carrera, por lo cual ya estoy familiarizado con el

entorno, además de ser una herramienta gratuita.

FreeCAD es una aplicación libre de diseño asistido por computadora en tres dimensiones para la asistencia en ingeniería mecánica y el diseño de elementos mecánicos. FreeCAD presenta un entorno de trabajo similar a CATIA, SolidWorks o SolidEdge. A diferencia de los CAD analíticos tradicionales, como puede ser AutoCAD, FreeCAD es un CAD paramétrico que utiliza parámetros para definir sus límites o acciones. FreeCAD utiliza un formato de archivo propio denominado FreeCAD Document (.FCStd), y puede utilizar también el formato Objeto 3D (.stl) que puede ser utilizado para la impresión 3d mediante Prusa.



Figura 5.18 Logo del software FreeCAD.

Algunas de las funciones más importantes de este programa son:

- Mediante la herramienta Sketcher se pueden dibujar las vistas del objeto, y mediante el PartDesign extruir, revolucionar, añadir taladros o agujeros utilizando el sketch creado.
- Las herramientas Part y PartDesign permiten generar volúmenes y dar forma al objeto.
- La herramienta TechDraw, permite obtener un plano con diferentes vistas del objeto, pudiendo acotar las medidas y con muchas opciones de representación, como mostrar líneas ocultas, cortes de la pieza, etc.

Si bien se podría explicar en profundidad todas las opciones y botones de la interfaz, esto sería demasiado extenso teniendo en cuenta que el modelado 3d no es el objetivo principal del proyecto. De todas maneras más adelante se explicará más en profundidad cómo se han fabricado las piezas, además de aportar planos con las medidas de las mismas.

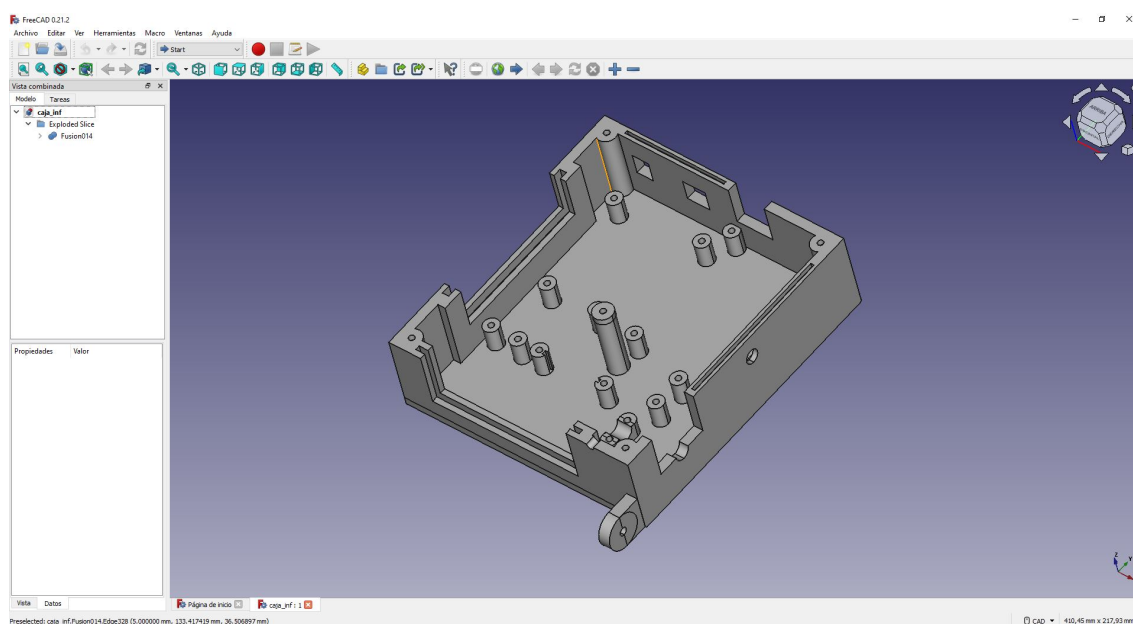


Figura 5.19 Interfaz del software FreeCAD.

5.4.2 Prusa

Prusa es un software de corte necesario para la impresión 3d de piezas, gratuito y de código abierto. En sus últimas versiones han mejorado la interfaz para hacer más sencillo su uso. Entre otras cosas, el programa permite laminar cuidadosamente el modelo de capas y generar soportes de impresión si es necesario.



Figura 5.20 Logo del software PrusaSlicer.

El software incluye opciones para permitir la impresión mediante máquinas de terceros. Incluye diferentes funciones para añadir soportes, elegir la altura de capa para cada parte de la pieza, perfiles de impresión de filamento y resina, etc. También ofrece algunas funciones más avanzadas para perfiles intermedios y expertos. El programa también es capaz de calcular los soportes y determinar las zonas en las que más se requieren. Prusa admite los archivos .stl generados mediante FreeCAD, además de generar su propio tipo de archivo llamado Archivo GCODE (.gcode), que ya se puede introducir en una impresora 3d.

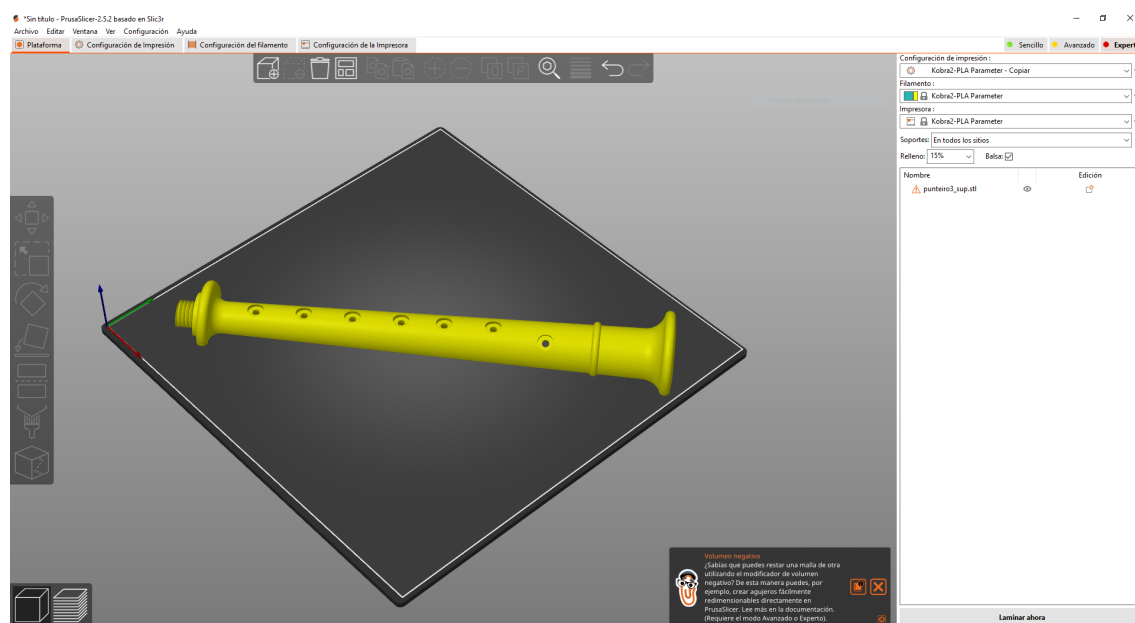


Figura 5.21 Interfaz del software PrusaSlicer.

5.5 Dibujado de esquemáticos y planos

Para dibujar los esquemáticos de las conexiones del Arduino se ha utilizado un programa llamado Fritzing. El programa es software libre, aunque realmente para descargar las últimas versiones tienes que hacer una donación obligatoria. Este sirve para montar circuitos, diseñar placas pcbs y hacer los esquemáticos de nuestros proyectos de electrónica. La interfaz es muy sencilla y permite importar

componentes desde archivos ".fzpz". Los esquemáticos realizados se incluyen en el apéndice "Esquemático de conexión completa del instrumento" o [B](#).

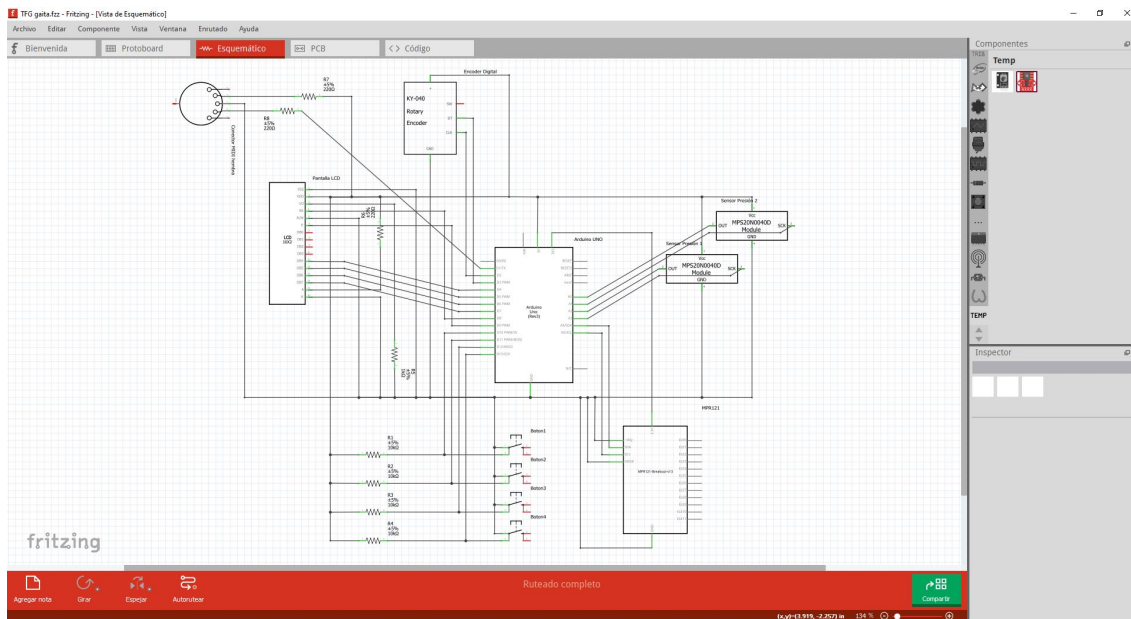


Figura 5.22 Interfaz de Fritzing.

Por otro lado para dibujar los planos de las piezas se ha utilizado LibreCAD, aprovechando el módulo TechDraw de FreeCAD, que se encarga de generar un plano de vistas de la pieza. Este plano se puede exportar como un archivo .dxf para retocarlo en libreCAD y generar los planos de las piezas que se ven en el apéndice [C](#).

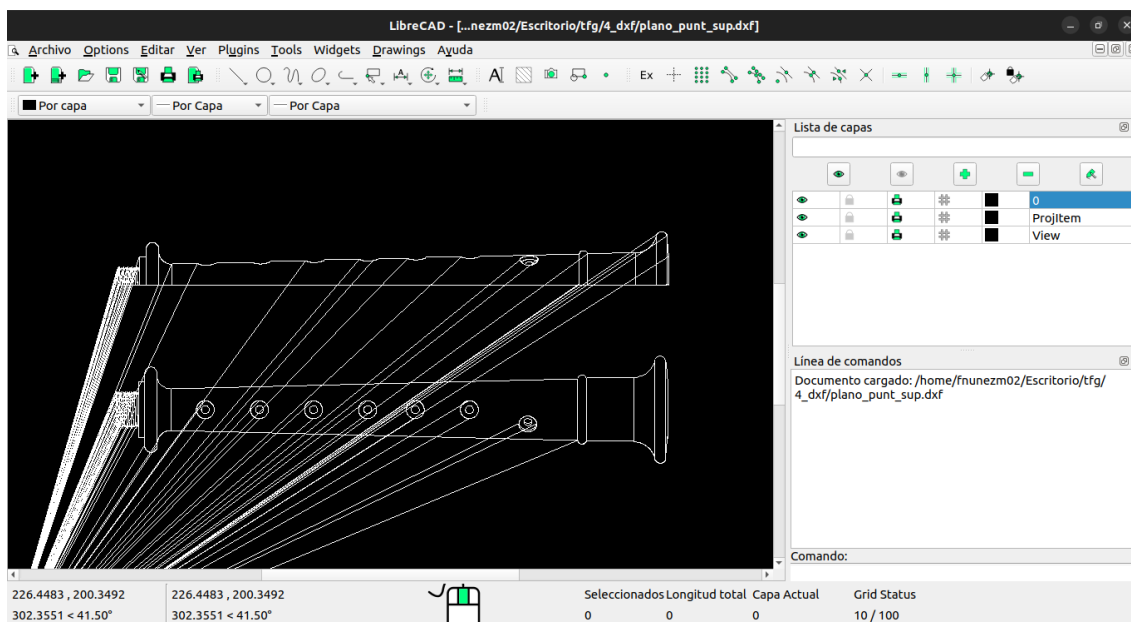


Figura 5.23 Interfaz de LibreCAD.

5.6 Librerías de Arduino

Dentro de este apartado se incluirán también las librerías de Arduino que se han utilizado para controlar los distintos componentes del instrumento, como los sensores de presión, capacitivos o la pantalla.

Precisamente una de las ventajas más importantes de utilizar este microcontrolador de Arduino para el proyecto es la grandísima cantidad de librerías que existen para estas placas. Dentro de cada librería que describa a continuación, se indicarán las funciones más importantes y que más he utilizado.

5.6.1 Librería MIDI.h

Permite crear entradas y salidas MIDI mediante un Arduino, creando mensajes en este formato que envía por el puerto serie. Dependiendo de la conexión que tenga la placa, esta comunicación serial se llevará a cabo mediante el USB o mediante alguna salida MIDI que se conecte con los pines TX y RX de la placa.

La librería es compatible con todas las placas de Arduino, y permite lectura de entrada OMNI (que lee todos los canales a la vez). La versión utilizada es la 5.0.2 y accediendo al [GitHub](#) de este [enlace](#), se puede encontrar más información.

Las funciones más importantes son las siguientes:

- `MIDI_CREATE_DEFAULT_INSTANCE()`: Esta función se utiliza para crear una instancia de la clase MIDI por defecto. Esto facilita la configuración y el inicio de la comunicación MIDI sin necesidad de especificar parámetros adicionales, como el puerto serial predeterminado y una configuración estándar para la velocidad de baudios, que generalmente es 31250 bps. Su contraparte sería la función `MIDI_CREATE_INSTANCE()`, que requiere como parámetros de entrada el tipo de puerto serie, el nombre del puerto serie y el nombre del canal MIDI asociado a esa comunicación. En el caso del Arduino UNO, solo tenemos un puerto serie, con lo cual esta función no sirve para mucho.
- `MIDI.begin()`: Inicializa la comunicación MIDI en el puerto serie especificado en la función `MIDI_CREATE_INSTANCE()`. Después de llamar a esta función debemos hacer `Serial.begin()` e indicar los baudios por segundo de la comunicación.
- `MIDI.sendNoteOn()`: implementa la función "Note ON" descrita en el apartado del protocolo MIDI. Debemos introducir 3 parámetros: número de la nota, velocidad y canal.
- `MIDI.sendNoteOff()`: implementa la función "Note OFF" descrita anteriormente también. Recibe los mismos parámetros que la función Note ON.
- `MIDI.sendControlChange()`: permite mandar mensajes de control. El funcionamiento de este tipo de mensajes ya se explicó. Debemos pasarle 3 parámetros: número del control, valor del control y canal.
- `MIDI.sendProgramChange()`: Envía un mensaje de cambio de programa. Esto se utiliza para indicar a un instrumento MIDI o dispositivo que debe cambiar su programa o preset. La función requiere de 2 parámetros de entrada (2 bytes de datos): número del programa y canal.
- `MIDI.sendPitchBend()`: se utiliza para enviar mensajes de pitch bend a través de la interfaz MIDI. El pitch bend permite variar el tono de una nota de manera continua. Requiere de 2 parámetros de entrada: el valor del pitch bend y el canal.
- `MIDI.read()`: esta función se utiliza para verificar y procesar los mensajes MIDI entrantes. Cuando un mensaje MIDI es recibido, `MIDI.read()` lo analiza y, si es un mensaje válido, ejecuta las funciones de callback asociadas o permite al usuario acceder a su contenido. Como

en nuestro caso el microcontrolador no recibe ningún mensaje MIDI por el puerto serie sino que solo emite, esta función no tiene utilidad.

5.6.2 Librería Q2HX711.h

Esta librería permite utilizar de forma sencilla sensores de presión HX711 o parecidos. Si bien el sensor para el que esta diseñada la librería no es el mismo que se ha utilizado, es bastante común para este uso, como se ha podido comprobar en el foro de Arduino. El github de la librería se puede encontrar en el siguiente [enlace](#).

La librería es muy sencilla y escueta. Su uso se resume en lo siguiente:

- Primero hay que definir los pines del Arduino a los que se van a conectar los pines OUT y SCK del sensor.
- Después se define un objeto del tipo Q2HX711 con el nombre que queramos y los pines de la señal de reloj y de los datos.
- Por último para acceder a las lecturas del sensor se utiliza la función `read()` y se guarda la lectura en una variable.

5.6.3 Librería Wire.h

Esta librería permite la comunicación con dispositivos I2C. Esta comunicación se usa en todas las placas de Arduino, si bien en cada placa los pines que lo soportan varían. En el Arduino UNO, los pines de I2C son el A4 (SDA o datos) y A5 (SCL o reloj). Esta librería es necesaria para implementar el módulo de sensores capacitivos MPR121. Podemos encontrar documentación en este [enlace](#).

5.6.4 Librería Adafruit_MPR121.h

Esta librería sirve para controlar el módulo MPR121 de sensores capacitivos. Este módulo se comunica mediante I2C con la placa, y permite hasta 12 sensores. Algunas funciones que pueden ser importantes:

- Lo primero, como siempre es definir un objeto de la clase del sensor, en este caso `Adafruit_MPR121`.
- Función `begin()`: inicia la comunicación con la placa. Como único parámetro, se debe indicar la dirección del dispositivo. Según a qué esté conectado el pin ADDR, se deberá introducir las direcciones 0x5A (si está conectado a tierra), 0x5B (si está conectado a alimentación), 0x5C (si está conectado al pin SDA) y 0x5D (si está conectado a SCL).
- `setThresholds()`: permite ajustar los umbrales que definen cuando el sensor está siendo pulsado y cuando no.
- `touched()`: devuelve un `true` cuando el sensor está siendo pulsado.
- `filteredData()` y `baselineData()`: permiten leer los datos crudos del sensor o los datos filtrados, pero no solamente un nivel alto o bajo. Esto puede servir para ajustar los umbrales de los sensores.

5.6.5 Librería LiquidCrystal.h

Esta biblioteca se utiliza para facilitar la implementación de las pantallas LCD, y se ha usado en algunas asignaturas de la carrera. La librería puede funcionar para modo de 4 u 8 bits, dependiendo de cuantos pines de la pantalla se usen. La versión utilizada ha sido la 1.0.7, disponible [aquí](#).

Las funciones que se han utilizado con un poco de explicación son:

- La clase `LiquidCrystal`: es el tipo de dato con el que se definirá el LCD. Debemos según cuantos pines estemos utilizando, usar más o menos parámetros para definir el objeto.
- `lcd.createChar()`: es una función que permite crear un carácter personalizado en la memoria temporal del display LCD, pudiendo definir hasta 16. Los parámetros que recibe son el número del carácter, que es simplemente un número que identifica al dibujo que crees, y el nombre de la variable tipo `byte` que indica los píxeles que están encendidos. Para diseñar los caracteres personalizados me he ayudado de la herramienta online del siguiente [enlace](#). La tabla de caracteres en memoria del LCD se muestran a continuación, donde podemos ver las posiciones libres donde definir los personalizados.
- `lcd.begin()`: inicializa la interfaz con el display LCD, y recibe como parámetros las dimensiones del display. En este caso, el LCD que se tiene es de 16 columnas por 2 filas. Esta función debe llamarse antes de poder utilizar el LCD.
- `lcd.clear()`: limpia la pantalla de los caracteres que se hubiesen pintado antes. Deja la pantalla sin ningún carácter pintado.
- `lcd.setCursor()`: indica la posición dentro de la matriz de posiciones dentro del LCD de inicio de los que sea que se vaya a escribir. Recordemos que tenemos 16 columnas por 2 filas, esto es desde la posición (0,0) a la (15,1) que serían las esquinas superior izquierda e inferior derecha del LCD.
- `lcd.print()`: permite representar un dato en el LCD. Esto puede ser una cadena de caracteres (`string`) o un solo carácter (`char`), enteros (`int`), tipo `byte`, etc. Muestra los caracteres que conformen el dato que le hayamos pasado.
- `lcd.write()`: hace prácticamente lo mismo que `print()`.
- Otras funciones con las que se ha trasteado pero que al final no se usaron son: `home()` que es un `setCursor()` a la posición superior derecha del LCD. También `scrollDisplayLeft()` y `scrollDisplayRight()`, que me permiten desplazar todos los caracteres una posición a la derecha o a la izquierda.

5.6.6 Librería Adafruit Debounce.h

Esta librería permite facilitar el debounce de los botones. Si bien esta tarea es muy simple de programar, cuando se tiene 4 botones, resulta más fácil y deja el código más limpio. Aunque se acabó usando la librería, cuando en el desarrollo se usaba solo un botón, se programó a mano. Las funciones son muy sencillas, y se puede encontrar documentación [aquí](#).

Un punto negativo de usar la librería, es que no he visto que se pueda cambiar el tiempo de debounce. En cualquier caso las funciones que se han utilizado son:

- En primer lugar crear un objeto de la clase `Adafruit_Debounce`, definiendo el pin al que está conectado y la polaridad.
- `begin()`: inicializa la clase del botón.
- `update()`: lee de nuevo el estado del botón.
- `justPressed()`: indica si se acaba de pulsar el botón devolviendo un `true`.
- `isPressed()`, `isReleased()`, `justReleased()`: son poco más o menos lo mismo. Devuelven un `true` si se cumple la condición que indica su nombre.

Lower 4 Bits	Upper 4 Bits																		
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111		
xxxx0000	CG RAM (1)				0	@	P	`	P					-	9	≡	α	p	
xxxx0001	(2)			!	1	A	Q	a	q					。	ア	チ	△	ä	q
xxxx0010	(3)			"	2	B	R	b	r					「	イ	ツ	×	ß	θ
xxxx0011	(4)			#	3	C	S	c	s					」	ウ	テ	モ	ε	∞
xxxx0100	(5)			\$	4	D	T	d	t					、	エ	ト	ヤ	μ	Ω
xxxx0101	(6)			%	5	E	U	e	u					・	オ	ナ	1	℃	Ü
xxxx0110	(7)			&	6	F	V	f	v					ヲ	カ	ニ	ヨ	ρ	Σ
xxxx0111	(8)			'	7	G	W	g	w					ア	キ	ヌ	ラ	g	π
xxxx1000	(1)			(	8	H	X	h	x					イ	ク	ネ	リ	フ	×
xxxx1001	(2)			)	9	I	Y	i	y					ウ	ケ	ノ	ル	´	y
xxxx1010	(3)			*	:	J	Z	j	z					エ	コ	ハ	レ	j	≠
xxxx1011	(4)			+	;	K	[	k	[					オ	サ	ヒ	ロ	*	π
xxxx1100	(5)			,	<	L	¥	l	¥					ヤ	シ	フ	ワ	φ	π
xxxx1101	(6)			-	=	M	]	m	]					ユ	ズ	ヘ	ン	も	÷
xxxx1110	(7)			.	>	N	^	n	^					ヨ	セ	ホ	°	π	
xxxx1111	(8)			/	?	O	_	o	_					ッ	リ	マ	°	ö	■

Figura 5.24 Tabla de caracteres en la CGROM del display LCD.

6 Desarrollo del trabajo

En este capítulo se van a explicar las diferentes etapas del desarrollo del proyecto, incluyendo aquellos prototipos o montajes que se han ido haciendo y descartando hasta llegar al prototipo final. Los códigos no se van a incluir enteramente en este apartado, sino que estarán en el apéndice de códigos, y si necesitase explicar algo se incluirán fragmentos o los números de línea del código correspondiente.

También se verán los distintos montajes y piezas utilizadas.

6.1 Prototipo inicial básico

La primera iteración del instrumento MIDI consistió en el punteiro únicamente. Para ello se montaron los sensores capacitivos TTP223 en una protoboard, y según cuales estuviesen activados, se genera un mensaje MIDI para reproducir una nota. El funcionamiento del prototipo se puede ver en el siguiente enlace: [Video del prototipo1](#).

El código completo se encuentra en el apartado de los apéndices [A.1](#)

Es muy sencillo. En primer lugar se incluye la librería MIDI, y se definen las notas que se van a utilizar. Estas se corresponden con la octava que la gaita puede reproducir.

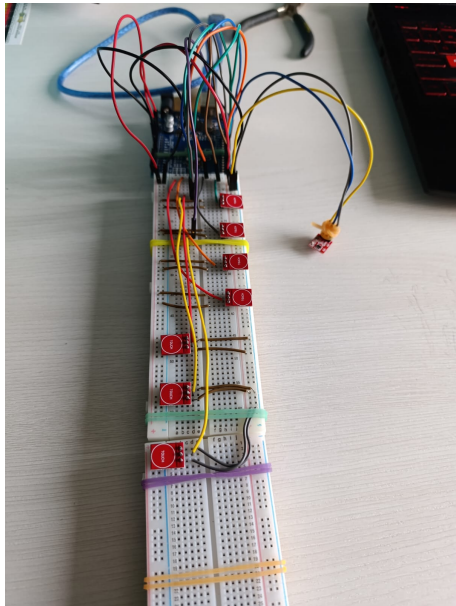
A continuación se definen los pines donde van conectados los sensores capacitivos y se crean sus variables de estado, con el nombre del número de agujero que corresponde. Los pines utilizados son los digitales desde el 2 al 9.

Ya dentro del void setup se indica que los pines donde están conectados los sensores serán de entrada y comenzamos la comunicación MIDI mediante el puerto serie a 115200 baudios. También se asocia el valor de cada nota a su valor MIDI, teniendo en cuenta la tabla de valores que se mostró anteriormente.

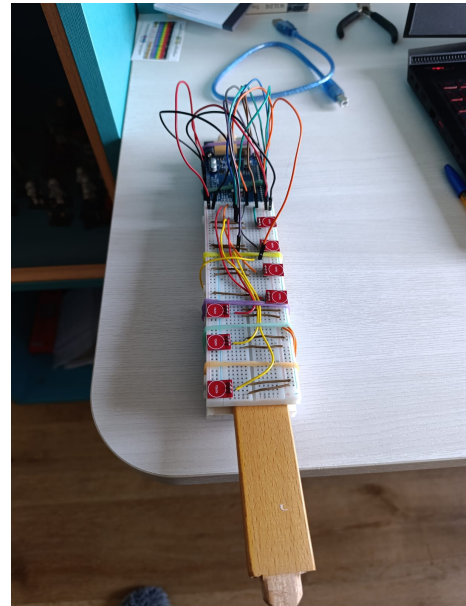
Dentro del void loop se lee el estado de los botones, y en función de qué botones estén pulsados, se envía un mensaje MIDI de la nota que corresponda. Como fué el primer prototipo y todavía no tenía claro el funcionamiento ni del protocolo MIDI ni del propio funcionamiento que iba a tener el instrumento, vemos que cada vez se manda un mensaje Note ON, después se aplica un delay de 10 ms y se manda un Note OFF. Esto es bastante poco eficiente, y en los prototipos posteriores se cambiaría para no estar todo el rato mandando mensajes.

También se debe comentar que el mapeo de los agujeros es parcial. Esto significa que todas las combinaciones de tapado de agujeros y su nota asociada no están contempladas en este primer código. Sin embargo es apropiado como primera aproximación, y más adelante se completará.

Podemos ver el aspecto del prototipo en la figura [6.1](#). En el vídeo del enlace indicado antes podemos ver algunos de los problemas que se han indicado, como por ejemplo que a pesar de estar pulsando algunos botones, estos se desactivan, o que después de levantar el dedo de algunos de ellos, estos siguen activados durante un periodo de tiempo.



(a) Montaje inicial con sensores TTP223.



(b) Vista superior del montaje inicial con botón inferior.



(c) Vista desde abajo del montaje inicial con botón inferior.

Figura 6.1 Imágenes del montaje inicial.

6.2 Prueba de los sensores de presión

El siguiente paso dentro del desarrollo del proyecto fue probar los sensores de presión HX710B. Para ello se utilizó un montaje que dejara ver los valores que devolvía el sensor cuando aumentaba la presión mediante la compresión del aire dentro de una jeringuilla. En las figuras 6.2 y 6.3 se puede observar claramente.

El código utilizado es muy sencillo, y permite observar mediante el Serial Plotter una gráfica de como varía en el tiempo las lecturas del sensor según comprimo más o menos el aire en la jeringuilla. Se encuentra en el siguiente apéndice: [A.2](#)

En la gráfica devuelta por el serial Plotter, se puede observar como varía la medida de presión cuando comprimimos el émbolo de la jeringuilla. El siguiente paso es verificar que este sensor sirve para la tarea de distinguir cuando se está soplando por un tubo y cuando no. Para ello se realizó mediante un tubo de PVC la primera iteración del soprete, que más tarde sería sustituida por una

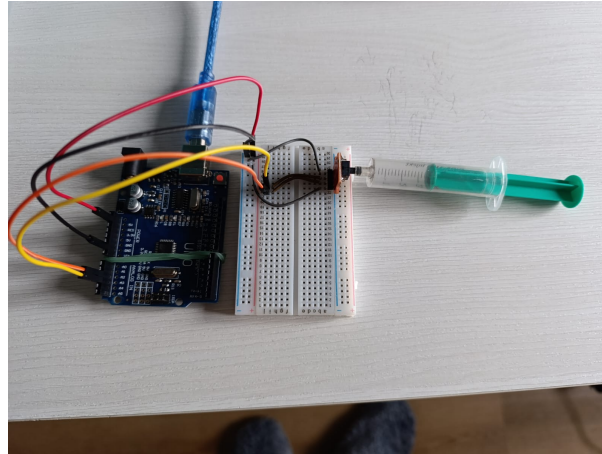


Figura 6.2 Montaje de prueba del sensor de presión.



Figura 6.3 Gráfica de las lecturas de presión del sensor.

pieza impresa en 3d.

Esta pieza consiste en un tubo de PVC, que en su parte superior tiene una pieza de madera que permita aguantar el soprete en los labios. En su parte inferior hay un agujero que permite el montaje del sensor, y en el final del tubo, este está parcialmente tapado. Esto es para imitar la resistencia al soplo que habría en una gaita normal.

En el vídeo de este enlace, se puede ver el funcionamiento del sensor de presión dedicado al soplo, junto con otras características como el encoder que se explicará a continuación: [Vídeo del prototipo 1 con fol virtual](#).



Figura 6.4 Primera iteración del soprete.

6.3 Prueba del encoder digital

El siguiente paso fue probar algún código sencillo para ver si el encoder rotatorio digital que había comprado funcionaba correctamente. Las conexiones del mismo fueron, a parte de la alimentación y la tierra, el pin S1 del encoder al pin digital 2 del micro, y el pin S2 del encoder al pin digital 3. Esto se eligió así debido a que estos dos pines del micro soportan interrupciones, y para el funcionamiento del encoder se van a utilizar.

Voy a ir explicando el código utilizado para probar el encoder, mediante el cual se controla el volumen de un instrumento MIDI. El código completo se puede encontrar en el apéndice de códigos [A.3](#)

Código 6.1 Fragmento de código de programación de encoder.

```
#include <MIDI.h>

#define ENC_A 2 //definimos los pines del encoder digital
#define ENC_B 3
MIDI_CREATE_DEFAULT_INSTANCE(); //creamos instancia MIDI

unsigned long _lastIncReadTime = micros();
unsigned long _lastDecReadTime = micros();
int _pauseLength = 25000;
int _fastIncrement = 10; //cuanto avanza si giramos rapido el encoder
volatile int counter = 0; //variable del giro del encoder
```

En primer lugar se incluye la biblioteca MIDI para poder mandar los mensajes de control del volumen. Se definen los pines donde conectaremos el encoder y se crea una instancia MIDI. A continuación se crean las variables donde se guardarán datos de tiempo entre las lecturas. En la variable counter se guarda el valor del contador que incrementamos mediante el encoder.

Código 6.2 Fragmento de código de programación de encoder.

```

void setup() {

    pinMode(ENC_A, INPUT_PULLUP); //definimos pines del encoder como
        entradas
    pinMode(ENC_B, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(ENC_A), read_encoder, CHANGE);
        //habilitamos las interrupciones en los pines del encoder
    attachInterrupt(digitalPinToInterrupt(ENC_B), read_encoder, CHANGE);

    MIDI.begin(); //comenzamos la comunicación MIDI
    Serial.begin(115200);
}

```

Dentro del void setup definimos los pines donde se conecta el encoder como entradas. Se habilitan las interrupciones en ambos pines, indicando la función de interrupción que se explicará más adelante. Por último se comienza la comunicación MIDI como siempre.

Código 6.3 Fragmento de código de programación de encoder, void loop.

```

void loop() {
    static int lastCounter = 0;
    MIDI.sendNoteOn(60, 60, 1); //mandamos una nota para comprobar el
        volumen

    if (counter != lastCounter) { //si el valor de counter cambio
        lastCounter = counter;
        MIDI.sendControlChange(1, counter, 1); //cambiamos el volumen
    }
}

```

En el void loop lo que se hace es mandar una nota continua para poder apreciar si el volumen está cambiando o no. Después, si el valor del contador varía eso significa que el encoder se ha girado y se envía un mensaje de control MIDI al canal 1, de valor igual al contador. Recordemos que el campo del valor de un mensaje de Control Change era un valor entre 0 y 127, con lo cual debemos saturar la variable counter. Esto se hace en la función de interrupción.

Código 6.4 Rutina de interrupción del encoder.

```

void read_encoder() {
    //rutina de interrupcion para los 2 pines del encoder. Se utiliza la
        variable counter
    //para ver cuanto ha girado, y se actualiza si se cumplen ciertas
        condiciones

    static uint8_t old_AB = 3;
                                                                    // Lookup
        table
    static int8_t encval = 0;
                                                                    // calor
        del encoder

```

```

static const int8_t enc_states[] = { 0, -1, 1, 0, 1, 0, 0, -1, -1, 0,
                                     0, 1, 0, 1, -1, 0 }; // Lookup table

old_AB <= 2; // recordamos estado anterior

if (digitalRead(ENC_A)) old_AB |= 0x02; // estado actual del pin A
if (digitalRead(ENC_B)) old_AB |= 0x01; // estado actual de pin B

encval += enc_states[(old_AB & 0x0f)];

//actualizamos counter si el encoder ha girado un tramo completo, por
//lo menos 4 pasos
if (encval > 3) { // si hemos avanzado 4 pasos
    int changevalue = 1;
    if ((micros() - _lastIncReadTime) < _pauseLength) {
        changevalue = _fastIncrement * changevalue;
    }
    _lastIncReadTime = micros();
    counter = counter + changevalue; //actualizamos counter
    if (counter < 0) {
        counter = 0; //debemos saturar para que no de valores negativos
    }
    if (counter > 127) {
        counter = 127; //debemos saturar para que no de valores mayores de
        127
    }
    encval = 0;
} else if (encval < -3) { //si hemos retrocedido 4 pasos
    int changevalue = -1;
    if ((micros() - _lastDecReadTime) < _pauseLength) {
        changevalue = _fastIncrement * changevalue;
    }
    _lastDecReadTime = micros();
    counter = counter + changevalue; //actualizamos counter
    if (counter < 0) { //saturacion
        counter = 0;
    }
    if (counter > 127) { //saturacion
        counter = 127;
    }
    encval = 0;
}
}

```

Esta es la función de interrupción del encoder. Esta función es la que gestiona completamente el funcionamiento del encoder, devolviendo solo el valor del contador. Voy a explicarla en profundidad puesto que es más complicada.

Primero se definen unas variables del tipo static, lo que significa que se definen la primera vez que se ejecute la función, y luego son constantes. Después se define la variable old_AB que guarda el estado del encoder. Es igual a 3 porque 3 en número binario de 8 bits es igual a 0000 0011 que se

correspondería con el estado rest del encoder.

Cuando se ejecuta la función, desplazamos el valor de S1 y S2 dos posiciones hacia la izquierda para dejar espacio para guardar los valores actuales del encoder.

Código 6.5 fragmento de la rutina de interrupción del encoder.

```
if (digitalRead(ENC_A)) old_AB |= 0x02; // estado actual del pin A
if (digitalRead(ENC_B)) old_AB |= 0x01; // estado actual de pin B
```

De esta forma voy guardando tanto el estado actual como los anteriores, y puedo identificar en que sentido estoy girando el encoder. Para ello se comparan los últimos 4 bits del encoder y se traducen en una posición de la LUT definida. Hay que tener en cuenta que las posiciones de la LUT que valen 1 significan un desplazamiento hacia la derecha, y las -1 hacia la izquierda.

Después mediante la condición de que encvalue sea mayor que 3 o menor que -3, se aumenta la variable del contador en 1. Si este cambio se hubiese producido en un tiempo menor que la constante `_pauseLength`, entonces se aumentaría el contador con un factor de 10.

Por último, debemos saturar el valor máximo y mínimo que puede tomar el contador para adaptarlo a los 7 bits que pueden ocupar el valor del volumen en un mensaje midi tipo Control Change. Esto es como ya sabemos, que el valor del contador debe estar entre 0 y 127.

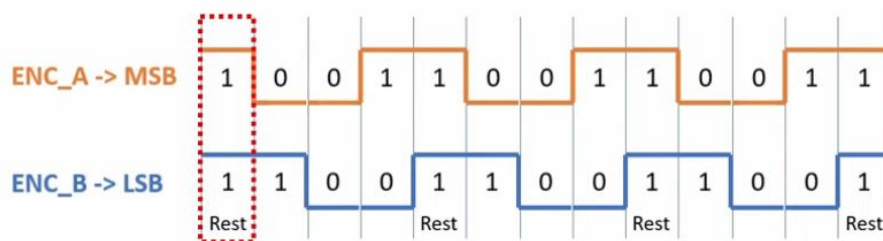


Figura 6.5 Tabla de valores de S1 y S2 del encoder digital.

6.4 Prototipo 1 final

Ya habiendo probado los sensores capacitivos, los sensores de presión y el encoder, se realizó el montaje de los que vamos a denominar como el prototipo 1. Este no es más que el aglutinar los sensores que se han usado hasta este momento, dándoles el uso que van a tener en la iteración final. Por tanto el código no se va a explicar con mucha profundidad puesto que ya se han explicado sus partes por separado. Podemos encontrarlo en el apéndice [A.4](#)

El funcionamiento del prototipo sería el siguiente: Mediante un botón, que en este caso sería el botón que ofrece el propio encoder digital, se activa el sonido de los bordones, o roncón. Girando el encoder digital, se varía el volumen general del sintetizador.

Mediante el sensor de presión, se mide si se esta soplando a través del tubo de PVC que mencionamos antes. Si la presión es mayor que un determinado umbral, se incrementa un contador. Si la presión es inferior a ese umbral, se decrementa este contador hasta que llega a cero. Esto sirve para modelar la bolsa de aire de la gaita, de forma que si el contador de aire fuese cero, las notas no sonarían.

Por último, mediante la pulsación de los sensores capacitivos se varía el valor de la variable nota, que dependiendo del valor del contador de aire, se enviaría como mensaje MIDI o no.

Algunos aspectos importantes del código pueden ser:

Código 6.6 Definición de variables del volumen de aire.

```
const int volmax = 1000;
double volaire = 0;
```

Como volumen de aire máximo del fol se ha puesto 1000, que equivaldría a un litro de aire. Mediante la variable volaire, se almacena el volumen virtual que hay dentro del fol.

Código 6.7 Código de inflado del fol virtual.

```
if (hx711.read() / 1000.0 > 10500) {
    volaire = volaire + 12;
    if (volaire >= 1000) {
        volaire = 1000;
    }
} else {
    volaire = volaire - 6;
    if (volaire < 0) {
        volaire = 0;
    }
}
```

Dentro del void loop tenemos la función que define cómo se llena el aire virtual. Si la presión supera el umbral, se aumenta en 12 el contador, y si no se decrementa a la mitad de ese valor.

Por último mencionar como se hace el debounce del botón. El tiempo de delay es de 50 milisegundos. Primero se lee el estado actual del botón. Si el estado leído es diferente del último estado, actualiza lastDebounceTime con el tiempo actual dado por la función millis(). Si ha pasado más tiempo que debounceDelay desde el último cambio de estado, actualiza el estado del botón. Si el estado del botón ha cambiado y es HIGH, envía el mensaje de control MIDI para activar el ronco. Por el contrario si el flanco ha sido de bajada, lo desactiva.

Código 6.8 código de debounce de un botón.

```
int reading = digitalRead(buttonPin); //programacion boton
if (reading != lastButtonState) {
    lastDebounceTime = millis();
}

if ((millis() - lastDebounceTime) > debounceDelay) { //debounce del
    boton
    if (reading != buttonState) {
        buttonState = reading;
        if (buttonState == HIGH) {
            ronco = HIGH;
            MIDI.sendControlChange(3, 127, 1); //mando contro change para el
            volumen de valor HIGH
        } else if (buttonState == LOW) {
            MIDI.sendControlChange(3, 0, 1); //mando contro change para el
            volumen de valor LOW
        }
    }
}
```



```
}  
lastButtonState = reading;
```

El resto del código ya se ha explicado en las secciones anteriores, y no tiene mayor interés.

Como se puede apreciar en el video a continuación, su funcionamiento es bastante malo por culpa de los sensores capacitivos TTP223. Sin embargo se comprueba que tanto el control del volumen como la medición de la presión para distinguir el soplido funcionan. [Video del prototipo 1 con fol virtual](#).

6.5 Prueba de sensores capacitivos caseros

Después de comprobar que los sensores TTP223 no eran apropiados para la realización del instrumento, se intentó utilizar unos sensores capacitivos caseros.

El montaje en este caso consistió en un tubo de PVC con tornillos en las posiciones de los agujeros, que estaban soldados a los cables que se conectaban a la placa. Haciendo uso de la librería `CapacitiveSensor.h`, se puede obtener una medida de la capacitancia cuando se tocaba un tornillo. Sin embargo, es necesario que cada sensor utilice 2 pines de la placa, un sender y un receiver. Además, las lecturas devueltas oscilaban mucho, y no era posible establecer un umbral que separase completamente la pulsación de la no pulsación.

Para solucionar esto se programó un filtro de la media, que obtenía los 4 últimos valores de capacitancia recibidos y hacía una media, que era lo que después se comparaba con el umbral. A pesar de esto, la medida seguía sin ser binaria, por lo que su uso se descartó en favor del módulo de sensores capacitivos MPR121.

El código correspondiente se encuentra en el apéndice [A.5](#).

6.6 Prueba del módulo de sensores capacitivos MPR121

Finalmente, después de descartar las demás opciones de implementación de sensores capacitivos, se llegó a este módulo. Es bastante fácil de usar, aunque si es cierto que en ocasiones falla. Como primera prueba se hizo uso de un programa de ejemplo incluido en la librería `Adafruit_MPR121.h` llamado `MPR121test`. A partir de este ejemplo se desarrolló el código de prueba del módulo que podemos ver en el apéndice [A.6](#). El código es bastante fácil, y permite ver por el monitor serie cual de los electrodos está siendo tocado.

Se pudo comprobar que todas las entradas de electrodos funcionaban correctamente.

6.7 Prueba del sensor de presión conectado a bolsa de aire

Una vez pudimos ver que el soplido se podía detectar mediante el sensor de presión adquirido, se comprobó que ese mismo modelo de sensor pudiese detectar un aumento de la presión en una bolsa de aire.

Para ello, en un inicio se utilizó el fol de una gaita real. Mostrando las medidas que devolvía el sensor por el serial plotter se pudo comprobar que su rango era también válido para esta función. Entonces se empezaron a barajar opciones de como implementar una bolsa de aire.

La idea final fue, mediante una bolsa de agua caliente de goma, que estuviese sellada, simular el fol del instrumento. El sensor iría conectado a la bolsa, pudiendo detectar así la presión ejercida con el brazo. Como la bolsa no es perfectamente estanca, se debió incluir también una válvula de reinflado. Sin embargo esto trajo consigo ciertos problemas de cara a la medición de la presión, puesto que esta variaría dependiendo de lo inflada que este la bolsa, dificultando la comparación

con umbrales. Para comprobar las medidas devueltas en las diferentes condiciones de inflado se utilizó el código disponible en el apéndice [A.7](#).

La solución a esto se puede ver en el código del programa de prueba (apéndice [A.9](#)). Lo que se hace es tomar una medida inicial de la presión, y establecer los umbrales en base a variaciones sobre esta medida inicial. Al probar este método se consiguió el resultado deseado.

6.8 Prueba de la pantalla

Al igual que con el resto de sensores, se comprobó que la pantalla LCD funcionaba correctamente. Para ello se conectaron los pines necesarios y se escribió un código de prueba. Este está disponible en el apéndice [A.10](#). En primer lugar se crean los caracteres personalizados de la forma en que se indicó en el apartado de la librería LiquidCrystal, que serán un símbolo de una nota musical y 4 flechas para indicar las direcciones de los botones. Después simplemente creamos una función `pantallaInicial()` que muestre el texto deseado, y la llamamos en el void `setup`.

7 Gaita electrónica MIDI. Resultado final

En este capítulo se verá el resultado final conseguido. Esto incluye las conexiones finales de los sensores al microcontrolador, los aspectos del montaje como piezas, soldaduras, etc, además del código final y el funcionamiento final del instrumento electrónico. A continuación se verán por separado cada uno de estos aspectos.

7.1 Conexiones eléctricas y soldaduras

En primer lugar se conectaron todos los sensores necesarios para el funcionamiento del instrumento. El esquemático de la conexión se incluye en el apéndice B. Voy a explicar brevemente todas las conexiones:

- Sensor de presión dedicado al fol: conectado a 5V y a GND. Pin OUT a A0 y pin SCK conectado a A1.
- Sensor de presión dedicado al soprete: conectado a 5V y a GND. Pin OUT a A2 y pin SCK conectado a A3.
- Encoder: conectado a 5V y a GND. Pin S1 conectado al pin digital 2. Pin S2 conectado al pin digital 3. Pin Key desconectado.
- Pulsadores: uno de los pines conectado a tierra, el otro conectado a los pines digitales desde el 10 al 13, en paralelo con la alimentación a 5V a través de una resistencia de 10K ohmios.
- Módulo de sensores capacitivos MPR121: Pines IRQ, ADDR y GND a tierra. Pin de alimentación a 3.3V. Pin SDA al pin analógico de la placa A4, y pin SCL del módulo a A5.
- Conector MIDI hembra: Pin 4 a través de una resistencia de 220 ohmios al pin TX de la placa. Pin 2 a GND. Pin 5 a través de una resistencia de 220 ohmios a 5V.
- Pantalla LCD: Pin VSS a GND, pin VDD a alimentación de 5V. Pin VO a través de una resistencia de 1k ohmio a GND. Pin a el pin digital 8 del micro. Pin RW a GND. Pin E al pin digital 9. Pines D0, D1, D2, D3 desconectados. Pines D4, D5, D6 y D7 conectados a los pines digitales 4, 5, 6 y 7 consecutivamente. Pin A a través de una resistencia de 220 ohmios a 5V. Pin K a GND.

Con estas conexiones todos los pines digitales y analógicos de la placa quedan usados y conectados. Para poder hacer estas conexiones de forma cómoda y duradera, se soldaron la mayoría de ellas. Para ello se utilizaron los pines del micro sin conector DuPont y unas pequeñas placas PCB.

En la figura 7.1 podemos ver la parte inferior de la placa PCB donde se soldaron los botones y las resistencias necesarias para su funcionamiento. En la figura 7.2 vemos la parte superior de la misma PCB con los pulsadores expuestos.

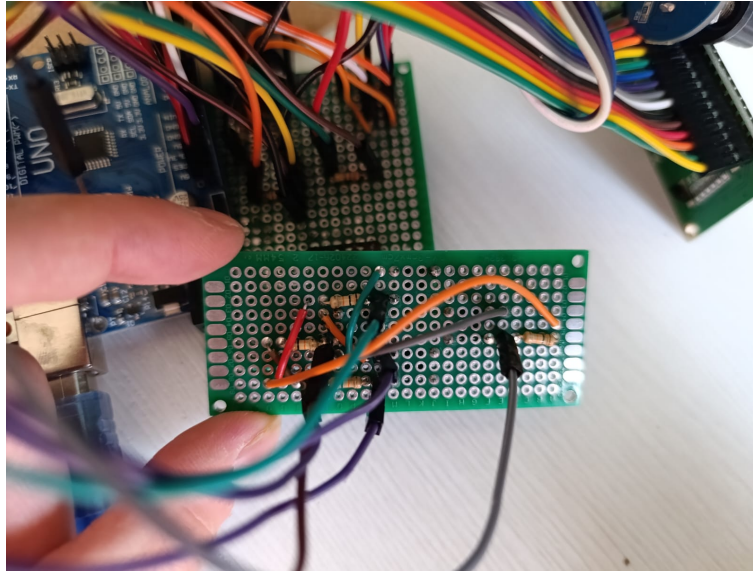


Figura 7.1 Vista inferior de la PCB con los pulsadores.

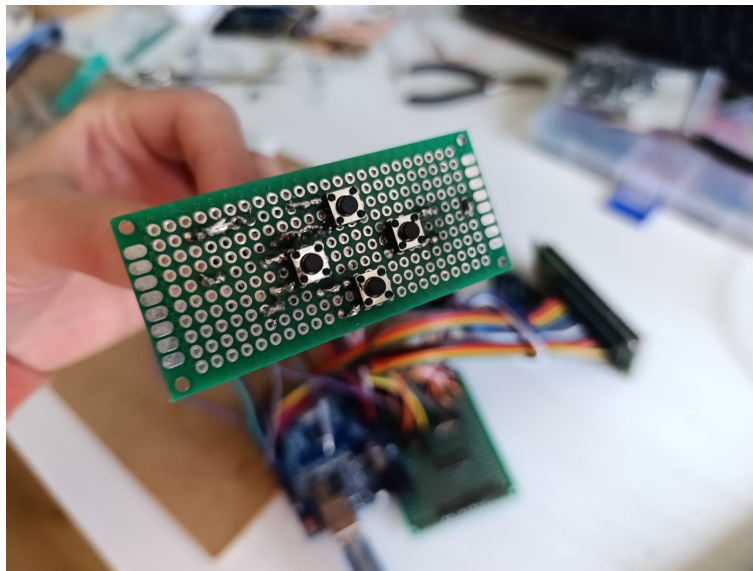


Figura 7.2 Vista superior de la PCB con los pulsadores.

En la figura 7.3 vemos la conexión con el encoder. Para esta conexión no se hizo ninguna soldadura, sino que se utilizaron conectores DuPont pegados entre sí, para evitar que se desconecten. De esta misma forma se han conectado los sensores de presión, si bien antes se soldaron los conectores pin header puesto que el sensor venía sin conectores.

Para la conexión de la pantalla se soldaron los cables DuPont tipo macho-macho directamente a los pines del LCD. Los cables que no se utilizaron se cortaron para liberar espacio y que sea más sencillo.

En el módulo MPR121 se soldaron unos pin header, y después se pegaron unos cables DuPont hembra-macho para las conexiones con la placa, y los cables que vienen de los electrodos se soldaron en sus correspondientes pines del módulo.

El conector MIDI hembra se soldó en una placa PCB más grande, donde después se realizarían ciertas conexiones intermedias que ahora veremos.

Sobre esta placa PCB se realizaron algunas conexiones intermedias, como por ejemplo las

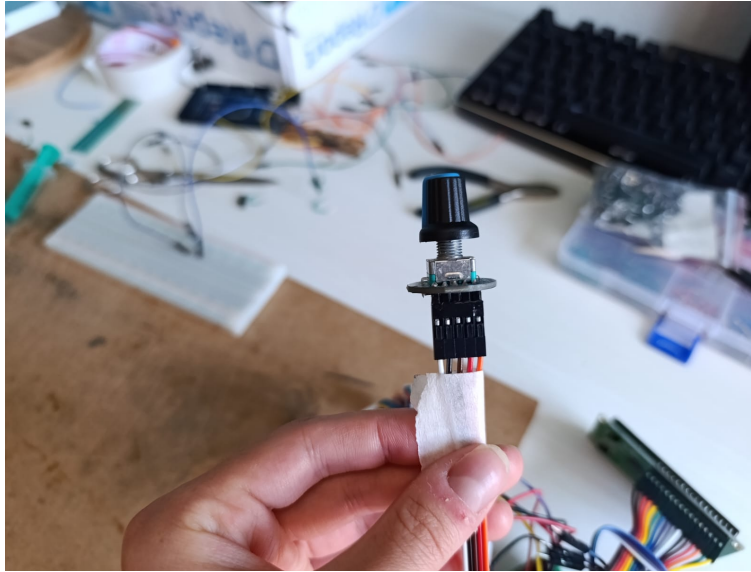


Figura 7.3 Encoder rotatorio conectado.

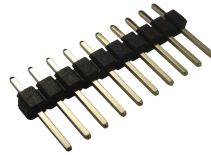


Figura 7.4 Pin header.

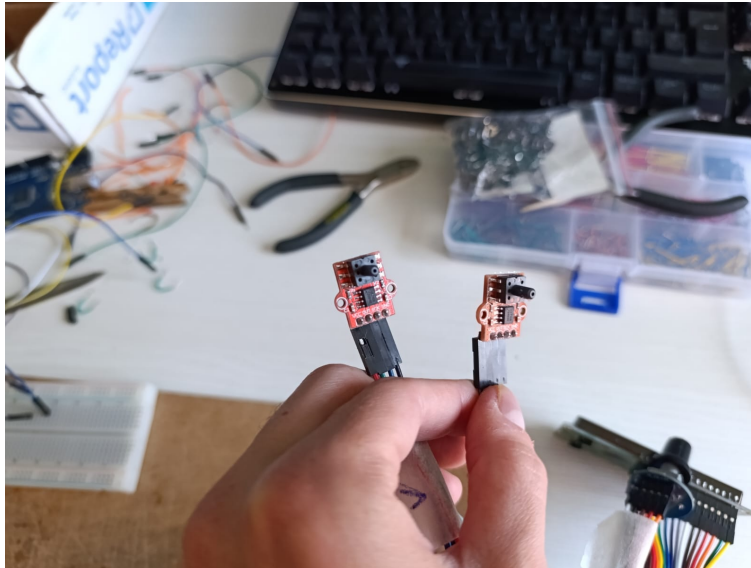


Figura 7.5 Conexión de los sensores de presión.

resistencias a través de las cuales se conectan a alimentación o a tierra algunos pines de la pantalla. También se soldaron aquí sobre una misma línea las alimentaciones en paralelo de la mayoría de los sensores, puesto que todos utilizan 5V y en la placa solo tenemos 2 pines de alimentación a 5. Lo mismo se hizo con las tierras.

Después de soldar los sensores necesarios en la placa PCB, se llevan cables desde la PCB hasta los pines del micro, donde se sueldan también para evitar que se desconecten y puedan provocar

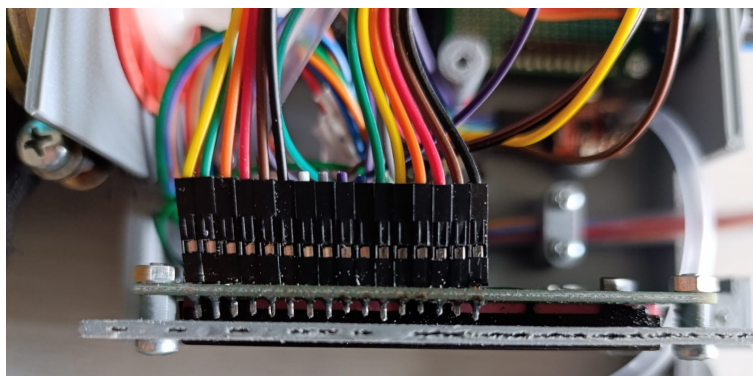


Figura 7.6 Soldadura de los cables de la pantalla.

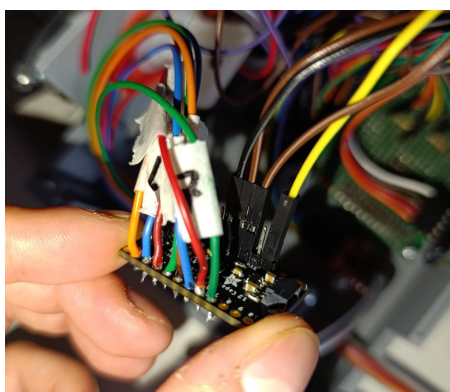


Figura 7.7 Conexiones con el módulo MPR121.

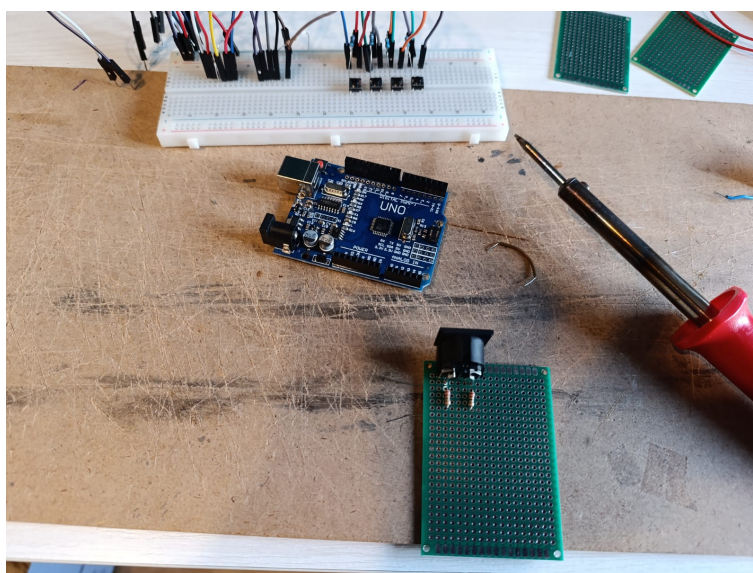


Figura 7.8 Conector MIDI hembra sobre la placa PCB.

fallos en el funcionamiento.

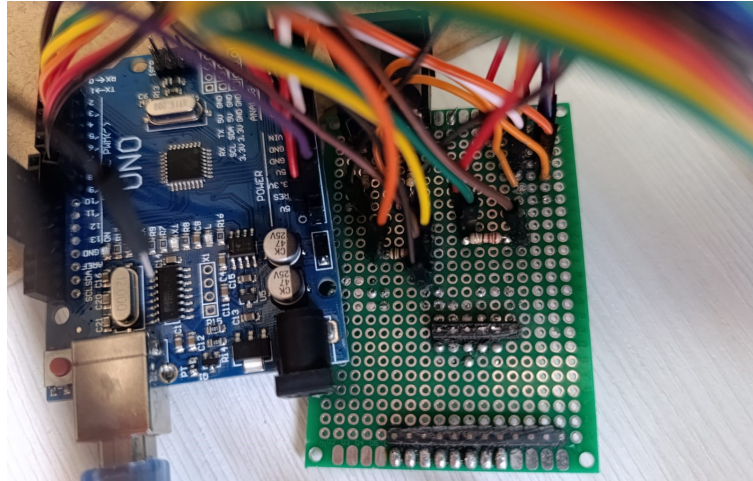


Figura 7.9 Soldaduras sobre la placa PCB.

7.2 Modelado e impresión de piezas

Una vez tenemos el micro conectado a la PCB y a todos los sensores, debemos pensar en un montaje. La primera pieza del montaje, que posteriormente se tuvo que diseñar e imprimir fue lógicamente el punteiro.

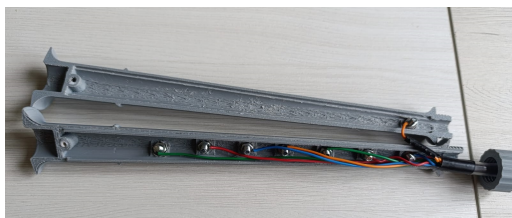
7.2.1 Punteiro

Utilizando la herramienta FreeCad, se diseñaron varias versiones de punteiro. La idea de esta pieza es un tubo con una serie de agujeros que permitan insertar unos tornillos que actuarán de electrodos para el módulo de sensores capacitivos. Para definir la distancia entre los agujeros así como otras características se utilizó un punteiro de una gaita real, tomando medidas sobre el mismo.

Para facilitar el montaje se divide la pieza en dos mitades de forma longitudinal. Además, debemos tener en cuenta que ambas mitades deben sujetarse la una a la otra sólidamente, para lo que se incluyen unos soportes de tornillos. La boca superior del punteiro debe tener un diámetro suficiente para permitir la salida de los cables, además del grosor extra añadido por una cubierta de los mismos.

Además debido a la cercanía de los agujeros superiores con la boca superior del punteiro, se planteó la idea de unir ambas mitades mediante una pieza en forma de tuerca. Con todas estas especificaciones y buscando el aspecto estético del punteiro de una gaita real, se hicieron varios modelos.

Este primer modelo se descartó debido a que la boca superior no dejaba pasar los cables por ser demasiado estrecha.



(a) Vista inferior de la primera versión del punteiro.



(b) Vista superior de la primera versión del punteiro.

Figura 7.10 Vistas de la primera versión del punteiro.

Corrigiendo este fallo y cambiando un par de aspectos estéticos, se creó el segundo modelo de punteiro, que sería el definitivo del instrumento. El modelo 3d en formato .FCStd se encuentra en la

entrega de este trabajo. Los planos de todas las piezas están adjuntados en el apéndice planos de esta memoria. Ahí podemos encontrar las medidas exactas de todas las piezas, incluida esta. Por tanto, las dimensiones de la pieza se pueden encontrar en el apéndice B.

El modelado en FreeCad no tiene mucha explicación. Primeramente se crea un sketch con la forma de la pared del punteiro, después se revoluciona alrededor de un eje, creando el tubo cilíndrico. Cabe mencionar que FreeCad necesita que todas las medidas de los objetos queden definidas, lo que se puede hacer un poco complicado (constraints). Se hacen los agujeros para los electrodos y los tornillos, utilizando primero un diámetro más grande y después atravesando con otro menor mediante la herramienta "agujero". Esto para que los tornillos no queden expuestos, sobre todo los que sujetan las piezas. Después se añade la rosca mediante la herramienta "hélice sustractiva" y se corta longitudinalmente para trabajar de la misma forma la otra mitad del punteiro.



Figura 7.11 Sketch base del punteiro.

Una vez modeladas las dos mitades, exportamos el archivo en formatos .stl para poder abrirlo mediante PrusaSlicer. En este programa configuras distintas opciones de la impresión, como el relleno del objeto, los apoyos, etc y se crea el archivo .gcode que utiliza la impresora. Finalmente se imprime la pieza. Una vez está impresa, retiramos los apoyos y lijamos cualquier imperfección que haya quedado.

Con las piezas ya impresas, insertamos los tornillos de los agujeros, soldamos un cable a cada tornillo y juntamos ambas piezas mediante los tornillos de sujeción. El resultado serían las figuras 7.24.



(a) Pieza final del punteiro, vista superior.



(b) Pieza final del punteiro, vista inferior.

Figura 7.12 Vistas de la versión final del punteiro.

7.2.2 Tuerca

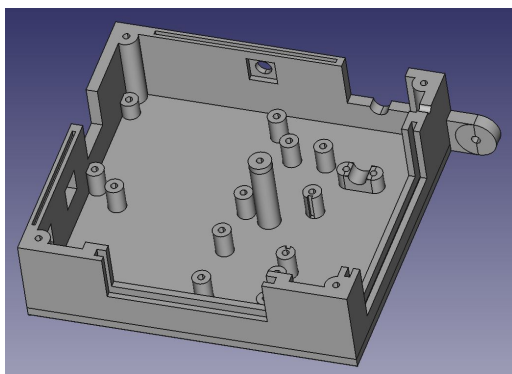
A continuación se modela la tuerca que mantendrá sujeta la parte superior del punteiro. El proceso de construcción no presenta ninguna novedad, por lo que se obviará. El plano de la tuerca se encuentra en el apéndice B.



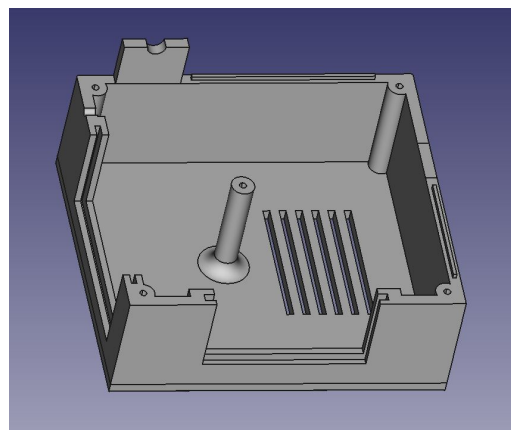
Figura 7.13 Tuerca de sujeción.

7.2.3 Caja

Terminado el punteiro, se pasó al modelado de la caja que contendría los componentes electrónicos. Esta es seguramente la pieza más complicada. La idea es diseñar dos piezas que encajen entre sí y que conformen la caja. En la pieza inferior se atornillarían los componentes electrónicos, mientras que en la superior se encontrarían los soportes de los tornillos que mantendrán las dos piezas unidas. Algunos componentes encajarán en unas ranuras comunes a las dos piezas. Además, se deben dejar agujeros para el encoder, los tubos de cables y los conectores de salida del propio Arduino y MIDI. La distancia entre los soportes de los tornillos que sujetan la PCB, el micro, los sensores de presión y el MPR121 así como la disposición general de los elementos se hizo midiendo las dimensiones de cada elemento y todos juntos, de forma que ocuparan el menor espacio posible. Con todo esto, se diseñaron las piezas de la caja. Los planos se encuentran en el apéndice B.



(a) Pieza inferior de la caja.



(b) Pieza superior de la caja.

Figura 7.14 Vistas de las piezas de la caja.

Mediante las vistas de la parte inferior de la caja, podemos ver la disposición de los distintos elementos.

Para agarrar la cubierta de los cables del punteiro, se utilizará un soporte especial junto con una pieza que se atornille encima.

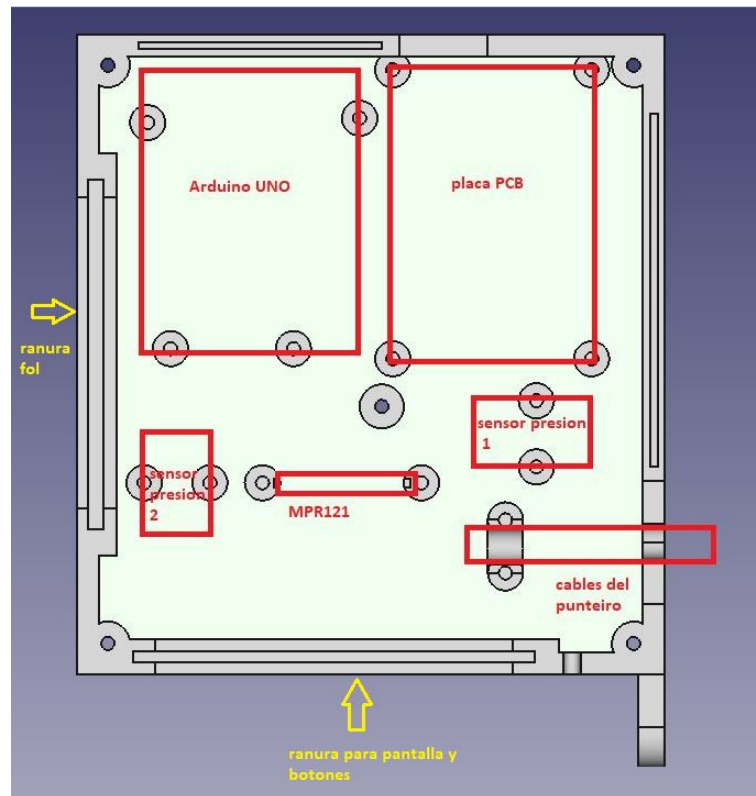


Figura 7.15 Disposición de los elementos dentro de la caja.

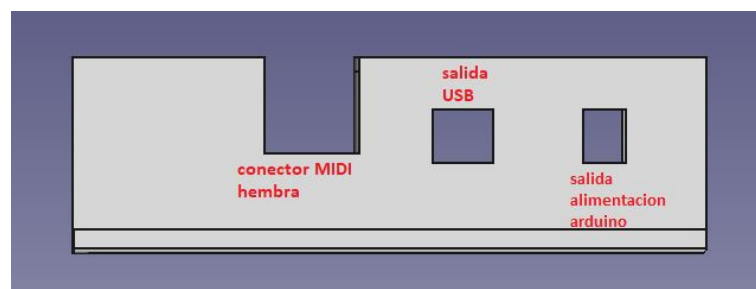


Figura 7.16 Agujeros de salidas de la caja.

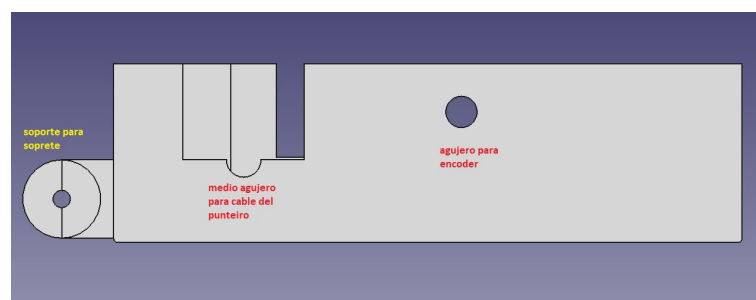
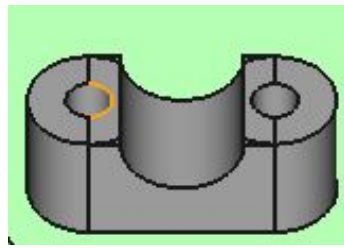


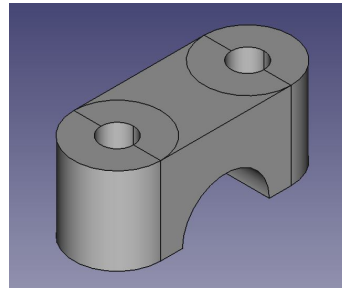
Figura 7.17 Más elementos de la caja.

7.2.4 Soportes de la pantalla, los botones y el fol

La pantalla, los botones y el fol irían empotrados en una plancha fina que se encajaría dentro de las ranuras que están presentes en las dos piezas de la caja. Por ello se diseñaron las dos planchas



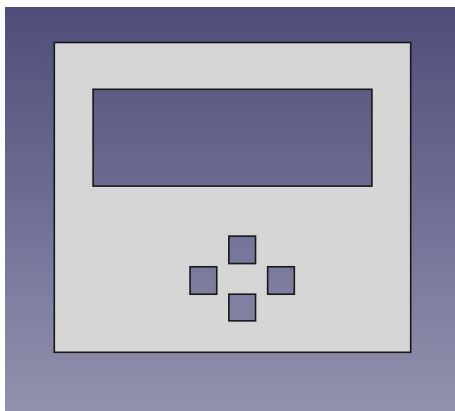
(a) Pieza 1 del soporte del punteiro.



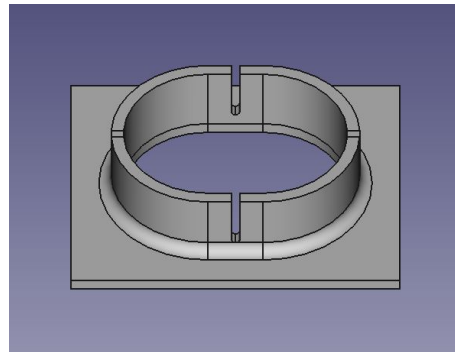
(b) Pieza 2 del soporte del punteiro.

Figura 7.18 Vistas de la pieza de soporte del punteiro.

dedicadas, una al fol y otra a la pantalla junto con los botones.



(a) Plancha para pantalla y botones.



(b) Conector del fol.

Figura 7.19 Piezas que soportan pantalla, botones y fol.

7.2.5 Soprete

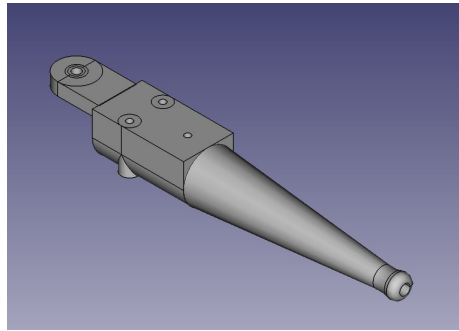
La última pieza que se fabricó fue la pieza del soprete. La idea es un tubo con una boquilla adecuada, que permita ser sujeta con la boca, y que se encuentre a una distancia pequeña de la cara del usuario. Esto es para no tener que hacer ningún movimiento antinatural para poder alcanzar la boquilla para soplar. El soprete debe sujetarse a la caja y tener alguna articulación que permita ajustar la distancia al interprete. Al soplar por su extremo, el aire debe escaparse hacia el exterior ofreciendo una resistencia al soplido parecida a la del instrumento real. En el camino de salida se encontrará el sensor de presión, de forma que pueda medir si se está soplando o no.

Los planos de las piezas del soprete se encuentran en el apéndice B.

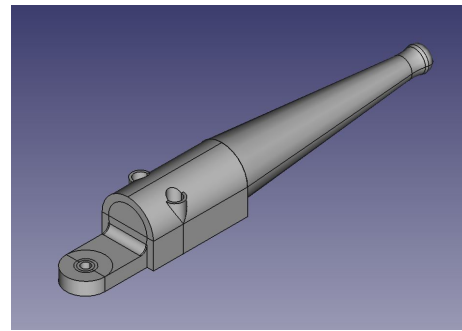
La pieza del soprete incluye también una caja dentro de la cual en un principio iba a ir montado el sensor de presión. Sin embargo después de soldar los componentes electrónicos, la longitud de los cables no dejaba llegar al sensor hasta este punto. Por ello se utilizó un tubo de catéter que va desde el agujero de la pieza del soprete hasta el sensor de presión que está dentro de la caja.

7.2.6 Bolsa de aire o fol

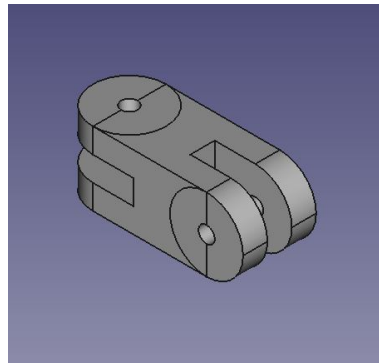
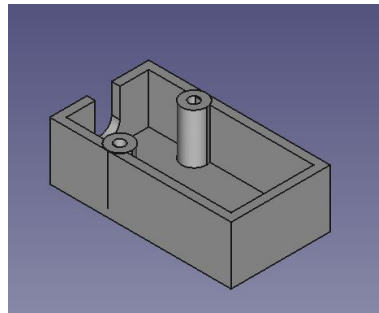
Aunque esta pieza no fue fabricada por mi utilizando FreeCad e impresión 3d, también fue necesaria cierta manipulación. Para implementar el fol se utilizó una bolsa de agua caliente, que se llena de aire. Las posibles salidas de aire se sellaron para evitar pérdidas, y se instaló una válvula de bicicleta



(a) Vista inferior del soprete.



(b) Vista superior del soprete.

Figura 7.20 Vistas de la pieza del soprete.**Figura 7.21** Pieza de articulación del soprete.**Figura 7.22** Caja del soprete para montaje del sensor.

para poder volver a inflarla de aire, debido que unas pérdidas mínimas son inevitables. Para sujetar el fol a la caja se utilizó la pieza antes mencionada de conector de fol. Además se fabricó una funda o "vestido" al fol por razones estéticas, que se sujeta en la posición adecuada con una brida en un extremo y un velcro en el otro.

7.3 Aspecto final del instrumento

Como podemos ver, el aspecto del instrumento no es del todo malo. Quizá la caja cuadrada y esquinosa no pega demasiado con la forma de una gaita, pero la bolsa de aire con el vestido negro, el soprete y el punteiro quedan muy bien. Además la caja siempre más sencillo el montaje de la electrónica. A continuación se muestran una serie de imágenes del aspecto final que tiene el instrumento.



(a) Vista 1.



(b) Vista 2.



(c) Vista 3.



(d) Punteiro.

Figura 7.23 Imágenes del aspecto final del instrumento.

7.4 Código del instrumento final y funcionamiento

Se va a explicar primeramente el funcionamiento del instrumento, para después comentar los aspectos del código más relevantes.

7.4.1 Funcionamiento del instrumento final

En el vídeo del siguiente enlace se explica el funcionamiento de la versión final del instrumento [Video de funcionamiento del instrumento final](#). Como se explica en el vídeo, lo primero sería tener abierto el software necesario para el funcionamiento. Esto es el programa Hairless MIDI<->Serial, el loopMIDI y el sintetizador Universal Piper, correctamente configurados con el puerto MIDI creado mediante loopMIDI a la salida del Hairless y como entrada del sintetizador. Una vez se conecta el instrumento, el puerto COM al que está conectado debería aparecer como entrada del Hairless.

Con el instrumento ya conectado y configurado nos fijamos en la pantalla LCD. La pantalla inicial muestra únicamente el nombre del instrumento. Si se pulsa el botón derecho pasamos a la pantalla de instrucciones. En esta se indica que mediante los botones laterales se pasa al siguiente menú o al anterior, mientras que con los botones superior e inferior se cambia la opción dentro del menú.

Si se pulsa de nuevo el botón lateral derecho pasamos a la pantalla de afinación. Aquí podemos elegir entre las 5 afinaciones diferentes que se han programado. Estas son Do, Re, Sib, Si y La. Pulsando los botones superior e inferior se puede elegir la que queramos.

Pulsando de nuevo el botón derecho se pasa a la pantalla de modos. En esta se puede elegir entre los 3 modos de funcionamiento. El primero consiste en el punteiro, lo que significa que no

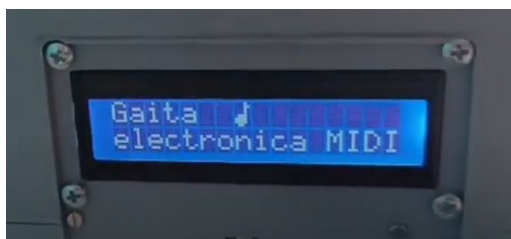
es necesario apretar la bolsa de aire ni soplar. Este modo serviría para practicar la digitación. El segundo modo obliga al intérprete a apretar la bolsa de aire con el brazo, mientras que el tercero obliga a soplar por el soprete además de apretar la bolsa y evidentemente, tocar las notas en el punteiro. Este último es el que se acerca más al funcionamiento real del instrumento, siendo también el más complicado.

La siguiente pantalla nos muestra el porcentaje de volumen que se ajusta girando el encoder del lateral de la caja.

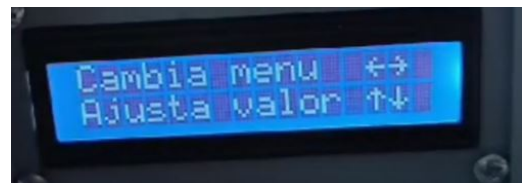
Apertando el botón lateral derecho pasamos a la pantalla del metrónomo. Esta permite activar un metrónomo del sintetizador. Los bpm del metrónomo no se pueden ajustar en el instrumento, sino que deben configurarse en el sintetizador.

La siguiente pantalla permite activar una 2ª voz. La configuración de la misma se debe realizar dentro del sintetizador, que permite un gran número de opciones. Por defecto en la configuración de este trabajo la 2ª voz toca una tercera por encima de la nota generada por el punteiro.

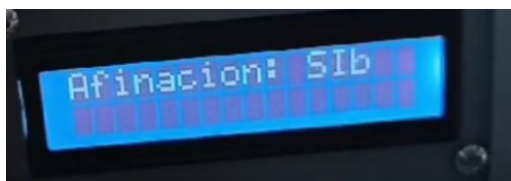
En la última pantalla si se quiere empezar a tocar se debe aplicar la opción ON. Esto se diseñó así porque resulta molesto que el instrumento esté sonando mientras se configuran las opciones. De esta forma cuando se haya terminado de configurar todo, se viene a esta pantalla y se activa la opción. Entonces el instrumento empieza a sonar con distintas condiciones, dependiendo del modo en que se esté tocando.



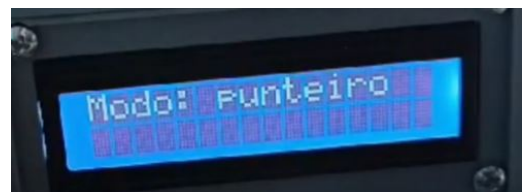
(a) Pantalla inicial.



(b) Pantalla de instrucciones.



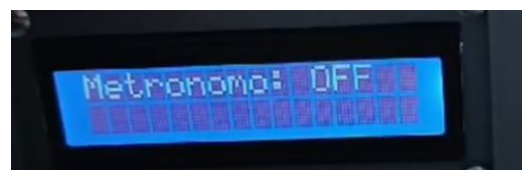
(c) Pantalla de afinación.



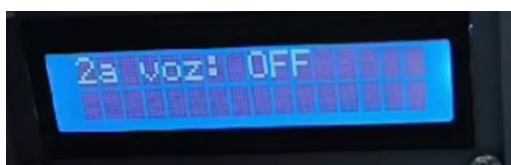
(d) Pantalla de modo.



(e) Pantalla de volumen.



(f) Pantalla de metrónomo.



(g) Pantalla de 2ª voz.



(h) Pantalla de empezar a tocar.

Figura 7.24 Pantallas de opciones del instrumento.

7.4.2 Tabla de funciones MIDI

En esta sección se van a indicar las funciones de control MIDI que se han definido, necesarias para que todas las funciones del instrumento actúen.

Tabla 7.1 Mensajes de Control MIDI.

Mensaje de Control	Número CC	Tipo
Control de volumen (volume)	1	
Control de los bordones (drones)	3	Momentary
Afinación (key)	16	
2ª voz (2nd voice)	15	Momentary
Metrónomo (metronome)	14	Momentary

7.4.3 Explicación del código

En este apartado se explicarán algunos de los aspectos del código, que provocan el funcionamiento deseado de instrumento. Realmente el código final es una conjunción de todos los códigos de prueba que se han ido programando. De esta forma se consigue cierta modularidad y se puede distinguir un fallo en alguna de las funciones. En el apéndice A.11 podemos encontrar el código completo.

En primer lugar se declaran las librerías que se van a necesitar. Después se declaran las variables y objetos necesarios para el módulo MPR121, la comunicación MIDI, las notas y las variables de estado de los botones. A continuación se definen los sensores de presión, indicando los pines a los que se conectan en la placa. Definimos los botones y sus pines, el encoder y la pantalla.

Después se declaran las variables necesarias. Para cada variable se definen la variable en si, y otra del estado anterior de la variable. Esto es para no estar todo el rato mandando mensajes y comprobando el estado de las variables, de forma que los cambios solo se produzcan cuando hay un flanco de subida o bajada en la variable.

Código 7.1 Variables necesarias para el instrumento final.

```
int contadorPantalla; //contador para las diferentes pantallas
int contadorPantallaAnterior;
int afinacion; //contador para el menu de afinacion
int afinacionAnterior;
int modo; //contador para el menu de modo
int modoAnterior;
int metronomo; //contador para el menu del metronomo
int metronomoAnterior;
int reverb; //contador para el menu del reverb
int reverbAnterior;
int empezar;
int empezarAnterior;
int ronco;
int roncoAnterior;
int punteiro;
int punteiroAnterior;
float presionInicial;
const int volmax = 1000; //volumen maximo de aire con que se puede
llenar
```

```
double volaire;           //variable para almacenar el volumen de aire
double volaireAnterior;

unsigned long previousMillis = 0; // Almacena la última vez que se
    ejecutó la función
const long interval = 500;      // Intervalo de tiempo en milisegundos
```

A continuación en el setup se inicializan todas estas variables, además de definir los pines del encoder como entradas PULL-UP, habilitar las interrupciones en sus pines, definir los pines de los botones y empezar la comunicación MIDI mediante el puerto serie a una tasa de 115200 baudios. También se guardan los caracteres personalizados en los espacios libres de la memoria del LCD, se toma el valor de la presión inicial del aire en el fol, y se manda una nota para que el intérprete pueda saber si el instrumento funciona correctamente y está bien configurado.

Ya dentro del loop, se actualizan las variables de los botones y se empieza con la programación de la interfaz mediante la pantalla y los botones. Como tenemos 8 pantallas, debemos hacer que los menús sean circulares, de forma que al llegar al último menú, si se avanza a la derecha se vuelva a la pantalla inicial. En función del valor de la variable contadorPantalla se programa que la variable de cada función aumente o disminuya si se pulsan los botones superior o inferior, con la misma circularidad que los menús. Si llegas a la última opción y pulsas de nuevo el botón, vuelves a la primera.

A continuación mediante un switch case se programa en función de la variable contadorPantalla que se debe displayear en el LCD. Para ello se llaman a las funciones de las pantallas que se encargan de colocar los elementos en las posiciones del LCD adecuadas. Un ejemplo se puede ver a continuación, en el que primero se escribe la información inicial, y si se ha pulsado un botón para cambiar la opción se redibuja la pantalla con la opción nueva.

Código 7.2 Función de la pantalla.

```
void pantallaEmpezar() {
    lcd.setCursor(0, 0);
    lcd.print("Empezar");
    lcd.setCursor(0, 1);
    lcd.print("a tocar:");
    lcd.setCursor(9, 1);
    lcd.print("OFF");
    if (empezar != empezarAnterior || empezar != 0) {
        if (empezar == 0) {
            lcd.setCursor(9, 1);
            lcd.print("OFF");
        } else if (empezar == 1) {
            lcd.clear();
            lcd.setCursor(0, 0);
            lcd.print("Empezar");
            lcd.setCursor(0, 1);
            lcd.print("a tocar:");
            lcd.setCursor(9, 1);
            lcd.print("ON");
        }
    }
}
```


Después de esto se programan los mensajes de control MIDI que se deben enviar en función del menú en el que nos encontremos. Vale la pena mencionar que el valor del mensaje de control para la afinación es diferente del resto de valores de funciones como apagar o encender el metrónomo, que solo necesita el valor 0 para estar apagado y el 127 para encenderse.

A continuación se leen los datos devueltos por el módulo de sensores capacitivos, y se actualizan las variables botón en función de la lectura devuelta. Después se programan los modos de funcionamiento. Un ejemplo es el código a continuación. Es importante tener en cuenta todas las posibilidades para tocar las notas, activar los bordones y demás. Las lecturas de los sensores de presión se realizan cada 0.5 segundos, ya que si se comprueban todo el rato y no se introduce esta condición de tiempo, el instrumento se vuelve muy lento.

Código 7.3 Programación del modo 2.

```
//modo 2 toco tapando los agujeros del punteiro, apretando el fol y
soplando por el tubo
if (modo == 2) {
  if (empezar == 1) {
    nota = generaNotas();
    if (currentMillis - previousMillis >= interval) {
      previousMillis = currentMillis;
      volaire = compruebaPresionSoprete();
      ronco = compruebaPresionFol();
      punteiro = compruebaPresionPunteiro();
    }
    if (volaire > 0) {
      if (ronco != roncoAnterior && ronco == HIGH) {
        MIDI.sendControlChange(3, 127, 1);
      } else if (ronco != roncoAnterior && ronco == LOW) {
        MIDI.sendControlChange(3, 0, 1);
      }
      if (punteiro == HIGH && punteiroAnterior == LOW) {
        MIDI.sendNoteOn(nota, 60, 1);
      } else if (punteiro == LOW && punteiroAnterior == HIGH) {
        MIDI.sendNoteOff(nota, 60, 1);
      }
      if (punteiro == HIGH) {
        if (nota != notaAnterior) {
          MIDI.sendNoteOn(nota, 60, 1);
        }
      }
    }
    } else if (volaire == 0 && volaireAnterior > 0) {
      MIDI.sendControlChange(3, 0, 1);
      MIDI.sendNoteOff(nota, 60, 1);
    }
  } else if (empezar == 0 && empezarAnterior == 1) {
    MIDI.sendNoteOff(nota, 60, 1);
    MIDI.sendControlChange(3, 0, 1);
  }
}
```

Las funciones de comprobación de la presión se encuentran al final de código, y básicamente comparan la lectura del sensor con unos umbrales, y en función de si estos se superan, se cambia el valor de la variable asociada. Un ejemplo de una de estas funciones es el siguiente código:

Código 7.4 Función del sensor de presión para que empiece a sonar las notas del punteiro.

```
bool compruebaPresionPunteiro() {  
    if ((hx711Fol.read() / 1000.0) > presionInicial + 1000) {  
        punteiro = HIGH;  
    } else {  
        punteiro = LOW;  
    }  
    return punteiro;  
}
```

Por último se actualizan todas las variables de estado de las opciones. Para generar los valores de las notas se utiliza la función generaNotas(), que devuelve la variable nota con un valor que depende de los botones que se están pulsando del punteiro. El mapeado de los botones es parcial debido a que con 8 agujeros tendríamos que programar $2^8 = 256$ posibilidades diferentes. Sin embargo todas las posiciones típicas y correctas de las notas de la gaita están contempladas.

La cantidad de recursos utilizados con el programa final se puede observar en la consola del Arduino IDE al compilar el programa. Devuelve el siguiente mensaje:

El Sketch usa 16980 bytes (52 %) del espacio de almacenamiento de programa. El máximo es 32256 bytes.

Las variables Globales usan 1032 bytes (50 %) de la memoria dinámica, dejando 1016 bytes para las variables locales. El máximo es 2048 bytes.

Podemos ver que se han utilizado la mitad de los recursos de memoria de la placa.

8 Presupuesto

En este último capítulo se detallará el coste final de los componentes para poder evidenciar que el instrumento electrónico resulta más barato que su contraparte tradicional y que las opciones ofrecidas por otros fabricantes de instrumentos electrónicos. En la siguiente tabla se detallan los costes de los diferentes elementos utilizados.

Tabla 8.1 Presupuesto..

Descripción	Unidades	Coste/Ud (€)
Arduino UNO R3	1	4.12
Módulo de sensores capacitivos MPR121	1	1.18
Sensor de presión HX710B	2	1.29
pantalla LCD LCD1602	1	1.31
Encoder rotatorio EC-11	1	1.06
Pulsadores	4	0,01
Conector MIDI hembra	1	0.26
Cables DUPont	40	0.05
Placas PCB	2	0.36
Resistencias de distintos valores	8	0.01
PLA para la impresión 3d	1/2	20
Tornillos de aluminio	9	0.14
Tuercas de aluminio	9	0.01
Subtotal (€)		24.7

Debemos tener en cuenta que no se está teniendo en cuenta el coste de la electricidad necesaria para la impresión 3d de las piezas del instrumento. Tampoco se tiene en cuenta el coste del estaño o de la pasta para la soldadura, ni de las herramientas necesarias como el soldador, destornilladores o la máquina de impresión 3d.

También hay que especificar que algunos de los costes de los elementos son aproximados, como la cantidad de PLA utilizado, o el precio de elementos que se adquirieron en packs de muchas unidades, como las resistencias, tornillos o los pulsadores. El precio de estos elementos se obtuvo al dividir el coste del pack entre las unidades totales.

Por último comentar que los costes pueden variar según el distribuidor del que se obtengan. En mi caso la mayoría de elementos se obtuvieron de distribuidores no oficiales, lo que abarató el coste.

9 Conclusiones del proyecto y posibles mejoras

Como conclusión del proyecto, se ha conseguido construir un instrumento electrónico que imita el comportamiento de la gaita, logrando el objetivo que se fijó en un principio. Además, gracias a su realización electrónica se incluyen más funciones que las disponibles en el instrumento tradicional, y se consigue una mayor versatilidad y portabilidad.

El instrumento resulta ser de construcción mucho más barata que el resto de aparatos parecidos disponibles actualmente. Además añade funciones nuevas que ayudan al intérprete a desarrollar las habilidades necesarias para aprender a tocar el instrumento tradicional, como la necesidad de soplar aire y apretar una bolsa de aire a la vez que se practica la digitación.

Adicionalmente, se podrían añadir funciones nuevas o realizar ciertas adiciones sobre el instrumento. Algunas mejoras que se pueden implementar son:

- Mejora de la ergonomía. La caja esquinosa puede resultar un poco incómoda y no ajustarse demasiado a la estética del instrumento. Es posible que mediante un ajuste más fino del montaje se consiguiera un encapsulamiento más compacto.
- Adición de funciones. Algunas funciones que se pueden añadir podrían ser más efectos o alguna función disponible en el sintetizador.
- Conversión del instrumento de controlador MIDI a instrumento MIDI. Esto significa añadir algún tipo de shield o módulo al controlador MIDI desarrollado, que permita que el aparato emita sonido por sí mismo, sin necesidad de conectarse a un ordenador y al sintetizador. Esto implicaría seguramente el cambio del microcontrolador utilizado, que tenga más pines de salida.

Con todo esto, personalmente quedo muy satisfecho con lo conseguido. Debido a mi interés personal con el instrumento, es muy posible que posteriormente a la entrega de este trabajo, añada alguna mejora o siga con el desarrollo. Desde luego es un instrumento que voy a utilizar para practicar, y es posible que comente sus funciones por algún foro de gaitas, por si puede servir de utilidad a otras personas que estén empezando a aprender el instrumento, o que quieran empezar. En cualquier caso, en el siguiente enlace de [github](#) queda subido lo necesario para la construcción del instrumento [Gaita_electronica_MIDI](#).

Por último, para el desarrollo de este trabajo se han puesto en práctica los conocimientos adquiridos durante el grado, como el diseño de modelos de piezas, la cumplimentación de planos y esquemáticos de las conexiones, además de las conexiones electrónicas y uso de microcontroladores. Adicionalmente se han adquirido conocimientos nuevos como el uso del protocolo MIDI, el uso de una máquina de impresión 3d o los distintos softwares de sintetizadores.

Apéndice A

Códigos de Arduino

A.1 Código del prototipo inicial

```
1  #include <MIDI.h>
2
3
4  MIDI_CREATE_DEFAULT_INSTANCE();
5  // Definicion de las notas
6  int SI3;
7  int DO4;
8  int REB4;
9  int RE4;
10 int MIB4;
11 int MI4;
12 int FA4;
13 int SOLB4;
14 int SOL4;
15 int LAB4;
16 int LA4;
17 int SIB4;
18 int SI4;
19 int DO5;
20 int RE5;
21 int MIB5;
22 int MI5;
23 int FA5;
24
25 // constantes de los pines de los botones
26 const int agujero0 = 9;
27 const int agujero1 = 8;
28 const int agujero2 = 7;
29 const int agujero3 = 6;
30 const int agujero4 = 5;
31 const int agujero5 = 4;
32 const int agujero6 = 3;
33 const int agujero7 = 2;
34
```

```
35 // variables del estado de los botones
36 int uno = 0;
37 int dos = 0;
38 int tres = 0;
39 int cuatro = 0;
40 int cinco = 0;
41 int seis = 0;
42 int siete = 0;
43 int ocho = 0;
44
45 void setup() {
46     // primero definimos los pines de los botones como entradas
47     pinMode(agujero0, INPUT);
48     pinMode(agujero1, INPUT);
49     pinMode(agujero2, INPUT);
50     pinMode(agujero3, INPUT);
51     pinMode(agujero4, INPUT);
52     pinMode(agujero5, INPUT);
53     pinMode(agujero6, INPUT);
54     pinMode(agujero7, INPUT);
55
56     //valor midi de las notas
57     SI3 = 59;
58     DO4 = 60;
59     REB4 = 61;
60     RE4 = 62;
61     MIB4 = 63;
62     MI4 = 64;
63     FA4 = 65;
64     SOLB4 = 66;
65     SOL4 = 67;
66     LAB4 = 68;
67     LA4 = 69;
68     SIB4 = 70;
69     SI4 = 71;
70     DO5 = 72;
71     RE5 = 74;
72     MIB5 = 75;
73     MI5 = 76;
74     FA5 = 77;
75
76     // comenzamos la comunicacion MIDI
77     MIDI.begin();
78     Serial.begin(115200); //Cambiamos la velocidad del puerto serie
79 }
80
81 void loop() {
82     // leemos el estado de los botones
83     ocho = digitalRead(agujero0);
84     siete = digitalRead(agujero1);
```



```

85  seis = digitalRead(agujero2);
86  cinco = digitalRead(agujero3);
87  cuatro = digitalRead(agujero4);
88  tres = digitalRead(agujero5);
89  dos = digitalRead(agujero6);
90  uno = digitalRead(agujero7);
91
92
93  // generacion de las notas en funcion del agujero tapado
94  //en funcion de si tapamos agujero 8 tenemos la 1a octava o la 2a
95  if (ocho == HIGH) //primera octava, Seguimos el esquema dado en
96      dixitacion
97  {
98      if (uno == HIGH && dos == HIGH && tres == HIGH && cuatro == HIGH &&
99          cinco == HIGH && seis == HIGH && siete == HIGH) {
100         MIDI.sendNoteOn(SI3, 60, 1); //Genera la nota
101         // delay(10);
102         MIDI.sendNoteOff(SI3, 60, 1); //Genera la nota
103     }
104     else if (uno == LOW && dos == HIGH && tres == HIGH && cuatro == HIGH
105         && cinco == HIGH && seis == HIGH && siete == HIGH) {
106         MIDI.sendNoteOn(DO4, 60, 1); //Genera la nota
107         // delay(10);
108         MIDI.sendNoteOff(DO4, 60, 1);
109     }
110     else if (uno == LOW && dos == LOW && tres == HIGH && cuatro == HIGH
111         && cinco == HIGH && seis == HIGH && siete == HIGH) {
112         MIDI.sendNoteOn(RE4, 60, 1); //Genera la nota
113         // delay(10);
114         MIDI.sendNoteOff(RE4, 60, 1);
115     }
116     else if (uno == LOW && dos == HIGH && tres == LOW && cuatro == HIGH
117         && cinco == HIGH && seis == HIGH && siete == HIGH) {
118         MIDI.sendNoteOn(MIB4, 60, 1); //Genera la nota
119         // delay(10);
120         MIDI.sendNoteOff(MIB4, 60, 1);
121     }
122     else if (uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
123         && cinco == HIGH && seis == HIGH && siete == HIGH) {
124         MIDI.sendNoteOn(MI4, 60, 1); //Genera la nota
125         // delay(10);
126         MIDI.sendNoteOff(MI4, 60, 1);
127     }
128     // para permitir el picado solo levantando el agujero 4
129     else if ((uno == HIGH && dos == HIGH && tres == HIGH && cuatro ==
130         LOW && cinco == HIGH && seis == HIGH && siete == HIGH) ||
131         (uno == LOW && dos == HIGH && tres == HIGH && cuatro == LOW &&
132         cinco == HIGH && seis == HIGH && siete == HIGH) ||
133         (uno == LOW && dos == LOW && tres == HIGH && cuatro == LOW &&
134         cinco == HIGH && seis == HIGH && siete == HIGH)) {

```

```

135     MIDI.sendNoteOn(MI4, 60, 1); //Genera la nota
136     // delay(10);
137     MIDI.sendNoteOff(MI4, 60, 1);
138 }
139 else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
140         cinco == HIGH && seis == HIGH && siete == HIGH) {
141     MIDI.sendNoteOn(FA4, 60, 1); //Genera la nota
142     // delay(10);
143     MIDI.sendNoteOff(FA4, 60, 1);
144 }
145 else if (uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
146         && cinco == LOW && seis == HIGH && siete == HIGH) {
147     MIDI.sendNoteOn(SOLB4, 60, 1); //Genera la nota
148     // delay(10);
149     MIDI.sendNoteOff(SOLB4, 60, 1);
150 }
151 else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
152         cinco == LOW && seis == HIGH && siete == HIGH) {
153     MIDI.sendNoteOn(SOL4, 60, 1); //Genera la nota
154     // delay(10);
155     MIDI.sendNoteOff(SOL4, 60, 1);
156 }
157 else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
158         cinco == HIGH && seis == LOW && siete == HIGH) {
159     MIDI.sendNoteOn(LAB4, 60, 1); //Genera la nota
160     // delay(10);
161     MIDI.sendNoteOff(LAB4, 60, 1);
162 }
163 else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
164         cinco == LOW && seis == LOW && siete == HIGH) {
165     MIDI.sendNoteOn(LA4, 60, 1); //Genera la nota
166     // delay(10);
167     MIDI.sendNoteOff(LA4, 60, 1);
168 }
169 else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
170         cinco == LOW && seis == HIGH && siete == LOW) {
171     MIDI.sendNoteOn(SIB4, 60, 1); //Genera la nota
172     // delay(10);
173     MIDI.sendNoteOff(SIB4, 60, 1);
174 }
175 else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
176         cinco == LOW && seis == LOW && siete == LOW) {
177     MIDI.sendNoteOn(SI4, 60, 1); //Genera la nota
178     // delay(10);
179     MIDI.sendNoteOff(SI4, 60, 1);
180 }
181 //permitir picados a partir del MI4
182 else if ((uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
183         && cinco == HIGH && seis == HIGH && siete == LOW) ||

```

```

184     (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
185         cinco == HIGH && seis == HIGH && siete == LOW)) {
186     MIDI.sendNoteOn(SIB4, 60, 1); //Genera la nota
187     // delay(10);
188     MIDI.sendNoteOff(SIB4, 60, 1);
189 }
190 else{ //para cualquier combinacion de agujeros no contemplada de
191     momento
192     MIDI.sendNoteOn(SIB4, 60, 1); //Genera la nota
193     // delay(10);
194     MIDI.sendNoteOff(SIB4, 60, 1);
195 }
196 }
197 else //segunda octava
198 {
199     if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
200         cinco == LOW && seis == LOW && siete == LOW){
201         MIDI.sendNoteOn(D05, 60, 1); //Genera la nota
202         // delay(10);
203         MIDI.sendNoteOff(D05, 60, 1);
204     }
205     else if (uno == HIGH && dos == HIGH && tres == HIGH && cuatro ==
206         HIGH && cinco == HIGH && seis == HIGH && siete == HIGH) {
207         MIDI.sendNoteOn(SI4, 60, 1); //Genera la nota
208         // delay(10);
209         MIDI.sendNoteOff(SI4, 60, 1);
210     }
211     else if (uno == LOW && dos == LOW && tres == HIGH && cuatro == HIGH
212         && cinco == HIGH && seis == HIGH && siete == HIGH) {
213         MIDI.sendNoteOn(RE5, 60, 1); //Genera la nota
214         // delay(10);
215         MIDI.sendNoteOff(RE5, 60, 1);
216     }
217     else if (uno == LOW && dos == HIGH && tres == LOW && cuatro == HIGH
218         && cinco == HIGH && seis == HIGH && siete == HIGH) {
219         MIDI.sendNoteOn(MIB5, 60, 1); //Genera la nota
220         // delay(10);
221         MIDI.sendNoteOff(MIB5, 60, 1);
222     }
223     else if (uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
224         && cinco == HIGH && seis == HIGH && siete == HIGH) {
225         MIDI.sendNoteOn(MI5, 60, 1); //Genera la nota
226         // delay(10);
227         MIDI.sendNoteOff(MI5, 60, 1);
228     }
229     else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
230         cinco == HIGH && seis == HIGH && siete == HIGH) {
231         MIDI.sendNoteOn(FA5, 60, 1); //Genera la nota
232         // delay(10);
233         MIDI.sendNoteOff(FA5, 60, 1);

```

```

234     }
235     else{
236         MIDI.sendNoteOn(D05, 60, 1); //Genera la nota
237         // delay(10);
238         MIDI.sendNoteOff(D05, 60, 1);
239     }
240 }
241 }
242

```

A.2 Código de prueba de los sensores de presión

```

1
2  #include <Q2HX711.h>
3
4  const byte pin_Datos = A2; // En la plata del sensor es el pin OUT
5  const byte pin_Relej = A3; // En la placa del sensor es SCK
6  Q2HX711 hx711(pin_Datos, pin_Relej);
7
8  void setup() {
9      Serial.begin(9600);
10 }
11
12 void loop() {
13     Serial.println(hx711.read() / 1000.0);
14 }
15

```

A.3 Código para probar el encoder y controlar el volumen:

```

1
2  #include <MIDI.h>
3
4  #define ENC_A 2 //definimos los pines del encoder digital
5  #define ENC_B 3
6  MIDI_CREATE_DEFAULT_INSTANCE(); //creamos instancia MIDI
7
8  unsigned long _lastIncReadTime = micros();
9  unsigned long _lastDecReadTime = micros();
10 int _pauseLength = 25000;
11 int _fastIncrement = 10; //cuanto avanza si giramos rapido el encoder
12 volatile int counter = 0; //variable del giro del encoder
13
14 void setup() {
15
16     pinMode(ENC_A, INPUT_PULLUP); //definimos pines del encoder como
17     entradas
18     pinMode(ENC_B, INPUT_PULLUP);
19     attachInterrupt(digitalPinToInterrupt(ENC_A), read_encoder, CHANGE);
20     //habilitamos las interrupciones en los pines del encoder

```

```

21   attachInterrupt(digitalPinToInterrupt(ENC_B), read_encoder, CHANGE);
22
23   MIDI.begin(); //comenzamos la comunicación MIDI
24   Serial.begin(115200);
25 }
26
27 void loop() {
28     static int lastCounter = 0;
29     MIDI.sendNoteOn(60, 60, 1); //mandamos una nota para comprobar el
30         volumen
31
32     if (counter != lastCounter) { //si el valor de counter cambio
33         lastCounter = counter;
34         MIDI.sendControlChange(1, counter, 1); //cambiamos el volumen
35     }
36 }
37
38 void read_encoder() {
39     //rutina de interrupcion para los 2 pines del encoder. Se utiliza la
40         variable counter
41     //para ver cuanto ha girado, y se actualiza si se cumplen ciertas
42         condiciones
43
44     static uint8_t old_AB = 3;
45
46         table
47     static int8_t encval = 0;
48
49         del encoder
50     static const int8_t enc_states[] = { 0, -1, 1, 0, 1, 0, 0, -1, -1, 0,
51         0, 1, 0, 1, -1, 0 }; // Lookup table
52
53     old_AB <= 2; // recordamos estado anterior
54
55     if (digitalRead(ENC_A)) old_AB |= 0x02; // estado actual del pin A
56     if (digitalRead(ENC_B)) old_AB |= 0x01; // estado actual de pin B
57
58     encval += enc_states[(old_AB & 0x0f)];
59
60     //actualizamos counter si el encoder ha girado un tramo completo, por
61         lo menos 4 pasos
62     if (encval > 3) { // si hemos avanzado 4 pasos
63         int changevalue = 1;
64         if ((micros() - _lastIncReadTime) < _pauseLength) {
65             changevalue = _fastIncrement * changevalue;
66         }
67         _lastIncReadTime = micros();
68         counter = counter + changevalue; //actualizamos counter
69         if (counter < 0) {
70             counter = 0; //debemos saturar para que no de valores negativos

```

```

71     }
72     if (counter > 127) {
73         counter = 127; //debemos saturar para que no de valores mayores de
74             127
75     }
76     encval = 0;
77 } else if (encval < -3) { //si hemos retrocedido 4 pasos
78     int changevalue = -1;
79     if ((micros() - _lastDecReadTime) < _pauseLength) {
80         changevalue = _fastIncrement * changevalue;
81     }
82     _lastDecReadTime = micros();
83     counter = counter + changevalue; //actualizamos counter
84     if (counter < 0) { //saturacion
85         counter = 0;
86     }
87     if (counter > 127) { //saturacion
88         counter = 127;
89     }
90     encval = 0;
91 }
92 }
93

```

A.4 Código final del primer prototipo

```

1
2 //aquí declaración de librerías
3 #include <Q2HX711.h>
4 #include <MIDI.h>
5 MIDI_CREATE_DEFAULT_INSTANCE();
6
7 //aquí cosas del sensor de presión
8 const byte pin_Datos = A2; // En la placa del sensor es el pin OUT
9 const byte pin_Reloj = A3; // En la placa del sensor es SCK
10 const int volmax = 1000;
11 double volaire = 0;
12 Q2HX711 hx711(pin_Datos, pin_Reloj);
13
14 //aquí cosas del ronco
15 const int buttonPin = 4; // the number of the pushbutton pin
16 bool ronco = HIGH; // estado del ronco
17 int buttonState; // the current reading from the input
18     pin
19 int lastButtonState = LOW; // the previous reading from the input
20     pin
21 unsigned long lastDebounceTime = 0; // the last time the output pin was
22     toggled
23 unsigned long debounceDelay = 50; // the debounce time; increase if the
24     output flickers

```

```
25
26 //aqui cosas del control de volumen
27 const int S1 = 2;
28 const int S2 = 3;
29 unsigned long _lastIncReadTime = micros();
30 unsigned long _lastDecReadTime = micros();
31 int _pauseLength = 25000;
32 int _fastIncrement = 10;
33 volatile int counter = 40;
34
35 // Definicion de las notas midi
36 const int SI3 = 59;
37 const int DO4 = 60;
38 const int REB4 = 61;
39 const int RE4 = 62;
40 const int MIB4 = 63;
41 const int MI4 = 64;
42 const int FA4 = 65;
43 const int SOLB4 = 66;
44 const int SOL4 = 67;
45 const int LAB4 = 68;
46 const int LA4 = 69;
47 const int SIB4 = 70;
48 const int SI4 = 71;
49 const int DO5 = 72;
50 const int RE5 = 74;
51 const int MIB5 = 75;
52 const int MI5 = 76;
53 const int FA5 = 77;
54
55 int nota;
56
57 // constantes de los pines de los botones
58 const int agujero0 = 13;
59 const int agujero1 = 12;
60 const int agujero2 = 11;
61 const int agujero3 = 10;
62 const int agujero4 = 9;
63 const int agujero5 = 8;
64 const int agujero6 = 7;
65 const int agujero7 = 6;
66
67 // variables del estado de los botones
68 int uno = 0;
69 int dos = 0;
70 int tres = 0;
71 int cuatro = 0;
72 int cinco = 0;
73 int seis = 0;
74 int siete = 0;
```

```

75  int ocho = 0;
76
77  void setup() {
78      // primero definimos los pines de los botones como entradas
79      pinMode(agujero0, INPUT);
80      pinMode(agujero1, INPUT);
81      pinMode(agujero2, INPUT);
82      pinMode(agujero3, INPUT);
83      pinMode(agujero4, INPUT);
84      pinMode(agujero5, INPUT);
85      pinMode(agujero6, INPUT);
86      pinMode(agujero7, INPUT);
87
88      //empezamos comunicacion serie con la libreria MIDI
89      MIDI.begin();
90      Serial.begin(115200);
91
92      //definimos los pines del encoder
93      pinMode(S1, INPUT_PULLUP);
94      pinMode(S2, INPUT_PULLUP);
95      attachInterrupt(digitalPinToInterrupt(S1), read_encoder, CHANGE);
96      attachInterrupt(digitalPinToInterrupt(S2), read_encoder, CHANGE);
97
98      //defino el pin del boton del encoder
99      pinMode(buttonPin, INPUT);
100
101      //inicio las variables en el valor deseado
102      ronco = HIGH;
103      volaire = 0;
104      nota = D05;
105  }
106
107  void loop() {
108      if (hx711.read() / 1000.0 > 10500) {
109          volaire = volaire + 12;
110          if (volaire >= 1000) {
111              volaire = 1000;
112          }
113      } else {
114          volaire = volaire - 6;
115          if (volaire < 0) {
116              volaire = 0;
117          }
118      }
119
120      int reading = digitalRead(buttonPin); //programacion boton
121      if (reading != lastButtonState) {
122          lastDebounceTime = millis();
123      }
124

```



```

125   if ((millis() - lastDebounceTime) > debounceDelay) { //debounce del
126       boton
127       if (reading != buttonState) {
128           buttonState = reading;
129           if (buttonState == HIGH) {
130               ronco = HIGH;
131               MIDI.sendControlChange(3, 127, 1); //mando contro change para el
132                   volumen de valor HIGH
133           } else if (buttonState == LOW) {
134               MIDI.sendControlChange(3, 0, 1); //mando contro change para el
135                   volumen de valor LOW
136           }
137       }
138   }
139   lastButtonState = reading;
140
141   static int lastCounter = 0; //programacion encoder
142   if (counter != lastCounter) {
143       lastCounter = counter;
144       MIDI.sendControlChange(1, counter, 1);
145   }
146
147   // leemos el estado de los botones
148   ocho = digitalRead(agujero0);
149   siete = digitalRead(agujero1);
150   seis = digitalRead(agujero2);
151   cinco = digitalRead(agujero3);
152   cuatro = digitalRead(agujero4);
153   tres = digitalRead(agujero5);
154   dos = digitalRead(agujero6);
155   uno = digitalRead(agujero7);
156
157   // generacion de las notas en funcion del agujero tapado
158   //en funcion de si tapamos agujero 8 tenemos la 1a octava o la 2a
159   if (ocho == HIGH) //primera octava, Seguimos el esquema dado en
160       dixitacion
161   {
162       if (uno == HIGH && dos == HIGH && tres == HIGH && cuatro == HIGH &&
163           cinco == HIGH && seis == HIGH && siete == HIGH) {
164           nota=SI3;
165       } else if (uno == LOW && dos == HIGH && tres == HIGH && cuatro ==
166           HIGH && cinco == HIGH && seis == HIGH && siete == HIGH) {
167           nota=DO4;
168       } else if (uno == LOW && dos == LOW && tres == HIGH && cuatro ==
169           HIGH && cinco == HIGH && seis == HIGH && siete == HIGH) {
170           nota=RE4;
171       } else if (uno == LOW && dos == HIGH && tres == LOW && cuatro ==
172           HIGH && cinco == HIGH && seis == HIGH && siete == HIGH) {
173           nota=MIB4;

```

```

174     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
175             && cinco == HIGH && seis == HIGH && siete == HIGH) {
176         nota=MI4;
177     }
178     // para permitir el picado solo levantando el agujero 4
179     else if ((uno == HIGH && dos == HIGH && tres == HIGH && cuatro ==
180             LOW && cinco == HIGH && seis == HIGH && siete == HIGH) || (uno ==
181             LOW && dos == HIGH && tres == HIGH && cuatro == LOW && cinco ==
182             HIGH && seis == HIGH && siete == HIGH) || (uno == LOW && dos ==
183             LOW && tres == HIGH && cuatro == LOW && cinco == HIGH && seis ==
184             HIGH && siete == HIGH)) {
185         nota=MI4;
186     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW
187             && cinco == HIGH && seis == HIGH && siete == HIGH) {
188         nota=FA4;
189     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
190             && cinco == LOW && seis == HIGH && siete == HIGH) {
191         nota=SOLB4;
192     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW
193             && cinco == LOW && seis == HIGH && siete == HIGH) {
194         nota=SOL4;
195     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW
196             && cinco == HIGH && seis == LOW && siete == HIGH) {
197         nota=LAB4;
198     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW
199             && cinco == LOW && seis == LOW && siete == HIGH) {
200         nota=LA4;
201     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW
202             && cinco == LOW && seis == HIGH && siete == LOW) {
203         nota=SIB4;
204     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW
205             && cinco == LOW && seis == LOW && siete == LOW) {
206         nota=SI4;
207     }
208     //permitir picados a partir del MI4
209     else if ((uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
210             && cinco == HIGH && seis == HIGH && siete == LOW) || (uno == LOW
211             && dos == LOW && tres == LOW && cuatro == LOW && cinco == HIGH &&
212             seis == HIGH && siete == LOW)) {
213         nota=SIB4;
214     } else {
215         nota=SIB4; //para cualquier combinacion de agujeros
216         no contemplada de momento
217     }
218 } else //segunda octava
219 {
220     if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW &&
221         cinco == LOW && seis == LOW && siete == LOW) {
222         nota=D05;

```

```

223     } else if (uno == HIGH && dos == HIGH && tres == HIGH && cuatro ==
224         HIGH && cinco == HIGH && seis == HIGH && siete == HIGH) {
225         nota=SI4;
226     } else if (uno == LOW && dos == LOW && tres == HIGH && cuatro ==
227         HIGH && cinco == HIGH && seis == HIGH && siete == HIGH) {
228         nota=RE5;
229     } else if (uno == LOW && dos == HIGH && tres == LOW && cuatro ==
230         HIGH && cinco == HIGH && seis == HIGH && siete == HIGH) {
231         nota=MIB5;
232     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == HIGH
233         && cinco == HIGH && seis == HIGH && siete == HIGH) {
234         nota=MI5;
235     } else if (uno == LOW && dos == LOW && tres == LOW && cuatro == LOW
236         && cinco == HIGH && seis == HIGH && siete == HIGH) {
237         nota=FA5;
238     } else {
239         nota=D05;
240     }
241 }
242
243 if (volaire>0){
244     MIDI.sendNoteOn(nota, 60, 1);
245 }
246 else{
247     MIDI.sendNoteOff(nota, 60, 1);
248 }
249 }
250
251 void read_encoder() { //funcion para el encoder digital
252     // Encoder interrupt routine for both pins. Updates counter
253     // if they are valid and have rotated a full indent
254
255     static uint8_t old_AB = 3;
256
257     table index
258     static int8_t encval = 0;
259
260     value
261     static const int8_t enc_states[] = { 0, -1, 1, 0, 1, 0, 0, -1, -1, 0,
262         0, 1, 0, 1, -1, 0 }; // Lookup table
263
264     old_AB <= 2; // Remember previous state
265
266     if (digitalRead(S1)) old_AB |= 0x02; // Add current state of pin A
267     if (digitalRead(S2)) old_AB |= 0x01; // Add current state of pin B
268
269     encval += enc_states[(old_AB & 0x0f)];
270
271     // Update counter if encoder has rotated a full indent, that is at
272     least 4 steps

```

```

273     if (encval > 3) { // Four steps forward
274         int changevalue = 1;
275         if ((micros() - _lastIncReadTime) < _pauseLength) {
276             changevalue = _fastIncrement * changevalue;
277         }
278         _lastIncReadTime = micros();
279         counter = counter + changevalue; // Update counter
280         if (counter < 0) {
281             counter = 0;
282         }
283         if (counter > 127) {
284             counter = 127;
285         }
286         encval = 0;
287     } else if (encval < -3) { // Four steps backward
288         int changevalue = -1;
289         if ((micros() - _lastDecReadTime) < _pauseLength) {
290             changevalue = _fastIncrement * changevalue;
291         }
292         _lastDecReadTime = micros();
293         counter = counter + changevalue; // Update counter
294         if (counter < 0) {
295             counter = 0;
296         }
297         if (counter > 127) {
298             counter = 127;
299         }
300         encval = 0;
301     }
302 }
303

```

A.5 Código para probar los sensores capacitivos caseros:

```

1
2  #include <CapacitiveSensor.h>
3
4  // Create an instance of the library
5  CapacitiveSensor capSensor0 = CapacitiveSensor(4, 13);
6
7  // Threshold for turning the lamp on
8  int threshold = 1000;
9
10 // Pin the LED is connected to
11 const int ledPin = 2;
12
13 int sensorvalue;
14 int sensorvalue1;
15 int sensorvalue2;
16 int sensorvalue3;

```

```

17 long media;
18
19 // Function to read sensor and calculate average
20 bool checkThreshold(int threshold) {
21     // Store the value reported by the sensor in a variable
22     sensorvalue = capSensor0.capacitiveSensor(100);
23     // Calculate the average of the last four sensor readings
24     media = (sensorvalue + sensorvalue1 + sensorvalue2 + sensorvalue3) /
25         4;
26
27     // Print media value (for debugging)
28     Serial.println(media);
29
30     // Update previous sensor values
31     sensorvalue3 = sensorvalue2;
32     sensorvalue2 = sensorvalue1;
33     sensorvalue1 = sensorvalue;
34
35     // Return true if the media value is greater than the threshold,
36     // otherwise return false
37     return (media > threshold);
38 }
39

```

A.6 Código para probar el módulo MPR121:

```

1
2 #include <MIDI.h>
3 #include <Wire.h>
4 #include "Adafruit_MPR121.h"
5
6 #ifndef _BV
7 #define _BV(bit) (1 << (bit))
8 #endif
9
10 // You can have up to 4 on one i2c bus but one is enough for testing!
11 Adafruit_MPR121 cap = Adafruit_MPR121();
12
13 // Keeps track of the last pins touched
14 // so we know when buttons are 'released'
15 uint16_t lasttouched = 0;
16 uint16_t currntouched = 0;
17
18 int nota;
19
20 // variables del estado de los botones
21 int cero = 0;
22 int uno = 0;
23 int dos = 0;
24 int tres = 0;

```

```
25 int cuatro = 0;
26 int cinco = 0;
27 int seis = 0;
28 int siete = 0;
29 int ocho = 0;
30 int nueve = 0;
31 int diez = 0;
32 int once = 0;
33
34 bool boton[11];
35
36 void setup() {
37     cap.begin(0x5A);
38
39     //MIDI.begin();
40     Serial.begin(9600); //Cambiamos la velocidad del puerto serie
41
42     // Default address is 0x5A, if tied to 3.3V its 0x5B
43     // If tied to SDA its 0x5C and if SCL then 0x5D
44 }
45
46 void loop() {
47
48     currouched = cap.touched();
49
50     for (uint8_t i = 0; i < 12; i++) {
51         // it if *is* touched and *wasnt* touched before, alert!
52         if ((currouched & _BV(i)) && !(lasttouched & _BV(i))) {
53             boton[i] = 1;
54         }
55         // if it *was* touched and now *isnt*, alert!
56         if (!(currouched & _BV(i)) && (lasttouched & _BV(i))) {
57             boton[i] = 0;
58         }
59     }
60     // reset our state
61     lasttouched = currouched;
62
63     cero = boton[0];
64     uno = boton[1];
65     dos = boton[2];
66     tres = boton[3];
67     cuatro = boton[4];
68     cinco = boton[5];
69     seis = boton[6];
70     siete = boton[7];
71     ocho = boton[8];
72     nueve = boton[9];
73     diez = boton[10];
74     once = boton[11];
```

```

75
76 Serial.print(cero);Serial.print(unos);Serial.print(dos);Serial.print(tres
77 );Serial.print(cuatro);Serial.print(cinco);Serial.print(seis);Serial.
78 print(siete);Serial.print(ocho);Serial.print(nueve);Serial.print(
79 diez);Serial.println(once);
80 delay(100);
81 }
82

```

A.7 Código para probar el sensor de presión del fol:

```

1
2 //aquí voy a probar el ronco
3 //aquí cosas del sensor de presión
4 #include <Q2HX711.h>
5 #include <MIDI.h>
6 const byte pin_Datos = A2; // En la placa del sensor es el pin OUT
7 const byte pin_Reloj = A3; // En la placa del sensor es SCK
8 Q2HX711 hx711(pin_Datos, pin_Reloj);
9 MIDI_CREATE_DEFAULT_INSTANCE();
10
11 bool ronco; //variable del estado del ronco
12 bool roncoanterior;
13
14 float presionInicial;
15
16 void setup() {
17     MIDI.begin();
18     Serial.begin(115200);
19     presionInicial=hx711.read() / 1000.0; //defino una presión inicial
20     dentro de Fol
21
22     ronco = LOW; //inicializo a 0 el ronco, apagado
23 }
24
25 void loop() {
26     if ((hx711.read()/1000.0) >presionInicial+1000) { //si la presión del
27         sensor supera la inicial mas mil
28         ronco = HIGH; //activo el ronco
29     } else {
30         ronco = LOW;
31     }
32
33     //solo envío mensajes de control MIDI cuando hay un flanco de subida o
34     bajada
35     if (ronco != roncoanterior && ronco == HIGH) {
36         MIDI.sendControlChange(3, 127, 1);
37     }
38     else if(ronco != roncoanterior && ronco == LOW){
39         MIDI.sendControlChange(3, 0, 1);

```

```

40     }
41
42     roncoanterior = ronco;
43     // Serial.println(hx711.read() / 1000.0);
44 }
45

```

A.8 Código para probar cuatro pulsadores:

```

1
2 int buttonState10 = 0; //variables de estado de los botones
3 int buttonState11 = 0; //el numero indica el pin al que se conectan
4 int buttonState12 = 0;
5 int buttonState13 = 0;
6
7 void setup() {
8     Serial.begin(9600); //comenzamos comunicación por puerto serie
9     pinMode(10, INPUT); //definimos los pines de los botones como entradas
10    pinMode(11, INPUT);
11    pinMode(12, INPUT);
12    pinMode(13, INPUT);
13 }
14
15 void loop() {
16     buttonState10 = digitalRead(10); //leemos el estado de los botones
17     buttonState11 = digitalRead(11);
18     buttonState12 = digitalRead(12);
19     buttonState13 = digitalRead(13);
20
21     if (buttonState10 == HIGH) { //si el boton esta pulsado escribo el
22         numero de pin al monitor serie
23         Serial.println(10);
24     } else if (buttonState11 == HIGH) {
25         Serial.println(11);
26     } else if (buttonState12 == HIGH) {
27         Serial.println(12);
28     } else if (buttonState13 == HIGH) {
29         Serial.println(13);}
30     else {
31         Serial.println(0);
32     }
33 }
34

```

A.9 Código de prueba del sensor de presión para el fol

```

1
2 //aquí voy a probar el soplado y el ronco
3 //aquí cosas del sensor de presion
4 ////////////////////////////////////

```



```

5  #include <Q2HX711.h>
6  #include <MIDI.h>
7  const byte pin_Datos = A2; // En la plata del sensor es el pin OUT
8  const byte pin_Relej = A3; // En la placa del sensor es SCK
9  Q2HX711 hx711(pin_Datos, pin_Relej);
10
11 void setup() {
12     Serial.begin(9600);
13 }
14
15 void loop() {
16     Serial.println(hx711.read() / 1000.0);
17     //medidas para incrementos
18     //bastante inflado
19     //sin tocar no llega a 10500, debajo del brazo no llega a 11500, si
20     aprieto pasa de 12500
21
22     //poco inflado
23     //sin tocar no llega a 9000, debajo del brazo no llega a 10500, si
24     aprieto pasa de 10500
25
26     //muy inflado
27     //sin apretar no llega a 13000, debajo del brazo no llega a 14000, si
28     aprieto pasa de 14000
29 }
30

```

A.10 Código para probar la pantalla LCD:

```

1  //prueba de la pantalla
2  #include <LiquidCrystal.h> //incluimos libreria
3
4
5  //Creamos el objeto LCD con los números correspondientes (rs, en, d4, d5
6  , d6, d7)
7  LiquidCrystal lcd(8, 9, 4, 5, 6, 7);
8  byte nota[8] = { //definimos los caracteres especiales que necesitamos
9      B00010,
10     B00011,
11     B00010,
12     B00010,
13     B01110,
14     B11110,
15     B11110,
16     B01100
17 };
18 byte flechaArriba[8] = {
19     B00000,
20     B00100,
21     B01110,

```

```
22     B10101,
23     B00100,
24     B00100,
25     B00100,
26     B00000
27 };
28 byte flechaAbajo[8] = {
29     B00000,
30     B00100,
31     B00100,
32     B00100,
33     B10101,
34     B01110,
35     B00100,
36     B00000
37 };
38 byte flechaDerecha[8] = {
39     B00000,
40     B00000,
41     B00100,
42     B00010,
43     B11111,
44     B00010,
45     B00100,
46     B00000
47 };
48 byte flechaIzquierda[8] = {
49     B00000,
50     B00000,
51     B00100,
52     B01000,
53     B11111,
54     B01000,
55     B00100,
56     B00000
57 };
58
59 void setup() {
60
61     lcd.createChar(0, nota); //creamos los caracteres personalizados y los
62         metemos en la memoria del LCD
63     lcd.createChar(1, flechaArriba);
64     lcd.createChar(2, flechaAbajo);
65     lcd.createChar(3, flechaDerecha);
66     lcd.createChar(4, flechaIzquierda);
67     lcd.begin(16, 2); //iniciamos el LCD
68     pantallaInicial(); //pintamos la pantalla inicial
69 }
70
71 void loop() {
```

```

72 }
73
74 void pantallaInicial() { //funcion de la pantalla inicial
75     lcd.setCursor(0, 0); //ponemos el cursor en la posicion
76     lcd.print("Gaita");
77     lcd.setCursor(7, 0);
78     lcd.write(byte(0));
79     lcd.setCursor(0, 1);
80     lcd.print("electronica MIDI");
81 }
82

```

A.11 Código del instrumento final

```

1  //version final 1 desarrollada por César Núñez Bayón
2  //gaita gallega electronica midi
3
4
5  ////////////////////////////////////
6  //incluimos librerias de I2C, MPR121, midi, sensor de presion, pantalla
7  y debounce de botones
8  #include <MIDI.h>
9  #include <Q2HX711.h>
10 #include <Wire.h>
11 #include "Adafruit_MPR121.h"
12 #include <LiquidCrystal.h>
13 #include "Adafruit_Debounce.h"
14 ////////////////////////////////////
15 //cosas del modulo MPR121 de Adafruit
16 #ifndef _BV
17 #define _BV(bit) (1 << (bit))
18 #endif
19 Adafruit_MPR121 cap = Adafruit_MPR121(); //defino objeto cap del tipo
20 del modulo
21 uint16_t lasttouched = 0; //variables para poder detectar
22 cuando un pin del MPR se ha destapado
23 uint16_t currntouched = 0;
24 ////////////////////////////////////
25 //cosas para la comunicacion MIDI
26 MIDI_CREATE_DEFAULT_INSTANCE();
27 // Definicion de las notas midi
28 const int SI3 = 59;
29 const int DO4 = 60;
30 const int REB4 = 61;
31 const int RE4 = 62;
32 const int MIB4 = 63;
33 const int MI4 = 64;
34 const int FA4 = 65;
35 const int SOLB4 = 66;
36 const int SOL4 = 67;

```

```

37 const int LAB4 = 68;
38 const int LA4 = 69;
39 const int SIB4 = 70;
40 const int SI4 = 71;
41 const int D05 = 72;
42 const int RE5 = 74;
43 const int MIB5 = 75;
44 const int MI5 = 76;
45 const int FA5 = 77;
46 int nota; //variable para reproducir cada nota en funcion de los
47 agujeros tapados
48 int notaAnterior;
49 //variables de estado de los agujeros
50 int uno = 0;
51 int dos = 0;
52 int tres = 0;
53 int cuatro = 0;
54 int cinco = 0;
55 int seis = 0;
56 int siete = 0;
57 int ocho = 0;
58 bool boton[11]; //vector de los pines del MPR121
59 ////////////////////////////////////
60 //variables del sensor de presion del soprete
61 const byte pin_Datos = A2; //en el sensor es el pin OUT
62 const byte pin_Relej = A3; //en el sensor es SCK
63 Q2HX711 hx711Soprete(pin_Datos, pin_Relej); //defino objeto del tipo de
64 mi sensor
65 ////////////////////////////////////
66 //variables del sensor de presion del fol
67 const byte pin_Datos_Fol = A0; //en el sensor es el pin OUT
68 const byte pin_Relej_Fol = A1; //en el sensor es SCK
69 Q2HX711 hx711Fol(pin_Datos_Fol, pin_Relej_Fol); //defino objeto del tipo
70 de mi sensor
71
72 ////////////////////////////////////
73 //botones de la pcb. usamos la libreria debounce de adafruit
74 #define BOTON10 10
75 #define BOTON11 11
76 #define BOTON12 12
77 #define BOTON13 13
78
79 Adafruit_Debounce button1(BOTON10, HIGH);
80 Adafruit_Debounce button2(BOTON11, HIGH);
81 Adafruit_Debounce button3(BOTON12, HIGH);
82 Adafruit_Debounce button4(BOTON13, HIGH);
83 ////////////////////////////////////
84 //control del volumen mediante encoder digital
85 const int S1 = 2; //pines del arduino UNO que permiten interrupcion
86 const int S2 = 3;

```

```

87 unsigned long _lastIncReadTime = micros();
88 unsigned long _lastDecReadTime = micros();
89 int _pauseLength = 25000;
90 int _fastIncrement = 10;
91 int counter;
92 int lastCounter;
93 ///////////////////////////////////////////////////////////////////
94 //cosas de la pantalla
95 LiquidCrystal lcd(8, 9, 4, 5, 6, 7);
96 byte notasimbolo[8] = {
97     B00010,
98     B00011,
99     B00010,
100    B00010,
101    B01110,
102    B11110,
103    B11110,
104    B01100
105 };
106 byte flechaArriba[8] = {
107     B00000,
108     B00100,
109     B01110,
110     B10101,
111     B00100,
112     B00100,
113     B00100,
114     B00000
115 };
116 byte flechaAbajo[8] = {
117     B00000,
118     B00100,
119     B00100,
120     B00100,
121     B10101,
122     B01110,
123     B00100,
124     B00000
125 };
126 byte flechaDerecha[8] = {
127     B00000,
128     B00000,
129     B00100,
130     B00010,
131     B11111,
132     B00010,
133     B00100,
134     B00000
135 };
136 byte flechaIzquierda[8] = {

```

```

137     B00000,
138     B00000,
139     B00100,
140     B01000,
141     B11111,
142     B01000,
143     B00100,
144     B00000
145 };
146 int contadorPantalla; //contador para las diferentes pantallas
147 int contadorPantallaAnterior;
148 int afinacion; //contador para el menu de afinacion
149 int afinacionAnterior;
150 int modo; //contador para el menu de modo
151 int modoAnterior;
152 int metronomo; //contador para el menu del metronomo
153 int metronomoAnterior;
154 int reverb; //contador para el menu del reverb
155 int reverbAnterior;
156 int empezar;
157 int empezarAnterior;
158 int ronco;
159 int roncoAnterior;
160 int punteiro;
161 int punteiroAnterior;
162 float presionInicial;
163 const int volmax = 1000; //volumen maximo de aire con que se puede
164 llenar
165 double volaire; //variable para almacenar el volumen de aire
166 double volaireAnterior;
167
168 unsigned long previousMillis = 0; // Almacena la última vez que se
169 ejecutó la función
170 const long interval = 500; // Intervalo de tiempo en milisegundos
171
172 ////////////////////////////////////
173 void setup() {
174
175     pinMode(S1, INPUT_PULLUP); //pin del
176     encoder S1
177     pinMode(S2, INPUT_PULLUP); //pin del
178     encoder S2
179     attachInterrupt(digitalPinToInterrupt(S1), read_encoder, CHANGE); //
180     habilito interrupcion en el pin 2
181     attachInterrupt(digitalPinToInterrupt(S2), read_encoder, CHANGE); //
182     habilito interrupcion en el pin 3
183
184     button1.begin();
185     button2.begin();
186     button3.begin();

```

```

187 button4.begin();
188
189 MIDI.begin();           //inicio de la comunicacion MIDI
190 Serial.begin(115200); //puerto serie a 115200 baudios
191 cap.begin(0x5A);        //inicio de la comunicacion con MPR121
192
193 lcd.createChar(0, notasimbolo);
194 lcd.createChar(1, flechaArriba);
195 lcd.createChar(2, flechaAbajo);
196 lcd.createChar(3, flechaDerecha);
197 lcd.createChar(4, flechaIzquierda);
198 lcd.begin(16, 2);
199 pantallaInicial();
200
201 contadorPantalla = 0;
202 contadorPantallaAnterior = 0;
203 afinacion = 0;
204 afinacionAnterior = 0;
205 modo = 0;
206 modoAnterior = 0;
207 counter = 0;
208 lastCounter = 0;
209 metronomo = 0;
210 metronomoAnterior = 0;
211 reverb = 0;
212 reverbAnterior = 0;
213 nota = 72;
214 notaAnterior = 70;
215 empezar = 0;
216 empezarAnterior = 0;
217 ronco = 0;
218 roncoAnterior = 0;
219 punteiro = 0;
220 punteiroAnterior = 0;
221 volaire = 0; //inicializo el volumen de aire en 0
222 volaireAnterior = 0;
223 nota = D05; //incializo la variable nota en D05s
224
225 presionInicial = hx711Fol.read() / 1000.0;
226
227 MIDI.sendNoteOn(nota, 60, 1);
228 delay(10);
229 MIDI.sendNoteOff(nota, 60, 1);
230 MIDI.sendControlChange(16, 62, 1);
231 }
232
233 void loop() {
234
235     unsigned long currentMillis = millis();
236     //////////////////////////////////////

```

```

237 //pantallas y menus
238 button1.update();
239 button2.update();
240 button3.update();
241 button4.update();
242 if (contadorPantalla < 0) {
243     contadorPantalla = 7;
244 }
245 if (contadorPantalla > 7) {
246     contadorPantalla = 0;
247 }
248 if (button4.justPressed()) { //programacion de cambio de pantalla
249     mediante los botones laterales
250     contadorPantalla = contadorPantalla + 1;
251 } else if (button2.justPressed()) {
252     contadorPantalla = contadorPantalla - 1;
253 }
254 if (contadorPantalla == 2) { //en la pantalla 2 si pulso los botones
255     verticales cambio la afinacion
256     if (button1.justPressed()) {
257         afinacion = afinacion + 1;
258         if (afinacion < 0) { afinacion = 4; }
259         if (afinacion > 4) { afinacion = 0; }
260     } else if (button3.justPressed()) {
261         afinacion = afinacion - 1;
262         if (afinacion < 0) { afinacion = 4; }
263         if (afinacion > 4) { afinacion = 0; }
264     }
265 }
266
267 if (contadorPantalla == 3) { //en la pantalla 3 si pulso los botones
268     verticales cambio el modo
269     if (button1.justPressed()) {
270         modo = modo + 1;
271         if (modo < 0) { modo = 2; }
272         if (modo > 2) { modo = 0; }
273     } else if (button3.justPressed()) {
274         modo = modo - 1;
275         if (modo < 0) { modo = 2; }
276         if (modo > 2) { modo = 0; }
277     }
278 }
279
280 if (contadorPantalla == 5) { //en la pantalla 5 si pulso los botones
281     verticales cambio el metronomo de ON a OFF
282     if (button1.justPressed()) {
283         metronomo = metronomo + 1;
284         if (metronomo < 0) { metronomo = 1; }
285         if (metronomo > 1) { metronomo = 0; }
286     } else if (button3.justPressed()) {

```



```

287     metronomo = metronomo - 1;
288     if (metronomo < 0) { metronomo = 1; }
289     if (metronomo > 1) { metronomo = 0; }
290 }
291 }
292
293 if (contadorPantalla == 6) { //en la pantalla 6 si pulso los botones
294     verticales cambio el reverb de ON a OFF
295     if (button1.justPressed()) {
296         reverb = reverb + 1;
297         if (reverb < 0) { reverb = 1; }
298         if (reverb > 1) { reverb = 0; }
299     } else if (button3.justPressed()) {
300         reverb = reverb - 1;
301         if (reverb < 0) { reverb = 1; }
302         if (reverb > 1) { reverb = 0; }
303     }
304 }
305
306 if (contadorPantalla == 7) { //en la pantalla 7 empiezo o acabo de
307     tocar
308     if (button1.justPressed()) {
309         empezar = empezar + 1;
310         if (empezar < 0) { empezar = 1; }
311         if (empezar > 1) { empezar = 0; }
312     } else if (button3.justPressed()) {
313         empezar = empezar - 1;
314         if (empezar < 0) { empezar = 1; }
315         if (empezar > 1) { empezar = 0; }
316     }
317 }
318
319 switch (contadorPantalla) {
320     case 0:
321         if (contadorPantalla != contadorPantallaAnterior) {
322             lcd.clear();
323             pantallaInicial();
324         } else {
325             }
326         break;
327     case 1:
328         if (contadorPantalla != contadorPantallaAnterior) {
329             lcd.clear();
330             pantallaInstrucciones();
331         } else {
332             }
333         break;
334     case 2:
335         if (contadorPantalla != contadorPantallaAnterior || afinacion !=
336             afinacionAnterior) {

```

```

337     lcd.clear();
338     pantallaAfinacion();
339   } else {
340   }
341   break;
342 case 3:
343   if (contadorPantalla != contadorPantallaAnterior || modo !=
344       modoAnterior) {
345     lcd.clear();
346     pantallaModo();
347   } else {
348   }
349   break;
350 case 4:
351   if (contadorPantalla != contadorPantallaAnterior || counter !=
352       lastCounter) {
353     lcd.clear();
354     pantallaVolumen();
355   } else {
356   }
357   break;
358 case 5:
359   if (contadorPantalla != contadorPantallaAnterior || metronomo !=
360       metronomoAnterior) {
361     lcd.clear();
362     pantallaMetronomo();
363   } else {
364   }
365   break;
366 case 6:
367   if (contadorPantalla != contadorPantallaAnterior || reverb !=
368       reverbAnterior) {
369     lcd.clear();
370     pantallaReverb();
371   } else {
372   }
373   break;
374 case 7:
375   if (contadorPantalla != contadorPantallaAnterior || empezar !=
376       empezarAnterior) {
377     lcd.clear();
378     pantallaEmpezar();
379   } else {
380   }
381   break;
382 }
383
384 //////////////////////////////////////
385 //funciones MIDI de los menus
386 if (metronomo != metronomoAnterior) {

```

```

387     if (metronomo == 0) {
388         MIDI.sendControlChange(14, 0, 1);
389     } else if (metronomo == 1) {
390         MIDI.sendControlChange(14, 127, 1);
391     }
392 }
393
394 if (reverb != reverbAnterior) {
395     if (reverb == 0) {
396         MIDI.sendControlChange(15, 0, 1);
397     } else if (reverb == 1) {
398         MIDI.sendControlChange(15, 127, 1);
399     }
400 }
401
402 if (afinacion != afinacionAnterior) {
403     if (afinacion == 0) {
404         MIDI.sendControlChange(16, 62, 1);
405     } else if (afinacion == 1) {
406         MIDI.sendControlChange(16, 72, 1);
407     } else if (afinacion == 2) {
408         MIDI.sendControlChange(16, 55, 1);
409     } else if (afinacion == 3) {
410         MIDI.sendControlChange(16, 60, 1);
411     } else if (afinacion == 4) {
412         MIDI.sendControlChange(16, 50, 1);
413     }
414 }
415
416 // //////////////////////////////////////
417 //reading del modulo mpr121
418 currouched = cap.touched(); //leo que
419     electrodos se estan tocando
420 for (uint8_t i = 0; i < 8; i++) { //recorro las 11
421     entradas del modulo mpr121
422     if ((currouched & _BV(i)) && !(lasttouched & _BV(i))) { //si se ha
423         tocado y antes no lo estaba
424         boton[i] = 1; //pongo a 1 la
425             componente del vector correspondiente a el electrodo tocado
426     }
427     if (!(currouched & _BV(i)) && (lasttouched & _BV(i))) { //si estaba
428         tocado y ahora ya no
429         boton[i] = 0; //pongo a 0 la
430             componente del vector correspondiente a el electrodo soltado
431     }
432 }
433 lasttouched = currouched; //actualizo la variable de tocado
434
435 // //traduzco de las componentes del vector a variables mas intuitivas
436 ocho = boton[0];

```

```

437     siete = boton[1];
438     seis = boton[2];
439     cinco = boton[3];
440     cuatro = boton[4];
441     tres = boton[5];
442     dos = boton[6];
443     uno = boton[7];
444
445     //modo cero solo toco mediante el punteiro
446     if (modo == 0) {
447         if (empezar == 1) {
448             nota = generaNotas();
449             if (nota != notaAnterior) {
450                 MIDI.sendNoteOn(nota, 60, 1);
451             }
452         }
453     }
454     if (modo == 0) {
455         if (empezar == 1 && empezarAnterior == 0) {
456             MIDI.sendControlChange(3, 127, 1);
457         } else if (empezar == 0 && empezarAnterior == 1) {
458             MIDI.sendNoteOff(nota, 60, 1);
459             MIDI.sendControlChange(3, 0, 1);
460         }
461     }
462
463     //modo 1 toco mediante el punteiro y el fol apretandolo
464     if (modo == 1) {
465         if (empezar == 1) {
466             nota = generaNotas();
467             if (currentMillis - previousMillis >= interval) {
468                 previousMillis = currentMillis;
469                 ronco = compruebaPresionFol();
470                 punteiro = compruebaPresionPunteiro();
471             }
472             if (ronco != roncoAnterior && ronco == HIGH) {
473                 MIDI.sendControlChange(3, 127, 1);
474             } else if (ronco != roncoAnterior && ronco == LOW) {
475                 MIDI.sendControlChange(3, 0, 1);
476             }
477             if (punteiro == HIGH && punteiroAnterior == LOW) {
478                 MIDI.sendNoteOn(nota, 60, 1);
479             } else if (punteiro == LOW && punteiroAnterior == HIGH) {
480                 MIDI.sendNoteOff(nota, 60, 1);
481             }
482             if (punteiro == HIGH) {
483                 if (nota != notaAnterior) {
484                     MIDI.sendNoteOn(nota, 60, 1);
485                 }
486             }

```

```

487     } else if (empezar == 0 && empezarAnterior == 1) {
488         MIDI.sendNoteOff(nota, 60, 1);
489     }
490 }
491
492
493
494 //modo 2 toco tapando los agujeros del punteiro, apretando el fol y
495 soplando por el tubo
496 if (modo == 2) {
497     if (empezar == 1) {
498         nota = generaNotas();
499         if (currentMillis - previousMillis >= interval) {
500             previousMillis = currentMillis;
501             volaire = compruebaPresionSoprete();
502             ronco = compruebaPresionFol();
503             punteiro = compruebaPresionPunteiro();
504         }
505         if (volaire > 0) {
506             if (ronco != roncoAnterior && ronco == HIGH) {
507                 MIDI.sendControlChange(3, 127, 1);
508             } else if (ronco != roncoAnterior && ronco == LOW) {
509                 MIDI.sendControlChange(3, 0, 1);
510             }
511             if (punteiro == HIGH && punteiroAnterior == LOW) {
512                 MIDI.sendNoteOn(nota, 60, 1);
513             } else if (punteiro == LOW && punteiroAnterior == HIGH) {
514                 MIDI.sendNoteOff(nota, 60, 1);
515             }
516             if (punteiro == HIGH) {
517                 if (nota != notaAnterior) {
518                     MIDI.sendNoteOn(nota, 60, 1);
519                 }
520             }
521             } else if (volaire == 0 && volaireAnterior > 0) {
522                 MIDI.sendControlChange(3, 0, 1);
523                 MIDI.sendNoteOff(nota, 60, 1);
524             }
525         } else if (empezar == 0 && empezarAnterior == 1) {
526             MIDI.sendNoteOff(nota, 60, 1);
527             MIDI.sendControlChange(3, 0, 1);
528         }
529     }
530
531 ////////////////////////////////////
532 //encoder para control del volumen
533 if (counter != lastCounter) {
534     lastCounter = counter;
535     MIDI.sendControlChange(1, counter, 1);
536 }

```

```

537 ///////////////////////////////////////////////////////////////////
538
539 contadorPantallaAnterior = contadorPantalla;
540 afinacionAnterior = afinacion;
541 modoAnterior = modo;
542 metronomoAnterior = metronomo;
543 reverbAnterior = reverb;
544 notaAnterior = nota;
545 empezarAnterior = empezar;
546 roncoAnterior = ronco;
547 punteiroAnterior = punteiro;
548 volaireAnterior = volaire;
549 }
550
551 ///////////////////////////////////////////////////////////////////
552 int generaNotas() {
553     // //generacion de las notas en funcion del agujero tapado
554     // //en funcion de si tapamos agujero 8 tenemos la 1a octava o la 2a
555     if (ocho == 1) //primera octava, Seguimos el esquema dado en
556         dixitacion
557     {
558         if (uno == 1 && dos == 1 && tres == 1 && cuatro == 1 && cinco == 1
559             && seis == 1 && siete == 1) {
560             nota = SI3; //
561         } else if (uno == 0 && dos == 1 && tres == 1 && cuatro == 1 && cinco
562             == 1 && seis == 1 && siete == 1) {
563             nota = DO4; //
564         } else if ((uno == 0 && dos == 0 && tres == 1 && cuatro == 1 &&
565             cinco == 1 && seis == 1 && siete == 1) || (uno == 1 && dos == 0
566             && tres == 1 && cuatro == 0 && cinco == 1 && seis == 1 && siete
567             == 1)) {
568             nota = RE4; //
569         } else if (uno == 0 && dos == 1 && tres == 0 && cuatro == 1 && cinco
570             == 1 && seis == 1 && siete == 1) {
571             nota = MIB4;
572         } else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 1 &&
573             cinco == 1 && seis == 1 && siete == 1) || (uno == 1 && dos == 0
574             && tres == 0 && cuatro == 1 && cinco == 1 && seis == 1 && siete
575             == 1)) {
576             nota = MI4; //
577         } else if ((uno == 1 && dos == 1 && tres == 1 && cuatro == 0 &&
578             cinco == 1 && seis == 1 && siete == 1) || (uno == 0 && dos == 1
579             && tres == 1 && cuatro == 0 && cinco == 1 && seis == 1 && siete
580             == 1) || (uno == 0 && dos == 0 && tres == 1 && cuatro == 0 &&
581             cinco == 1 && seis == 1 && siete == 1)) {
582             nota = MI4; //
583         } else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 0 &&
584             cinco == 1 && seis == 1 && siete == 1) || (uno == 1 && dos == 0
585             && tres == 0 && cuatro == 0 && cinco == 1 && seis == 1 && siete
586             == 1)) {

```

```

587     nota = FA4; //
588 } else if (uno == 0 && dos == 0 && tres == 0 && cuatro == 1 && cinco
589     == 0 && seis == 1 && siete == 1) {
590     nota = SOLB4;
591 } else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 0 &&
592     cinco == 0 && seis == 1 && siete == 1) || (uno == 1 && dos == 0
593     && tres == 0 && cuatro == 0 && cinco == 0 && seis == 1 && siete
594     == 1)) {
595     nota = SOL4; //
596 } else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 0 &&
597     cinco == 1 && seis == 0 && siete == 1) || (uno == 1 && dos == 0
598     && tres == 0 && cuatro == 0 && cinco == 1 && seis == 0 && siete
599     == 1)) {
600     nota = LAB4;
601 } else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 0 &&
602     cinco == 0 && seis == 0 && siete == 1) || (uno == 1 && dos == 0
603     && tres == 0 && cuatro == 0 && cinco == 0 && seis == 0 && siete
604     == 1)) {
605     nota = LA4; //
606 } else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 0 &&
607     cinco == 0 && seis == 1 && siete == 0) || (uno == 1 && dos == 0
608     && tres == 0 && cuatro == 0 && cinco == 0 && seis == 1 && siete
609     == 0)) {
610     nota = SIB4;
611 } else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 0 &&
612     cinco == 0 && seis == 0 && siete == 0) || (uno == 1 && dos == 0
613     && tres == 0 && cuatro == 0 && cinco == 0 && seis == 0 && siete
614     == 0)) {
615     nota = SI4; //
616 } else if (uno == 1 && dos == 0 && tres == 0 && cuatro == 0 && cinco
617     == 0 && seis == 0 && siete == 0) {
618     nota = SI4; //
619 }
620 //permitir picados a partir del MI4
621 else if ((uno == 0 && dos == 0 && tres == 0 && cuatro == 1 && cinco
622     == 1 && seis == 1 && siete == 0) || (uno == 0 && dos == 0 && tres
623     == 0 && cuatro == 0 && cinco == 1 && seis == 1 && siete == 0)) {
624     nota = SIB4;
625 } else {
626     nota = SIB4; //para cualquier combinacion de agujeros no
627     contemplada de momento
628 }
629 } else if (ocho == 0) //segunda octava
630 {
631     if (uno == 0 && dos == 0 && tres == 0 && cuatro == 0 && cinco == 0
632         && seis == 0 && siete == 0) {
633         nota = D05; //
634     } else if (uno == 1 && dos == 1 && tres == 1 && cuatro == 1 && cinco
635         == 1 && seis == 1 && siete == 1) {
636         nota = SI4;

```

```
637     } else if (uno == 0 && dos == 0 && tres == 1 && cuatro == 1 && cinco
638         == 1 && seis == 1 && siete == 1) {
639         nota = RE5;
640     } else if (uno == 0 && dos == 1 && tres == 0 && cuatro == 1 && cinco
641         == 1 && seis == 1 && siete == 1) {
642         nota = MIB5;
643     } else if (uno == 0 && dos == 0 && tres == 0 && cuatro == 1 && cinco
644         == 1 && seis == 1 && siete == 1) {
645         nota = MI5;
646     } else if (uno == 0 && dos == 0 && tres == 0 && cuatro == 0 && cinco
647         == 1 && seis == 1 && siete == 1) {
648         nota = FA5;
649     } else {
650         nota = D05;
651     }
652 }
653 return nota;
654 }
655
656 int compruebaPresionFol() {
657     if ((hx711Fol.read() / 1000.0) > presionInicial + 900) {
658         ronco = 1;
659     } else {
660         ronco = 0;
661     }
662     return ronco;
663 }
664
665 bool compruebaPresionPunteiro() {
666     if ((hx711Fol.read() / 1000.0) > presionInicial + 1000) {
667         punteiro = HIGH;
668     } else {
669         punteiro = LOW;
670     }
671     return punteiro;
672 }
673
674 int compruebaPresionSoprete() {
675     if (hx711Soprete.read() / 1000.0 > 10500) {
676         volaire = volaire + 100;
677         if (volaire >= volmax) {
678             volaire = volmax;
679         }
680     } else {
681         volaire = volaire - 25;
682         if (volaire < 0) {
683             volaire = 0;
684         }
685     }
686     return volaire;
```



```

687 }
688
689 ///////////////////////////////////////////////////////////////////
690 //funcion del encoder digital para variar el volumen
691 void read_encoder() {
692     // Encoder interrupt routine for both pins. Updates counter
693     // if they are valid and have rotated a full indent
694     static uint8_t old_AB = 3;
695
696     // Lookup
697     table index
698     static int8_t encval = 0;
699
700     // Encoder
701     value
702     static const int8_t enc_states[] = { 0, -1, 1, 0, 1, 0, 0, -1, -1, 0,
703     0, 1, 0, 1, -1, 0 }; // Lookup table
704
705     old_AB <= 2; // Remember previous state
706
707     if (digitalRead(S1)) old_AB |= 0x02; // Add current state of pin A
708     if (digitalRead(S2)) old_AB |= 0x01; // Add current state of pin B
709
710     encval += enc_states[(old_AB & 0x0f)];
711
712     // Update counter if encoder has rotated a full indent, that is at
713     // least 4 steps
714     if (encval > 3) { // Four steps forward
715         int changevalue = 1;
716         if ((micros() - _lastIncReadTime) < _pauseLength) {
717             changevalue = _fastIncrement * changevalue;
718         }
719         _lastIncReadTime = micros();
720         counter = counter + changevalue; // Update counter
721         if (counter < 0) {
722             counter = 0;
723         }
724         if (counter > 127) {
725             counter = 127;
726         }
727         encval = 0;
728     } else if (encval < -3) { // Four steps backward
729         int changevalue = -1;
730         if ((micros() - _lastDecReadTime) < _pauseLength) {
731             changevalue = _fastIncrement * changevalue;
732         }
733         _lastDecReadTime = micros();
734         counter = counter + changevalue; // Update counter
735         if (counter < 0) {
736             counter = 0;
737         }
738         if (counter > 127) {

```

```

737     counter = 127;
738 }
739     encval = 0;
740 }
741 }
742
743 ///////////////////////////////////////////////////////////////////
744 //funciones para las pantallas
745 void pantallaInicial() {
746     lcd.setCursor(0, 0);
747     lcd.print("Gaita");
748     lcd.setCursor(7, 0);
749     lcd.write(byte(0));
750     lcd.setCursor(0, 1);
751     lcd.print("electronica MIDI");
752 }
753 void pantallaInstrucciones() {
754     lcd.setCursor(0, 0);
755     lcd.print("Cambia menu");
756     lcd.setCursor(13, 0);
757     lcd.write(byte(4));
758     lcd.setCursor(14, 0);
759     lcd.write(byte(3));
760
761     lcd.setCursor(0, 1);
762     lcd.print("Ajusta valor");
763     lcd.setCursor(13, 1);
764     lcd.write(byte(1));
765     lcd.setCursor(14, 1);
766     lcd.write(byte(2));
767 }
768
769 void pantallaEmpezar() {
770     lcd.setCursor(0, 0);
771     lcd.print("Empezar");
772     lcd.setCursor(0, 1);
773     lcd.print("a tocar:");
774     lcd.setCursor(9, 1);
775     lcd.print("OFF");
776     if (empezar != empezarAnterior || empezar != 0) {
777         if (empezar == 0) {
778             lcd.setCursor(9, 1);
779             lcd.print("OFF");
780         } else if (empezar == 1) {
781             lcd.clear();
782             lcd.setCursor(0, 0);
783             lcd.print("Empezar");
784             lcd.setCursor(0, 1);
785             lcd.print("a tocar:");
786             lcd.setCursor(9, 1);

```

```
787     lcd.print("ON");
788   }
789 }
790 }
791
792 void pantallaAfinacion() {
793   lcd.setCursor(0, 0);
794   lcd.print("Afinacion:");
795   lcd.setCursor(11, 0);
796   lcd.print("DO");
797   if (afinacion != afinacionAnterior || afinacion != 0) {
798     if (afinacion == 0) {
799       lcd.setCursor(11, 0);
800       lcd.print("DO");
801     } else if (afinacion == 1) {
802       lcd.setCursor(11, 0);
803       lcd.print("RE");
804     } else if (afinacion == 2) {
805       lcd.setCursor(11, 0);
806       lcd.print("SIb");
807     } else if (afinacion == 3) {
808       lcd.setCursor(11, 0);
809       lcd.print("SI");
810     } else if (afinacion == 4) {
811       lcd.setCursor(11, 0);
812       lcd.print("LA");
813     }
814   }
815 }
816
817 void pantallaModo() {
818   lcd.setCursor(0, 0);
819   lcd.print("Modo:");
820   lcd.setCursor(6, 0);
821   lcd.print("punteiro");
822   if (modo != modoAnterior || modo != 0) {
823     if (modo == 0) {
824       lcd.setCursor(6, 0);
825       lcd.print("punteiro");
826     } else if (modo == 1) {
827       lcd.setCursor(6, 0);
828       lcd.print("punteiro");
829       lcd.setCursor(0, 1);
830       lcd.print("y fol");
831     } else if (modo == 2) {
832       lcd.setCursor(6, 0);
833       lcd.print("punteiro");
834       lcd.setCursor(0, 1);
835       lcd.print("fol y soprete");
836     }
837   }
838 }
```

```
837     }
838 }
839
840 void pantallaVolumen() {
841     lcd.setCursor(0, 0);
842     lcd.print("Volumen:");
843     lcd.setCursor(0, 1);
844     lcd.print("gira el encoder");
845     if (counter != lastCounter) {
846         lcd.setCursor(9, 0);
847         lcd.print(int(counter * 100 / 127));
848         lcd.setCursor(13, 0);
849         lcd.print("%");
850     }
851 }
852
853 void pantallaMetronomo() {
854     lcd.setCursor(0, 0);
855     lcd.print("Metronomo:");
856     lcd.setCursor(11, 0);
857     lcd.print("OFF");
858     if (metronomo != metronomoAnterior || metronomo != 0) {
859         if (metronomo == 0) {
860             lcd.setCursor(11, 0);
861             lcd.print("OFF");
862         } else if (metronomo == 1) {
863             lcd.clear();
864             lcd.setCursor(0, 0);
865             lcd.print("Metronomo:");
866             lcd.setCursor(11, 0);
867             lcd.print("ON");
868         }
869     }
870 }
871
872 void pantallaReverb() {
873     lcd.setCursor(0, 0);
874     lcd.print("2a voz:");
875     lcd.setCursor(8, 0);
876     lcd.print("OFF");
877     if (reverb != reverbAnterior || reverb != 0) {
878         if (reverb == 0) {
879             lcd.setCursor(8, 0);
880             lcd.print("OFF");
881         } else if (reverb == 1) {
882             lcd.clear();
883             lcd.setCursor(0, 0);
884             lcd.print("2a voz:");
885             lcd.setCursor(8, 0);
886             lcd.print("ON");
```

887

}

888

}

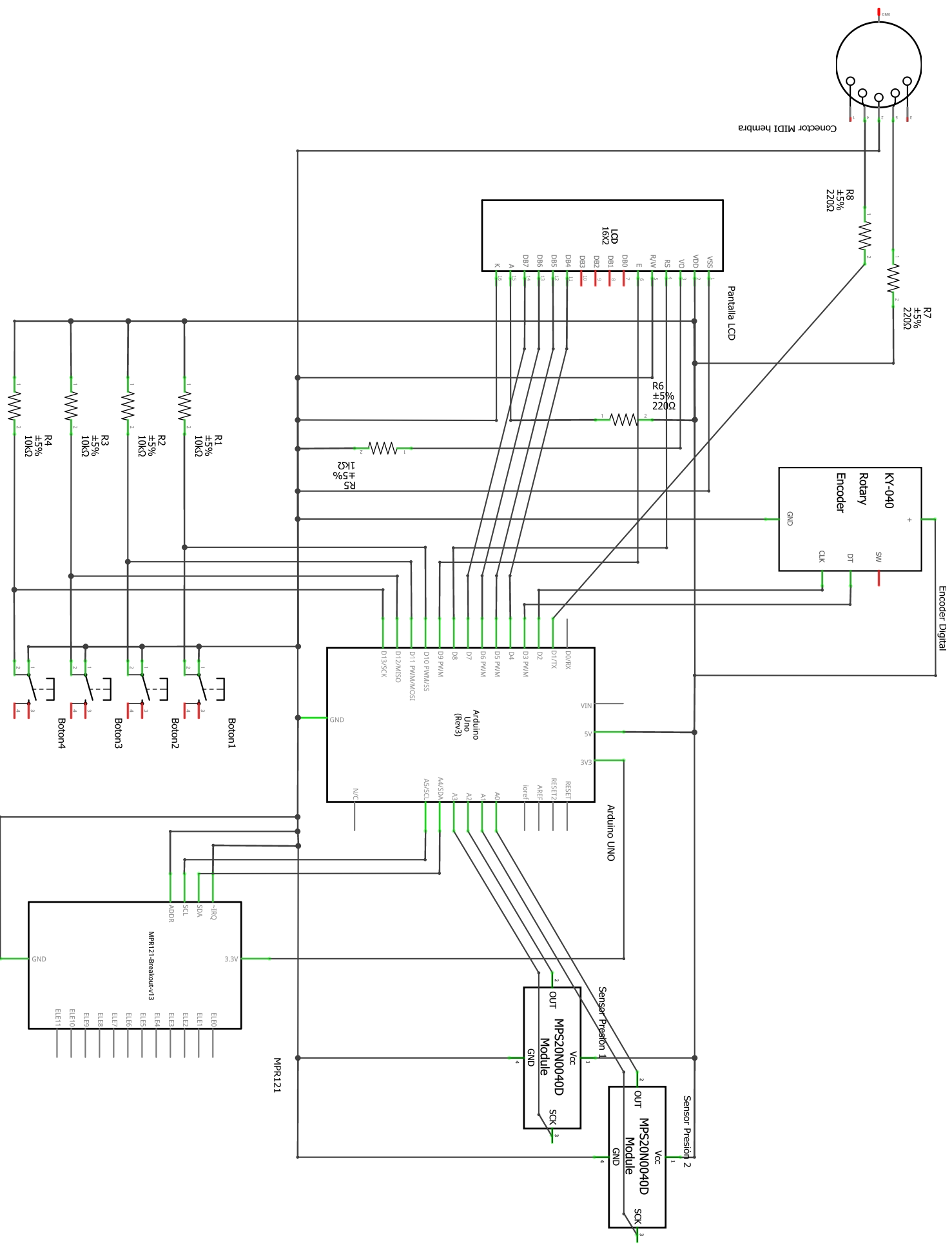
889

}

890

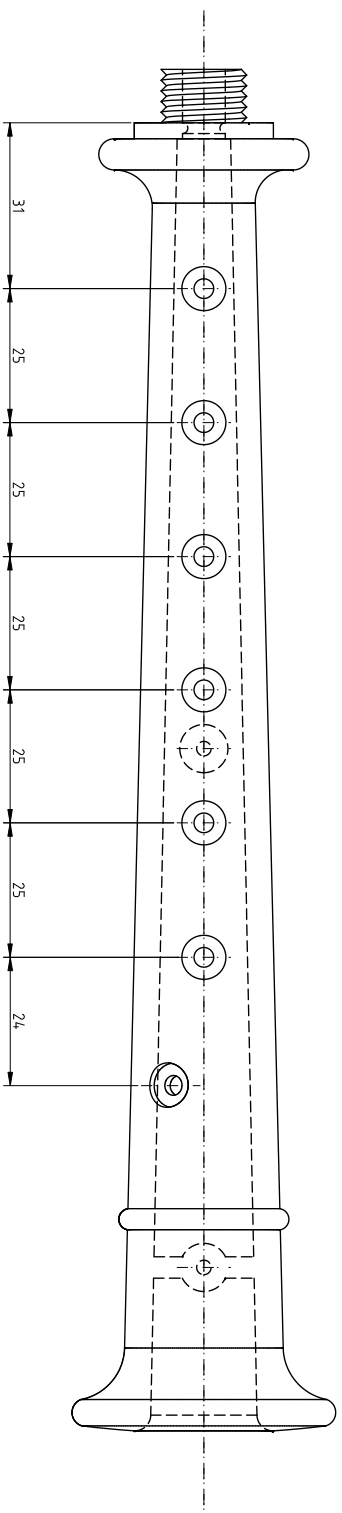
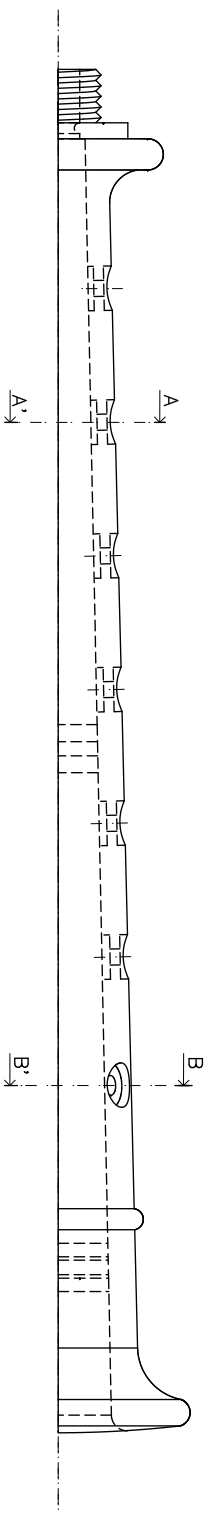
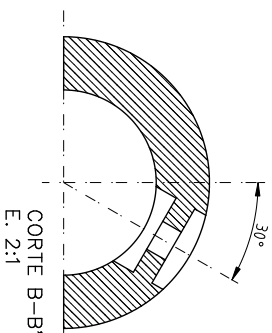
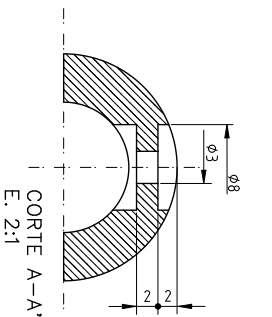
Apéndice B

**Esquemático de conexión completa del
instrumento**

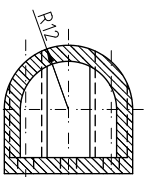
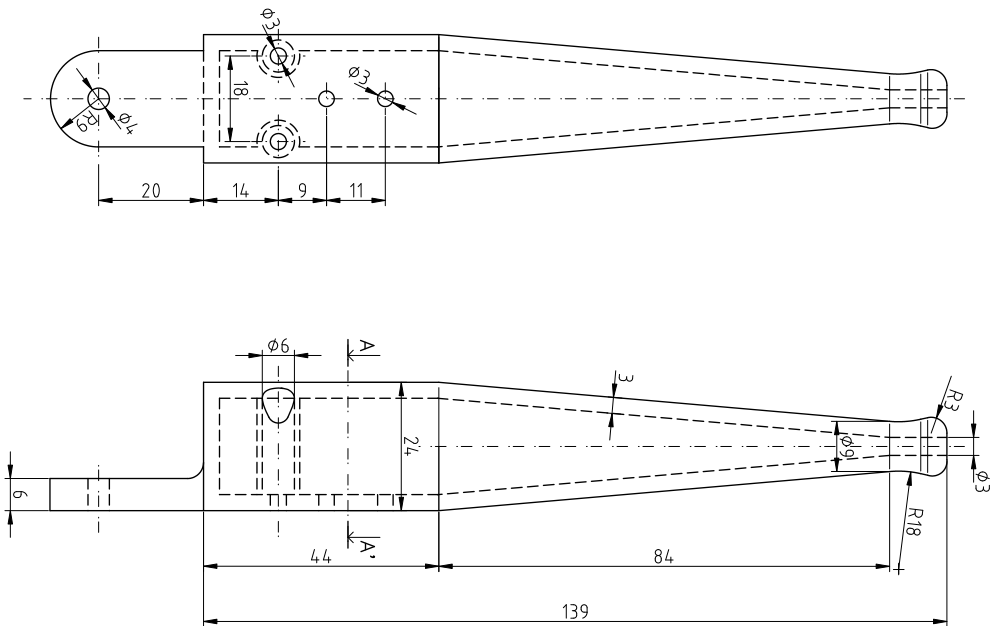


Apéndice C

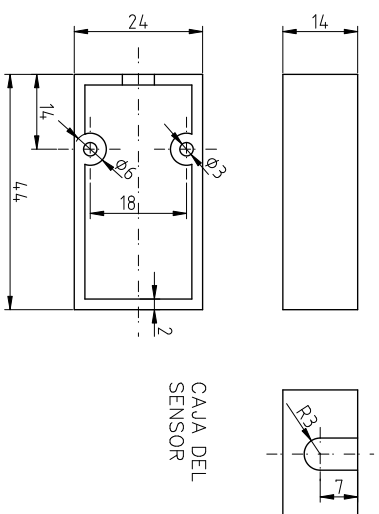
Planos de las piezas del instrumento



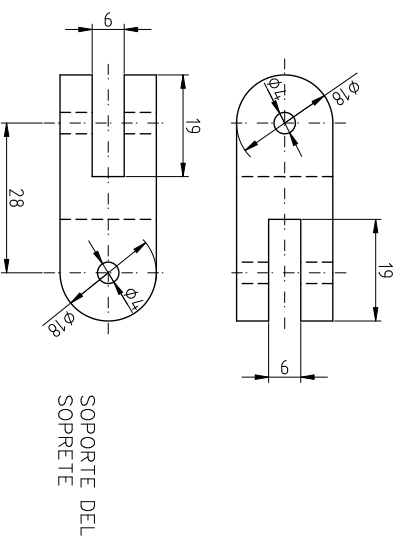
GRADO EN INGENIERÍA ELECTRÓNICA, ROBOTICA Y MECATRÓNICA		TÍTULO:	CAJA ELECTRÓNICA MIDI		FECHA:
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA		PLANO:	PUNTEIRO.		JUNIO 2024
UNIVERSIDAD DE SEVILLA		AUTOR:	PIEZA SUPERIOR		ESCALA:
TRABAJO FIN DE GRADO		CÉSAR NÚÑEZ BAYÓN		1:1	
				2	



CORTE A-A'



CAJA DEL
SENSOR



SOPORTE DEL
SOPRETE

GRADO EN INGENIERIA ELECTRÓNICA, ROBÓTICA Y MECATRÓNICA	TÍTULO: GAITA ELECTRÓNICA MIDI	FECHA: JUNIO 2024
ESCUELA TÉCNICA SUPERIOR DE INGENIERIA	PLANO: SOPRETE	ESCALA: 1:1
UNIVERSIDAD DE SEVILLA	AUTOR: CÉSAR NÚÑEZ BAYÓN	Nº PLANO: 3
TRABAJO FIN DE GRADO		

Índice de Figuras

2.1	Diferentes configuraciones de gaita en la antigüedad	4
2.2	Despiece de una gaita gallega	5
2.3	Digitación de gaita en DO	5
2.4	Digitación de los agujeros de la gaita	6
3.1	Modelos de controlador electrónico AKAI	8
3.2	Modelo WX5 de Yamaha	8
3.3	Modelo Roland Aerophone Pro AE-30	8
3.4	José Ángel Hevia utilizando una gaita electrónica	9
3.5	Punteiro electrónico de Represas	10
3.6	Modelos de Hevia Electronic Bagpipes	10
3.7	Gaita electrónica Mastergaita	11
4.1	Microcontrolador MSP430G2	13
4.2	Microcontrolador Teensy 4.1	14
4.3	Mapa de pines del Arduino UNO	14
4.4	Indicador del Arduino IDE de cantidad de memoria utilizada	15
4.5	Funcionamiento de un sensor capacitivo casero	17
4.6	Sensor TTP223	17
4.7	Realización de sensores capacitivos mediante librería CapSense	18
4.8	módulo MPR121 Breakout V12	19
4.9	sensor de presión absoluta MPX2010GP	21
4.10	sensor de presión absoluta HX710B	21
4.11	Pulsadores	21
4.12	Encoder digital	22
4.13	Pantalla LCD	22
4.14	Conector MIDI hembra	23
4.15	Jumper wire cables	23
4.16	Elementos de conexión utilizados	23
4.17	Impresora 3d Anycubic Kobra 2	24
5.1	Interfaz del Arduino IDE	25
5.2	Formato de un mensaje MIDI	26
5.3	Formato de un mensaje MIDI más detallado	27
5.4	codificación de notas MIDI	27
5.5	Mensajes de control MIDI	28
5.6	Interfaz del software LoopMIDI	29

5.7	Interfaz del software Hairless MIDI<->Serial	30
5.8	Logo de FL Studio	31
5.9	Interfaz de FL Studio	31
5.10	Interfaz del plugin SIM-BP	32
5.11	Interfaz del sintetizador Universal Piper. Pantalla Inicial.	33
5.12	Interfaz del sintetizador Universal Piper. Pantalla Harmony.	34
5.13	Interfaz del sintetizador Universal Piper. Pantalla Effects.	34
5.14	Interfaz del sintetizador Universal Piper. Pantalla MIDI.	35
5.15	Interfaz del sintetizador Universal Piper. Pantalla de la consola MIDI.	35
5.16	Interfaz del sintetizador Universal Piper. Pantalla Keyboard.	36
5.17	Interfaz del sintetizador Universal Piper. Pantalla Hardware.	36
5.18	Logo del software FreeCAD	37
5.19	Interfaz del software FreeCAD	37
5.20	Logo del software PrusaSlicer	38
5.21	Interfaz del software PrusaSlicer	38
5.22	Interfaz de Fritzing	39
5.23	Interfaz de LibreCAD	39
5.24	Tabla de caracteres en la CGROM del display LCD	43
6.1	Imágenes del montaje inicial	46
6.2	Montaje de prueba del sensor de presión	47
6.3	Gráfica de las lecturas de presión del sensor	47
6.4	Primera iteración del soprete	48
6.5	Tabla de valores de S1 y S2 del encoder digital	51
7.1	Vista inferior de la PCB con los pulsadores	56
7.2	Vista superior de la PCB con los pulsadores	56
7.3	Encoder rotatorio conectado	57
7.4	Pin header	57
7.5	Conexión de los sensores de presión	57
7.6	Soldadura de los cables de la pantalla	58
7.7	Conexiones con el módulo MPR121	58
7.8	Conector MIDI hembra sobre la placa PCB	58
7.9	Soldaduras sobre la placa PCB	59
7.10	Vistas de la primera versión del punteiro	59
7.11	Sketch base del punteiro	60
7.12	Vistas de la versión final del punteiro	60
7.13	Tuerca de sujeción	61
7.14	Vistas de las piezas de la caja	61
7.15	Disposición de los elementos dentro de la caja	62
7.16	Agujeros de salidas de la caja	62
7.17	Más elementos de la caja	62
7.18	Vistas de la pieza de soporte del punteiro	63
7.19	Piezas que soportan pantalla, botones y fol	63
7.20	Vistas de la pieza del soprete	64
7.21	Pieza de articulación del soprete	64
7.22	Caja del soprete para montaje del sensor	64
7.23	Imágenes del aspecto final del instrumento	65
7.24	Pantallas de opciones del instrumento	66

Índice de Tablas

7.1	Mensajes de Control MIDI	67
8.1	Presupuesto.	71

Índice de Códigos

6.1	Fragmento de código de programación de encoder	48
6.2	Fragmento de código de programación de encoder	48
6.3	Fragmento de código de programación de encoder, void loop	49
6.4	Rutina de interrupción del encoder	49
6.5	fragmento de la rutina de interrupción del encoder	51
6.6	Definición de variables del volumen de aire	52
6.7	Código de inflado del fol virtual	52
6.8	código de debounce de un botón	52
7.1	Variables necesarias para el instrumento final	67
7.2	Función de la pantalla	68
7.3	Programación del modo 2	69
7.4	Función del sensor de presión para que empiece a sonar las notas del punteiro	70

Bibliografía

- [1] A. López, «¿cuál es el origen de la gaita?», feb. de 2007. dirección: <https://blogs.20minutos.es/yaestaellistoquetodolosabe/el-origen-de-la-gaita/>.
- [2] V. Berzal. «Partes de la gaita en formato abecedario ¡visual e ilustrativo!» (Jun. de 2018), dirección: <https://www.apintegamarela.com/partes-de-la-gaita-gallega/>.
- [3] «Obradoiro de gaitas seivane manual de instruccions manual de instrucciones users guide»,
- [4] G. Ruiz, *EL CAMINO DEL GAITERO MÉTODO DE GAITA GALLEGA*.
- [5] «Punteiro electrónico tr». (). dirección: <https://arcoirismusic.com/gaitas-electronicas-midi/165-punteiro-electronico-tr.html>.
- [6] Harshad, «Electronic bagpipes: The historical view of electronic bagpipesórgin», *Compare Bagpipes*, mayo de 2020. dirección: <https://comparebagpipes.com/bagpipes/electronic-bagpipes/>.
- [7] «Electronic wind instruments | akai pro». (). dirección: <https://www.akaipro.com/products/ewi-series>.
- [8] «Wx5 - descripción - controladores midi - sintetizadores y herramientas de producción musical - productos - yamaha - méxico». (). dirección: https://mx.yamaha.com/es/products/music-production/midi_controllers/wx5/index.html#product-tabs.
- [9] «Roland - aerophone pro | instrumento de viento digital». (). dirección: https://www.roland.com/es-es/products/aerophone_pro/.
- [10] «History | hevia electronic bagpipes». (). dirección: <https://hvebp.wordpress.com/history/>.
- [11] N. Mirás, «La gaita electrónica pide pista», *La voz de Galicia*, nov. de 2012. dirección: https://www.lavozdegalicia.es/noticia/cultura/2012/11/17/gaita-electronica-pide-pista/0003_201211SC17P7992.htm.
- [12] «Master gaita». (). dirección: <https://mastergaita.com/>.
- [13] Xukyo, «Creando un sensor capacitivo con arduino • aranacorp», nov. de 2020. dirección: <https://www.aranacorp.com/es/creando-un-sensor-capacitivo-con-arduino/>.
- [14] D. Wang, «Fdc1004: Basics of capacitive sensing and applications», *Texas Instruments*, pág. 4, 2021. dirección: <https://www.ti.com/lit/an/snoa927a/snoa927a.pdf>.
- [15] L. K. Baxter e I. I. E. Society., «Capacitive sensors : Design and applications», pág. 302, 1997.
- [16] «Mpr121.pdf - google drive». (). dirección: https://drive.google.com/file/d/1zjkVXO_InvfoBlzrngHb55_4aNgSTLFr/view.

- [17] «Hx710b air pressure (0-40kpa) sensor module | saravana electronics». (), dirección: <https://www.alselectro.in/product-page/hx710b-air-pressure-0-40kpa-sensor-module>.
- [18] «Pressure sensor guide», dirección: <https://www.electroschematics.com/pressure-sensor-guide/>.
- [19] J. J. Purdum y J. J. Purdum, *Beginning C for Arduino : learn C programming for the Arduino*, Second, ISBN: 9781484209400. dirección: <https://www.worldcat.org/oclc/912875060>.
- [20] T. Erichsen. «Loopmidi | tobias erichsen». (), dirección: <https://www.tobias-erichsen.de/software/loopmidi.html>.
- [21] A. M., «Prusaslicer, el slicer gratuito y de código abierto para impresoras 3d fdm y sla - 3dnatives», *3Dnatives*, nov. de 2023. dirección: <https://www.3dnatives.com/es/prusaslicer-slicer-codigo-abierto-impresion-120520222/#>.
- [22] «The virtual bagpipe lab' - universal piper». (), dirección: <https://www.universal-piper.com/en/the-virtual-bagpipe-lab/>.
- [23] «Freecad: Tu propio modelador paramétrico 3d». (), dirección: https://www.freecad.org/index.php?lang=es_ES.
- [24] «Prusaslicer | impresoras 3d original prusa vendidas directamente por josef prusa». (), dirección: https://www.prusa3d.com/es/pagina/prusaslicer_424/.
- [25] A. Swift, «A brief introduction to midi», dirección: https://web.archive.org/web/20120830211425/http://www.doc.ic.ac.uk/~nd/surprise_97/journal/vol1/aps2/.
- [26] «Librecad - free open source 2d cad». (), dirección: <https://librecad.org/>.